Topological Data Analysis를 활용한 음악 구조 분석 및 위상 구조 보존 기반 AI 작곡 파이프라인

저자: 김민주 (POSTECH)
지도: 정재훈
작성일: 2026
키워드: Topological Data Analysis, Persistent Homology, Tonnetz, Discrete Fourier Transform, Overlap Matrix, Music Generation, Vietoris-Rips Complex, Jensen-Shannon Divergence, Ryuichi Sakamoto, Symbolic Music Generation

---

초록 (Abstract)

본 연구는 사카모토 류이치의 2009년 앨범 out of noise 수록곡 "hibari"를 대상으로, 음악의 구조를 **위상수학적으로 분석**하고 그 위상 구조를 **보존하면서 새로운 음악을 생성**하는 파이프라인을 제안한다. 전체 과정은 네 단계로 구성된다. (1) MIDI 전처리: 두 악기를 분리하고 8분음표 단위로 양자화. (2) Persistent Homology: 네 가지 거리 함수(frequency, Tonnetz, voice leading, DFT)로 note 간 거리 행렬을 구성한 뒤 Vietoris-Rips 복합체의 H_1 cycle을 추출. (3) 중첩행렬 구축: cycle의 시간별 활성화를 이진 또는 연속값 행렬로 기록. (4) 음악 생성: (3)에서 구한 중첩행렬을 seed로 사용하여 확률적 샘플링 기반의 Algorithm 1과 FC / LSTM / Transformer 신경망 기반의 Algorithm 2 두 방식을 제공.

$N=20$회 통계적 반복을 통한 정량 검증에서, **Algorithm 1**(확률적 샘플링) 기반으로 DFT 거리 함수가 frequency 거리 함수 대비 pitch Jensen-Shannon divergence를 0.0344 ± 0.0023에서 0.0213 ± 0.0021로 **약** 38.2% **감소**시켰으며, 이는 Welch's t-test $p < 10^{-20}$의 극도로 유의한 개선이다. 이후 DFT 기반 continuous OM에서 $\alpha = 0.25$ ($K=14$) 조건의 per-cycle τ_c 재탐색(α-grid 재탐색)을 통해 Algorithm 1 신규 최저 0.00902 ± 0.00170 ($N=20$)를 달성했다. 이는 $\alpha = 0.5$ 기준 Phase 1 (JS 0.01489) 대비 -39.4% 개선이며 (Welch $p = 1.44 \times 10^{-15}$), $\alpha = 0.25$가 §5.7 binary OM과 §5.8.1 per-cycle τ_c 양쪽 모두에서 최적임이 이중으로 확인되었다. **Algorithm 2**(DL 기반 생성)에서는 동일 DFT 조건의 연속값 중첩행렬을 입력으로 하는 FC 신경망이 0.00035 ± 0.00015 ($N=10$)를 달성하였고, Transformer 대비 Welch $p = 1.66 \times 10^{-4}$로 통계적으로 유의한 우위를 보였다. 이 두 최저값은 이론적 최댓값 $\log 2 \approx 0.693$의 각각 약 1.30% (Algo1)와 0.05% (Algo2)에 해당한다.

본 연구의 intra / inter / simul 세 갈래 가중치 분리 설계는 hibari의 두 악기 구조 — inst 1은 쉼 없이 연속 연주, inst 2는 모듈마다 규칙적 쉼을 두며 겹쳐 배치 — 를 수학적 구조에 반영한 것이며, 두 악기의 활성/쉼 패턴 관측 (inst 1 쉼 0개, inst 2 쉼 64개) 이 이 설계를 경험적으로 정당화한다. 본 논문은 수학적 정의부터 통계 실험, 시각자료, 향후 연구 방향까지를 하나의 일관된 흐름으로 정리한다.

---

1. 서론 — 연구 배경과 동기

1.1 연구 질문

음악은 시간 위에 흐르는 소리들의 집합이지만, 그 "구조"는 단순한 시간 순서만으로 포착되지 않는다. 같은 **동기(musical motive: 선율이나 리듬의 최소 반복 단위)**가 여러 번 반복되고, 서로 다른 선율이 같은 화성 기반 위에서 엮이며, 전혀 관계없어 보이는 두 음이 같은 조성 체계 안에서 등가적 역할을 한다. 이러한 층위의 구조를 수학적으로 포착하려면 "어떤 두 대상이 같다(혹은 가깝다)"를 정의하는 **거리 함수**와, 그로부터 파생되는 **위상 구조**를 다루는 도구가 필요하다.

본 연구는 다음의 세 가지 질문에서 출발한다.

1. **음악의 위상 구조는 어떻게 수학적으로 정의되는가?** 한 곡의 note들 사이에 거리 함수를 두고 Vietoris-Rips 복합체를 구성한 뒤, 거리 임계값 ε을 변화시키며 H_1 persistence를 추적하면 그 결과 나오는 cycle들은 음악적으로 어떤 의미를 가지는가?
2. **이 위상 구조를 "보존한 채" 새로운 음악을 생성할 수 있는가?** 보존의 기준은 무엇이며, 보존 정도를 어떻게 정량적으로 측정하는가?

3. **거리 함수의 선택이 실제로 생성 품질에 유의미한 영향을 주는가?** 단순 빈도 기반 거리 대신 음악 이론적 거리(Tonnetz, voice leading, DFT)를 사용하면 얼마나 나은가?

4. **위상 구조를 보존한 음악이 실제로 아름답게 들리는가?** 수학적으로 유사한 위상 구조를 가지도록 생성된 음악이 청각적으로도 원곡의 미학적 인상을 전달하는가? 본 보고서 말미에 첨부된 QR코드를 통해 생성된 음악을 직접 감상할 수 있다.

1.2 연구 대상 — 왜 hibari인가

본 연구의 대상곡은 사카모토 류이치의 out of noise (2009) 수록곡 "hibari" 이다. 이 곡을 선택한 이유는 다음과 같다.

선행연구의 확장에 적합. 단선율의 국악에 TDA를 적용한 정재훈 교수의 선행연구(정재훈 외, 2024)를 화성음악으로 확장함에 있어, hibari는 복잡성을 내포하면서도 규칙적인 모듈 구조로 일정한 패턴이 있어 모델링이 용이하였다.

미학적 특수성. out of noise 앨범은 "소음과 음악의 경계"를 탐구하는 실험적 작업이며, hibari는 전통적 선율 진행이 아니라 음들의 공간적 배치에 가까운 방식으로 구성된다. 이 특성은 본 연구의 실험 결과 (§4.5)에서 DL 모델 선택과 직접적으로 공명한다.

---

2. 수학적 배경

본 절에서는 본 연구의 파이프라인을 이해하기 위해 필요한 수학적 도구들을 정의하고, 각 도구가 음악 구조 분석에서 어떻게 사용되는지를 서술한다. TDA의 기본 개념에 대한 상호작용적 입문 자료로는 POSTECH MINDS 그룹의 튜토리얼(https://github.com/postech-minds/postech-minds/blob/main/tutorials/%5BGTDA_TUTO%5D01-Introduction_to_TDA.ipynb)을 참고할 수 있다.

2.1 Vietoris-Rips Complex

정의 2.1. 거리 공간 (X, d)와 양의 실수 $\varepsilon > 0$이 주어졌을 때, **Vietoris-Rips complex** $\mathrm{VR}_\varepsilon(X)$는 다음과 같이 정의되는 복합체(simplicial complex)이다:

$$\mathrm{VR}_\varepsilon(X) = \left\{\sigma \subseteq X \mid \forall x_i, x_j \in \sigma,\ d(x_i, x_j) \leq \varepsilon\right\}$$

즉, 점 집합 X의 부분집합 σ에 속한 **모든 점 쌍 사이의 거리가** ε **이하**이면 σ를 심플렉스(simplex)로 포함시킨다.

Filtration 구조와 포함관계(nested sequence). ε 값을 0부터 연속적으로 키우면, 점 집합 X 자체는 변하지 않은 채 **새로운 심플렉스만 점차 추가된다.** $\varepsilon = 0$일 때 $\mathrm{VR}_0(X)$는 각 점만을 0-simplex로 포함하는 이산적인 점 집합(discrete set)이다 — 아직 어떤 edge도 없으므로 이것은 X 그 자체와 같다. ε이 커지면서 두 점 사이 거리가 ε 임계를 처음 넘는 순간에 1-simplex(edge)가 추가되고, 세 점이 모두 ε 이내가 되면 2-simplex(삼각형)가 추가된다. 즉 $\varepsilon_1 < \varepsilon_2$이면 $\mathrm{VR}_{\varepsilon_1}(X)$의 모든 심플렉스가 $\mathrm{VR}_{\varepsilon_2}(X)$에도 그대로 들어 있으며, 새 심플렉스가 추가될 뿐이다. 따라서 다음의 포함관계는 항상 성립한다:

$$\mathrm{VR}_0(X) \subseteq \mathrm{VR}_{\varepsilon_1}(X) \subseteq \mathrm{VR}_{\varepsilon_2}(X) \subseteq \cdots \subseteq \mathrm{VR}_{\varepsilon_n}(X)$$

여기서 $0 < \varepsilon_1 < \varepsilon_2 < \cdots$는 새로운 심플렉스가 추가되어 복합체의 **위상 구조**가 변하는 임계값들이다. 점이 추가되거나 사라지는 것이 아니라, 같은 점들 사이에 새로운 연결(edge, 삼각형 등)이 생기면서 cycle이나 void가 형성되거나 채워지는 변화이다.

표기 편의를 위해 $K_i := \mathrm{VR}_{\varepsilon_i}(X)$로 두면:

$$K_0 \subseteq K_1 \subseteq K_2 \subseteq \cdots \subseteq K_n$$

이를 **filtration**이라 부르며, 변화가 일어나는 임계값 ε_i들이 곧 위상의 birth/death 시점이 된다.

본 연구에서의 사용: $X = \{n_1, n_2, \ldots, n_{23}\}$은 hibari에 등장하는 23개의 고유 note이며, $d(n_i, n_j)$는 두 note 간 거리이다.

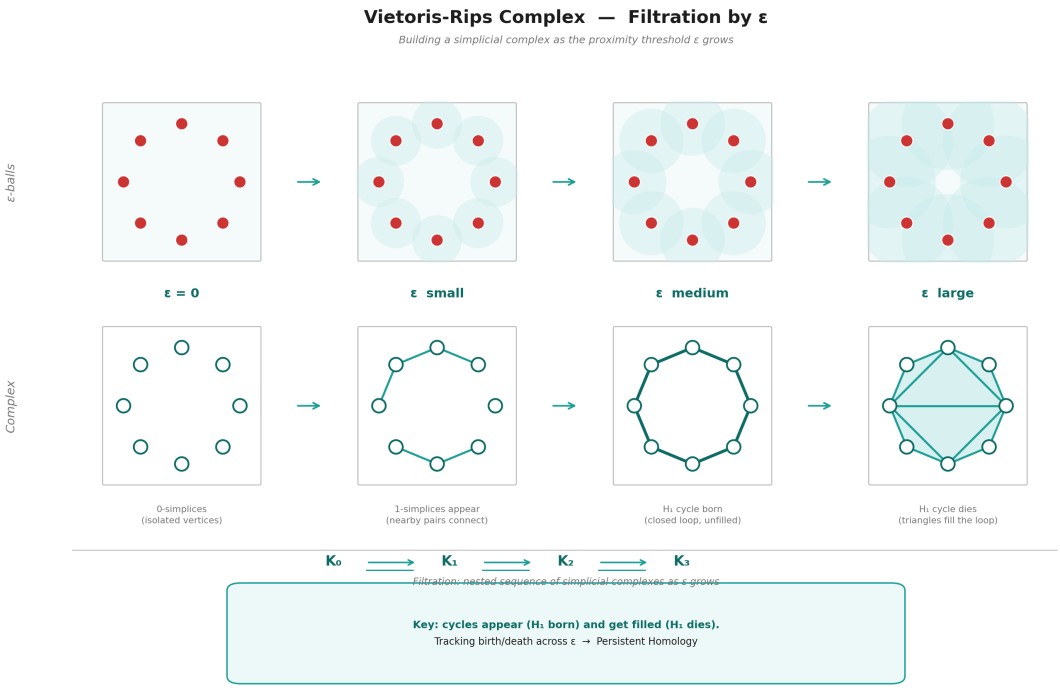


그림 2.1. Vietoris-Rips 필터레이션 — ε이 커지면서 볼이 겹치고 심플렉스가 추가되는 과정. 왼쪽부터 점 집합 → 엣지 생성 → 삼각형(2-단체) 채움 순으로 진행된다.

2.2 Simplicial Homology

정의 2.2. Simplex complex K에 대해 n차 호몰로지 군(homology group) $H_n(K)$는 K 안에 존재하는 n차원 "구멍"의 대수적 표현이다. 직관적으로:

$H_0(K)$: 연결 성분(connected components)의 수

$H_1(K)$: 1차원 cycle의 수 (닫힌 고리 모양으로 둘러싸인 영역)

$H_2(K)$: 2차원 빈 공간(void)의 수 (3차원 공동을 둘러싼 표면)

$\mathrm{rank}(H_n(K))$ = **Betti number** β_n은 서로 독립적인 n차원 구멍의 개수를 나타낸다. 엄밀히 말하면 $H_n(K)$의 원소는 개별 구멍이 아니라 n-cycle들의 **동치류**(boundary로 차이나는 두 cycle을 같다고 보는 관계)이며, 연산은 cycle의 chain 합이다. 이때 $\mathrm{rank}(H_n(K)) = \beta_n$이 '독립적인 구멍의 개수'를 정의한다. 예컨대 $\beta_1 = 3$이면 독립적인 1차원 cycle이 3개 있다는 뜻이다. 1차원 cycle은 최소 3개의 점이 필요하고(삼각형 모양의 폐곡선), 그 내부를 채우는 2-simplex가 없어야 cycle로 인식된다.

TDA의 핵심 강점은 **오직 점 사이의 쌍별(pairwise) 거리 정보만으로 위상 구조를 추출**할 수 있다는 것이다 (Carlsson, 2009). 유클리드 좌표가 필요 없으며, Tonnetz 격자 위 최단 경로 같은 추상적 거리에서도 VR complex가 잘 정의된다. Heo, Choi, Jung (2025)은 이러한 그래프 기반 거리를 **path-representable distance**로 일반화하고, 1차원 persistence barcode 사이에 단사함수가 존재함을 증명하여 Tonnetz 거리의 이론적 정당성을 뒷받침하였다.

Boundary 연산자와 호몰로지 — 선형대수적 계산 예시. H_n이 어떻게 계산되는지를 간단한 예시로 보인다. 점 4개 $\{a, b, c, d\}$와 edge $\{ab, bc, cd, da, ac\}$, 삼각형 $\{abc\}$로 이루어진 복합체 K를 생각하자.

boundary 연산자 ∂_1 (edge → vertex)과 ∂_2 (삼각형 → edge)를 $\mathbb{F}_2$ (mod 2) 위에서 행렬로 표현하면:

```
∂₁ (edge → vertex):
      ab bc cd da ac
a [   1  0  0  1  1 ]
b [   1  1  0  0  1 ]
c [   0  1  1  0  0 ]
d [   0  0  1  1  0 ]
```

```
∂₂ (triangle → edge):
      abc
ab [   1 ]
bc [   1 ]
cd [   0 ]
da [   0 ]
ac [   1 ]
```

$H_1(K) = \ker(\partial_1)/\mathrm{im}(\partial_2)$이다. $\ker(\partial_1)$은 "닫힌 edge 체인들"의 공간이고, $\mathrm{im}(\partial_2)$는 "삼각형의 경계인 edge 체인들"의 공간이다. 이 예시에서 $\ker(\partial_1)$의 차원은 2 (두 개의 독립적인 닫힌 고리: $ab + bc + ca$와 $ac + cd + da$), $\mathrm{im}(\partial_2)$의 차원은 1 (abc의 경계 $= ab + bc + ca$). 따라서 $\beta_1 = 2 - 1 = 1$, 즉 독립적인 cycle이 1개이다. 직관적으로, 삼각형 abc가 한 cycle을 "채워서" 없앴고, 남은 사각형 $a - c - d - a$에 해당하는 cycle 1개가 살아남는다.

본 연구에서의 사용: 본 연구는 주로 H_1 (1차 호몰로지)을 다룬다. 이는 음악 네트워크에서 서로 가까운 note들이 만드는 닫힌 cycle, 즉 순환적으로 연결된 note 그룹을 포착한다. 발견된 각 cycle은 곡의 구조적 반복 단위로 해석된다.

2.3 Persistent Homology

Filtration $K_0 \subseteq K_1 \subseteq \cdots \subseteq K_n$에서, 각 단계마다 H_1의 cycle 구성이 달라진다. **Persistent homology**는 이 과정에서 각 cycle이 어느 ε_i에서 처음 나타나고(**birth**) 어느 ε_j에서 사라지는지(**death**)를 추적한다.

Birth와 death의 음악적 의미:

- **Birth** b: 거리 임계값 ε가 충분히 커져서 새로운 cycle이 형성되는 순간. 음악적으로는 "이 거리 척도에서 처음으로 닫힌 반복 구조가 발견되는 시점".
- **Death** d: 더 큰 ε에서 그 cycle이 다른 cycle들의 합(정확히는 boundary)으로 표현될 수 있게 되어, 호몰로지 군 안에서 더 이상 독립적인 generator가 아니게 되는 순간. 음악적으로는 "거리 척도가 너무 느슨해져서 이 반복 구조가 다른 구조에 흡수되는 시점".

각 cycle은 (b, d) 쌍과, 그 cycle을 구성하는 vertex/edge 집합인 **cycle representative**가 함께 기록된다. 본 연구에서는 birth-death 쌍은 cycle의 "수명"을 측정하는 데, cycle representative는 어떤 note들이 그 cycle을 이루는지 식별하는 데 사용한다. 이 정보를 모은 것이 곡의 **위상적 지문(topological signature)**이다.

Persistence: $\mathrm{pers}(\mathrm{cycle}) = d - b$. (death가 birth보다 항상 크므로 양수.) 큰 persistence를 갖는 cycle은 다양한 거리 척도에서 살아남으므로 **위상적으로 안정한 구조**이며, 작은 persistence는 일시적이거나 노이즈에 가까운 구조이다.

알고리즘적 측면: 본 연구의 대부분의 실험에서는 선행연구(정재훈 외, 2024)의 **pHcol algorithm** 순수 Python 구현을 사용하였다. 이 구현은 cycle representative까지 함께 추출해주므로 본 연구의 후속 단계(중첩행렬 구축)에 그대로 활용할 수 있다. 별도로 계산 속도가 중요한 일부 단계에서는 C++ 기반 **Ripser** (Bauer, 2021) 구현을 보조적으로 활용하였으며, 두 구현이 동일한 birth-death 결과를 내는지 검증하였다.

본 연구에서의 사용: 거리 행렬 $D \in \mathbb{R}^{23 \times 23}$로부터 Vietoris-Rips filtration을 구성하고, 각 rate parameter r (가중치 비율, 후술)에서의 H_1 persistence를 계산한다. 발견된 모든 (b, d) 쌍과 cycle representative가 함께 cycle 집합을 정의하며, 이 cycle들이 다음 절의 중첩행렬 구축에 사용된다.

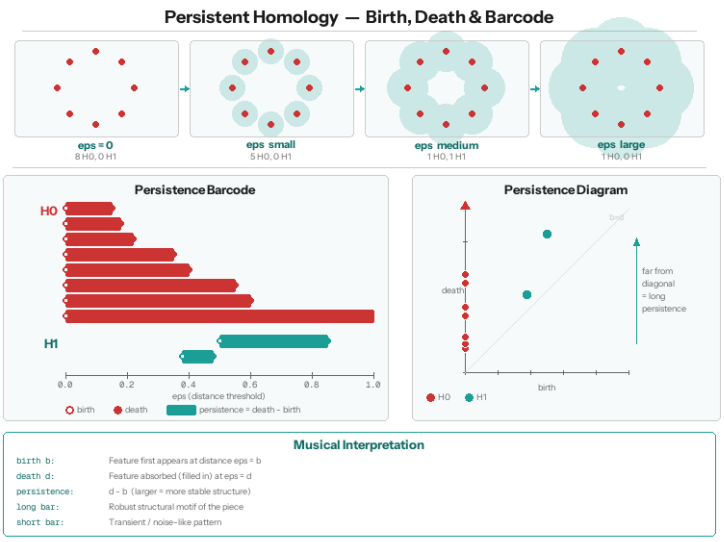


그림 2.3. Persistent Homology — filtration 단계별 barcode 구성과 persistence diagram. 각 막대의 길이(persistence)가 위상 구조의 중요도를 나타낸다.

2.4 Tonnetz와 음악적 거리 함수

정의 2.4. Tonnetz(또는 tone-network)는 pitch class 집합 $\mathbb{Z}/12\mathbb{Z}$를 평면 격자에 배치한 구조이다. 여기서 **pitch class**는 옥타브 차이를 무시한 음의 동치류(equivalence class)로, 예컨대 C4 (가운데 도), C5 (한 옥타브 위 도), C3 등은 모두 같은 pitch class "C"에 속한다. 12음 평균율(12-TET)에서는 한 옥타브 안에 12개의 pitch class가 있으며, 이를 정수 $\{0, 1, 2, \ldots, 11\}$에 대응시켜 $\mathbb{Z}/12\mathbb{Z}$로 표기한다 (0=C, 1=C , 2=D, ..., 11=B). 두 pitch class가 격자 위에서 가까운 것은 음악 이론적으로 어울리는 음(consonant)임을 의미한다.

Tonnetz의 격자 구조. pitch class $p \in \mathbb{Z}/12$를 좌표 (x, y)에 배치하되, 다음 관계를 만족시킨다:

가로 이동 (+1 in x): 완전5도 (perfect fifth, +7 semitones)

대각선 이동 (+1 in y): 장3도 (major third, +4 semitones)

이렇게 배치하면 자연스럽게 단3도(+3 semitones) 관계도 다른 대각선 방향으로 형성되어 삼각형 격자가 만들어진다. 그림 2.4는 hibari의 C장조 음역에 해당하는 일부분을 보여준다.

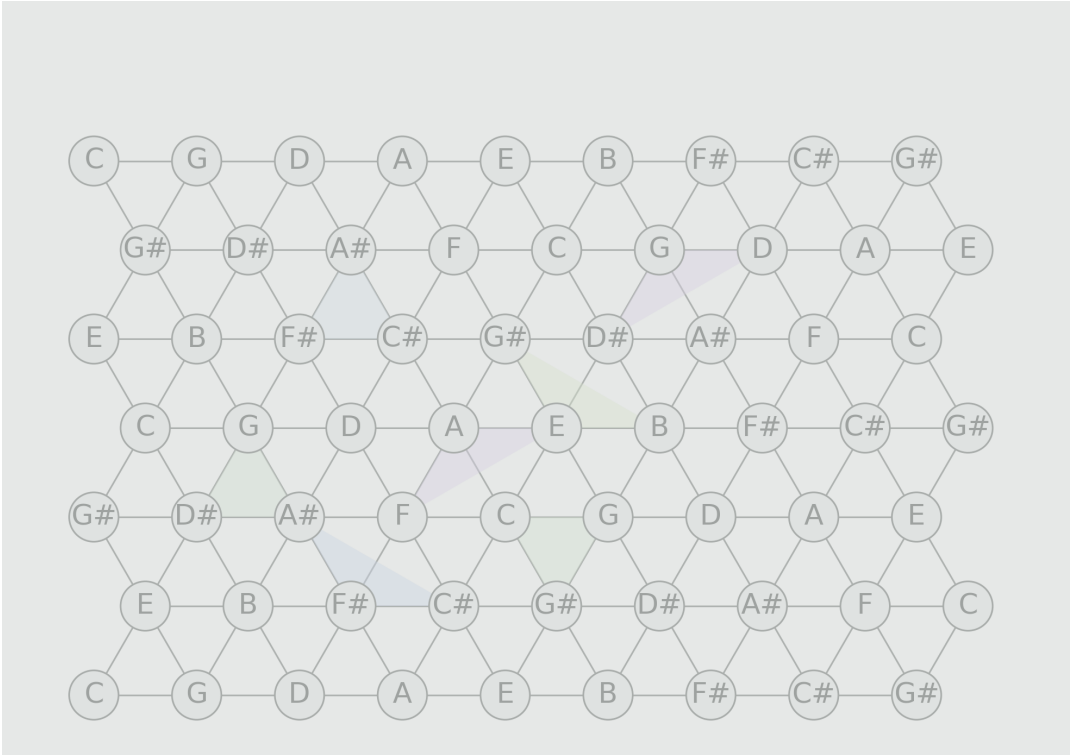


그림 2.4. Tonnetz 격자 구조. 가로 방향은 완전5도(C→G→D→A→E⋯), 대각선 방향은 장3도(C→E→G ⋯)와 단3도(C→A→F ⋯)로 이동한다. 삼각형 하나는 하나의 장3화음(major triad) 또는 단3화음(minor triad)에 대응된다.

리만 좌표. pitch class p의 Tonnetz 격자 좌표 (x_p, y_p)는 다음 선형 관계를 만족하도록 정의된다:

$$p \equiv 4\,x_p + 7\,y_p \pmod{12}$$

cifkao(2015)의 시각화 구현에서는 단위 간격 u를 이용해 실제 캔버스 좌표를 다음과 같이 설정한다:

$$\tilde{x}_p = u \cdot x_p, \quad \tilde{y}_p = u\sqrt{3} \cdot y_p$$

$\sqrt{3}$ 인수는 정삼각형 격자(정삼각형 타일링)의 기하학적 요건에서 비롯된다. 이 배치에서 인접한 두 pitch class 사이의 유클리드 거리는 일정($=u$)하며, 삼각형 하나가 정확히 하나의 장3화음 또는 단3화음에 대응된다.

Tonnetz 거리. 두 pitch class p_1, p_2 사이의 Tonnetz 거리 $d_T(p_1, p_2)$는 격자 위 최단 경로 길이(즉, edge 수)로 정의된다:

$$d_T(p_1, p_2) = \min\{|x_1 - x_2| + |y_1 - y_2| \mid (x_i, y_i) \text{ represents } p_i\}$$

본 연구에서는 12개 pitch class 모두에 대해 사전 계산된 12×12 거리 테이블을 사용한다. 이 테이블에서 **최솟값은** 0 (같은 pitch class), **최댓값은** 4 (예: F와 B, 또는 C와 F 처럼 Tonnetz 격자에서 가장 먼 쌍)이다. 이 최댓값 4는 Tonnetz 격자 위에서 어떤 경로로 이동해도 4번의 이동(장3도·단3도·완전5도) 이내에 모든 pitch class에 도달할 수 있음을 의미한다. 12음 평균율의 닫힌 구조(모든 pitch class가 순환적으로 연결됨)가 이를 보장한다.

빈도 기반 거리. 본 연구의 기준 거리 d_{freq}는 두 note의 인접도(adjacency)의 역수로 정의된다. 인접도 $w(n_i, n_j)$는 곡 안에서 note n_i와 n_j가 시간적으로 연달아 등장한 횟수이다:

$$w(n_i, n_j) = \#\{\, t : n_i \text{ at time } t \text{ and } n_j \text{ at time } t+1 \,\}$$

거리는 $d_{\text{freq}}(n_i, n_j) = 1/w(n_i, n_j)$로 정의되며 (인접도가 0인 경우는 도달 불가능한 큰 값으로 처리), 자주 연달아 등장하는 음일수록 가까워진다. 이는 곡의 통계적 흐름만 반영하며 화성 관계는 직접 포착하지 못한다는 한계가 있다.

그 외의 음악적 거리 함수.

(1) Voice-leading distance (Tymoczko, 2008): 두 pitch class 사이를 이동하기 위해 거쳐야 하는 반음의 개수와 같다.

$$d_V(p_1, p_2) = |p_1 - p_2|$$

이 정의가 "두 pitch class의 반음 차이"가 되는 이유는 12음 평균율에서 인접한 두 pitch class(예: C와 C , 또는 E와 F)의 정수 표현이 정확히 1만큼 차이나며, 그 음정 차이가 1 반음(semitone)이기 때문이다. 음악 이론에서 voice leading은 한 화음에서 다른 화음으로 옮겨갈 때 각 성부가 가능한 한 적은 음정으로 이동하는 것을 미덕으로 삼으며, 이 거리는 그러한 "최소 이동" 원리를 직접 수치화한 것이다.

(2) DFT distance (Tymoczko, 2008): 각 pitch class를 12차원 벡터로 표현한 뒤, 이산 푸리에 변환(DFT)으로 다른 공간으로 옮겨서 비교한다.

pitch class를 "함수"로 보는 이유. 12음 평균율에서 한 옥타브는 12개의 동일 간격 칸으로 나뉜다. pitch class p를 "12칸짜리 원형 자(ruler) 위에서 p번째 칸이 켜져 있고 나머지는 꺼져 있는 상태"로 생각하면, 이것은 $\{0, 1, \ldots, 11\} \to \{0, 1\}$인 함수이다. 이 함수를 **indicator vector** $e_p \in \mathbb{R}^{12}$ (p번째 성분만 1, 나머지 0)로 표현한다.

L_2 **거리란.** 두 벡터 $u, v \in \mathbb{R}^n$ 사이의 L_2 거리(유클리드 거리)는 성분별 차이의 제곱합에 루트를 씌운 것으로, 일상적인 "두 점 사이의 직선 거리"와 같다:

$$\|u - v\|_2 = \sqrt{\sum_{i=1}^{n} (u_i - v_i)^2}$$

푸리에 공간(Fourier space)이란. 원래 공간에서 indicator vector를 비교하면 "이 음과 저 음이 같은지 다른지"만 알 수 있다. DFT는 이 벡터를 **주기성 성분별로 분해**하여 새로운 좌표계 — 이것을 푸리에 공간이라 부른다 — 로 옮긴다. 각 좌표축(=**푸리에 계수** $\hat{f}_k$)은 특정 주기의 패턴에 대한 반응 강도를 나타낸다. 예를 들어 $k = 3$번 계수는 "옥타브를 4등분하는 단3도 간격에 대한 반응" (증3화음과 관련), $k = 5$번 계수는 "5도권을 따른 반응" (온음계적 구조와 관련)이다.

$$d_F(p_1, p_2) = \|\hat{f}(p_1) - \hat{f}(p_2)\|_2$$

여기서 $\hat{f}(p) \in \mathbb{C}^{12}$는 indicator vector e_p에 12점 DFT를 적용한 결과이다. 이 거리의 이론적 최댓값은 $\sqrt{2}$이며, C와 F 처럼 화성적으로 가장 동떨어진 pitch class 쌍에서 달성된다. 따라서 d_F의 치역은 $[0, \sqrt{2}]$이다. 따라서 DFT 거리는 "두 pitch class가 화성적 성격(온음계성, 증3화음 대칭성 등)에서 얼마나 다른가"를 측정한다.

복합 거리(Hybrid distance). 본 연구는 빈도 기반 거리 d_{freq}와 음악적 거리 d_{music} (Tonnetz, Voice-leading, DFT 중 하나)을 선형 결합한다:

$$d_{\text{hybrid}}(n_i, n_j) = \alpha \cdot d_{\text{freq}}(n_i, n_j) + (1 - \alpha) \cdot d_{\text{music}}(n_i, n_j)$$

여기서 $\alpha \in [0,1]$은 두 거리의 비중을 조절하는 파라미터이다. §5.7의 DFT hybrid $N=20$ 반복 실험 결과, $\alpha=0.25$가 JS 최적이었다 (Tonnetz 조건의 구 실험에서는 $\alpha=0.0$이 최적이었으나, DFT 기반 재실험에서 $\alpha=0.25$가 확정됨). **최종 기본 설정은** $\alpha=0.25$이다 (`config.py MetricConfig.alpha`). 상세 결과는 §5.7 참조.

본 연구에서의 사용: 거리 함수의 선택은 발견되는 cycle 구조에 직접적으로 영향을 미친다. 빈도 기반 거리만 사용하면 곡의 통계적 특성만 반영되어 화성적·선율적 의미가 있는 구조를 포착하지 못한다. Tonnetz 거리를 도입함으로써 hibari의 C장조/A단조 화성 구조와 정합적인 cycle을 발견할 수 있었다.

선행연구와의 관계. Tonnetz 표현을 딥러닝 음악 생성에 직접 적용한 선행연구로 Chuan & Herremans(2018)이 있다. 이들은 12×24 픽셀 Tonnetz 이미지를 CNN autoencoder로 압축한 뒤 LSTM으로 다음 박자를 예측하는 파이프라인을 제안했다. 본 연구는 Tonnetz를 이미지가 아닌 **거리 함수**로 활용한다는 점에서 차별화되며, 동일한 음악 이론적 기반 위에서 위상수학적 분석(Persistent Homology)과 결합한다.

주석 — metric 공리와 이론적 정당성

실험적 검증 결과(`scripts/verify_metric_axioms.py`, `docs/step3_data/metric_axiom_verification.json`)에 따르면, 네 거리 함수의 metric 공리 준수 현황은 다음과 같다.

거리 함수	nonneg	identity	symmetry	triangle	공리 상태
frequency		(23건)		(1327건)	non-metric
Tonnetz					metric
voice_leading					metric
dft		(66건)			pseudometric

위반 원인. frequency의 identity 위반은 시퀀스 전이 빈도 기반 정의에서 비롯된다 — 자기 자신으로의 전이 빈도가 낮으면 $d(x,x)>0$이 가능하다. triangle 위반 1327건은 이 identity 위반의 귀결이다. dft의 identity 위반은 magnitude-only DFT에서 서로 다른 pitch-class 벡터가 동일한 magnitude spectrum을 가질 수 있기 때문이며 (예: transposition/inversion 관련 chord 쌍), 이는 §5.9 transposition-invariance 정리의 의도된 설계 귀결이다.

고전 stability 정리의 한계. Cohen-Steiner-Edelsbrunner-Harer (2007) stability 정리는 두 metric d_1, d_2에 대해 $\|d_1 - d_2\|_\infty \le \varepsilon$이면 $d_B(\mathrm{bcd}(d_1), \mathrm{bcd}(d_2)) \le \varepsilon$을 보장한다. 그러나 이 보장은 삼각 부등식을 포함한 metric 공리 전체를 전제하므로, frequency와 dft에 대해 직접 적용할 수 없다.

Heo-Choi-Jung (2025)의 대안 보장. Heo, Choi, Jung (2025)은 완전그래프 위 edge-weighted distance에 대해 path-representable distance 프레임워크를 제시한다. 본 연구의 구현은 note 집합에 대한 거리 행렬 D를 완전그래프 K_n의 edge weight로 부여하는 방식이므로, path-representable + cost-dominated 조건이 자명하게 성립한다 (직접 간선만 사용하므로 우회 경로 없음). 동 논문 Theorem 3.4에 따르면, cost-dominated path-representable 거리 $d_i \le d_j$ 관계 하에서 $\mathrm{bcd}_1(d_i) \to \mathrm{bcd}_1(d_j)$의 **order-preserving injection**이 존재한다. 이는 frequency/dft가 엄밀한 metric이 아니어도, 1-cycle 구조의 탐지와 비교가 이론적으로 정당화됨을 의미한다. 단, $k \ge 2$ persistence에 대해서는 이 보장이 성립하지 않으며 (동 논문 Theorem 5.1의 반례), 본 연구가 H_1에 집중하는 사실과 이 이론이 정합적이다.

2.5 활성화 행렬과 중첩행렬

본 연구에서는 곡의 시간축 위에서 cycle 구조가 어떻게 전개되는지를 두 단계의 행렬로 표현한다. 첫 단계는 **활성화 행렬(activation matrix)**, 두 번째 단계는 그것을 가공한 **중첩행렬(overlap matrix, OM)**이다.

정의 2.5 (활성화 행렬). 음악의 시간축 길이를 T, 발견된 cycle의 수를 C라 하자. 활성화 행렬 $A \in \{0,1\}^{T \times C}$는 raw 활성 정보를 담는다:

시점 t에서 cycle c를 구성하는 note 중 **적어도 하나가 원곡에서 연주되고 있으면** $A[t,c]=1$, 아니면 $A[t,c]=0$이다. 형식적으로:

$$A[t,c] = \mathbb{1}[\,\exists\, n \in V(c) \text{ such that } n \text{ is played at time } t\,]$$

여기서 $V(c)$는 cycle c의 vertex(=note) 집합이며, $\mathbb{1}[\cdot]$은 indicator function이다. 활성화 행렬은 산발적인 단일 시점 활성화까지 모두 포함하므로 노이즈가 많다.

정의 2.6 (OM). OM $O \in \{0,1\}^{T \times C}$는 활성화 행렬에서 **연속적이고 충분히 긴 활성 구간만 남긴 것**이다.

$$O[t,c] = \mathbb{1}[\, t \in R(c)\,], \qquad R(c) = \bigcup_i [s_i,\ s_i + L_i]$$

여기서 $R(c)$는 cycle c의 "지속 활성 구간(sustained intervals)"의 합집합이며, 각 구간 $[s_i, s_i + L_i]$는 활성화 행렬 $A[\cdot, c]$에서 길이가 임계값 scale_c 이상인 연속 1의 구간이다. $\mathrm{scale}_c \in \mathbb{Z}_{>0}$은 cycle c의 **최소 활성 지속 길이(minimum sustained length)**이며, cycle마다 ON 비율 $\rho(c) = |R(c)|/T$가 목표치 $\rho^* = 0.35$에 근접하도록 동적으로 조정된다 (구간이 너무 많으면 scale을 키우고, 너무 적으면 줄인다).

활성화 행렬과 OM의 차이.

$A[t,c]$: 시점 t에 cycle c의 note가 단 한 번이라도 울리면 1. **순간적 활성을 모두 잡음.**

$O[t,c]$: cycle c의 활성이 일정 시간 이상 **지속되는 구간**에서만 1. 산발적 노이즈 제거됨.

예를 들어 $\mathrm{scale}_c = 3$일 때 (3 시점 이상 지속된 활성만 인정), 다음과 같은 cycle c의 한 행을 생각해보자.

```
    :  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
A[·,c]: 0  1  1  0  1  1  1  1  0  0  1  0  1  1  1
O[·,c]: 0  0  0  0  1  1  1  1  0  0  0  0  1  1  1
```

활성화 행렬 A는 시점 2~3, 5~8, 11, 13~15에서 모두 활성화되어 있다. OM O는 그중 길이가 $\mathrm{scale}_c = 3$ 이상인 두 구간(시점 5~8과 13~15)만 1로 남기고, 길이가 짧은 시점 2~3과 단발성 시점 11은 0으로 처리한다. 본 연구에서 OM을 음악 생성의 seed로 사용하는 이유는, 잠시 스쳐가는 활성보다 일정 시간 유지되는 cycle만이 곡의 구조적 단위로 의미 있다고 보기 때문이다.

구축 과정:

1. **활성화 행렬 계산**: 위 정의 2.5에 따라 $A \in \{0,1\}^{T \times C}$를 구한다.
2. **연속 활성 구간 추출**: 각 cycle c에 대해 길이가 scale_c 이상인 연속 1 구간을 모두 찾는다.
3. **Scale 동적 조정**: cycle마다 ON 비율 $\rho(c) = |R(c)|/T$가 목표치 $\rho^* = 0.35$에 근접하도록 scale_c를 조정한다 (구간이 너무 많으면 scale을 키우고, 너무 적으면 줄인다).

목표 ON 비율의 근거. $\rho^* = 0.35$는 본 연구에서 새로 결정한 것이 아니라 선행연구(정재훈 외, 2024)에서 사용된 휴리스틱 값을 계승한 것이다. 직관적으로 한 cycle이 곡 전체의 약 1/3 정도 활성화되면 "그 cycle이 곡의 구조적 모티프로서 충분히 자주 등장하면서도, 모든 시점을 점유하지 않아 다른 cycle과 구분된다"는 균형을 만든다. 이 값의 최적성은 본 연구에서 정량적으로 검증하지 않았으며, 향후 곡 또는 데이터에 따라 적응적으로 조정 가능한 파라미터로 일반화할 예정이다 (예: ON 비율 자체를 최적화 대상으로 설정).

연속값 확장. 본 연구에서는 이진 OM 외에, cycle의 활성 정도를 [0,1] 사이의 실수값으로 표현하는 연속값 버전도 도입하였다:

$$O_{\mathrm{cont}}[t,c] = \frac{\sum_{n \in V(c)} w(n) \cdot \mathbb{1}[\, n \text{ is played at time } t\,]}{\sum_{n \in V(c)} w(n)}$$

여기서 $V(c)$는 cycle c의 vertex 집합, $w(n) = 1/N_{\mathrm{cyc}}(n)$은 note n의 **희귀도 가중치**이며 $N_{\mathrm{cyc}}(n)$은 note n이 등장하는 cycle의 개수이다. 적은 cycle에만 등장하는 희귀한 note가 활성화되면 더 큰 가중치를 받는다.

음악적 의미: OM은 곡의 **위상적 뼈대(topological skeleton)**를 시각화한 것이다. 시간이 흐름에 따라 어떤 반복 구조가 켜지고 꺼지는지를 나타내며, 이것이 음악 생성의 seed 역할을 한다.

2.6 Jensen-Shannon Divergence — 생성 품질의 핵심 지표

JS divergence는 두 확률 분포가 얼마나 다른지를 대칭적으로 측정하는 지표이다. KL divergence(D_{KL}: "참 분포 P를 Q로 잘못 알 때의 정보 손실")를 대칭화한 것으로, $D_{\mathrm{JS}}(P\|Q)=\frac{1}{2}D_{\mathrm{KL}}(P\|M)+\frac{1}{2}D_{\mathrm{KL}}(Q\|M)$ ($M=(P+Q)/2$). 값의 범위는 $[0,\log 2]$이며, 0이면 두 분포가 동일하다.

본 연구에서 비교하는 두 가지 분포:

1. **Pitch 빈도 분포** — "어떤 음들이 얼마나 자주 쓰였는가" (시간 순서 무시)
2. **Transition 빈도 분포** — "어떤 음 다음에 어떤 음이 오는가" (시간 순서 반영)

두 지표를 함께 사용함으로써 "음을 비슷하게 쓰는가"와 "비슷한 순서로 쓰는가"를 별도로 측정할 수 있다. 본 연구의 최우수 조합에서 pitch JS divergence는 $D_{\mathrm{JS}}\approx 0.00035$으로, 이론적 최댓값($\log 2\approx 0.693$)의 약 0.05%에 해당한다.

2.7 Greedy Forward Selection

발견된 전체 cycle 집합 $\mathcal{C}$에서 원곡의 위상 구조를 가장 잘 보존하는 부분집합 $S\subseteq\mathcal{C}$를 선택해야 한다. 이를 위해 **greedy forward selection**을 사용한다: 보존도 함수 $f(S)=0.5\cdot J(S)+0.3\cdot C(S)+0.2\cdot B(S)$를 정의하고, 매 단계마다 f를 가장 크게 증가시키는 cycle을 하나씩 추가한다.

세 지표는 각각:

- **Note Pool Jaccard** $J(S)$: 선택된 cycle들이 전체 note를 얼마나 커버하는가
- **Overlap pattern correlation** $C(S)$: 시점별 활성 패턴이 원본과 얼마나 동조하는가 (Pearson 상관)
- **Betti curve similarity** $B(S)$: rate 변화에 따른 전체 위상 복잡도의 골격이 보존되는가

Note Pool Jaccard에 가장 큰 비중(0.5)을 둔 이유는 음악 생성의 직접적 입력이 cycle 구성 note이기 때문이다. 실험적으로 greedy 방법이 42개 cycle 중 15개로 90% 보존도를 달성하는 것을 확인하였다.

2.8 Multi-label Binary Cross-Entropy Loss

각 시점에서 동시에 여러 note가 활성화될 수 있으므로, 단일 클래스 예측인 categorical cross-entropy 대신 **multi-label BCE**를 사용한다. 각 note 채널마다 독립적인 binary cross-entropy를 계산하여 "note i가 활성인가?"를 개별 이진 문제로 학습한다. 모델 입력은 OM의 한 행 $O[t,:]\in\mathbb{R}^C$이고, 출력은 N차원 multi-hot vector이다.

Adaptive threshold: 추론 시 고정 임계값 0.5 대신, 원곡의 평균 ON 비율(약 15%)에 맞춰 sigmoid 출력 상위 15%를 활성으로 채택하는 동적 임계값을 사용한다. 이 임계값은 per-cycle이 아니라 **출력행렬** $P\in\mathbb{R}^{T\times N}$ **전체를 평탄화한 전역 기준**으로 계산한다(§4.4의 OM 임계값 $\tau\in\{0.3,0.5,0.7\}$ 실험과는 별개).

2.9 음악 네트워크 구축과 가중치 분리

정의 2.11. 음악 네트워크 $G=(V,E)$는 다음과 같이 정의된다:

- **Vertex set** V: 곡에 등장하는 모든 고유 (pitch, duration) 쌍. hibari의 경우 $|V|=23$.
- **Edge set** E: 두 vertex가 곡에서 인접하여 등장한 경우 연결.
- **Weight function** $w:E\to\mathbb{R}_{\geq 0}$: 인접 등장 빈도.

가중치 행렬의 분리 — hibari의 악기 배치 구조에 근거. 본 연구가 가중치를 intra / inter / simul 세 가지로 분리한 것은 hibari의 실제 구조에서 비롯된다. hibari에서 inst 1은 처음부터 끝까지 쉬지 않고 연주하는 반면, inst 2는 모듈마다 규칙적인 쉼을 두며 얹히는 방식으로 배치된다 (그림 2.9에서 시각적으로 확인). 즉 두 악기는 (1) 각각 독립적인 시간적 흐름을 갖고, (2) 서로 다른 시간 위상(phase)에서 상호작용한다. 본 연구는 이 구조를 수학적 가중치에 반영하여, intra weight는 "한 악기 내부의 시간 방향 흐름", inter weight는 "악기 1의 어떤 타건 다음 lag ℓ만큼 후에 악기 2의 어떤 타건이 오는가", simul weight는 "같은 시점에서의 즉시적 화음 결합"을 각각 독립적으로 표현한다.

rate parameter r_t의 의미. r_t는 timeflow weight에서 intra와 inter의 비중을 조절한다.

- $r_t=0$: $W=W_{\mathrm{intra}}$만 사용. 각 악기의 선율적 흐름만 반영.

$r_t = 1$: intra와 inter를 동등하게 결합. 선율과 상호작용을 균형 있게 반영.

$r_t > 1$: inter의 비중이 intra보다 커짐. 악기 간 상호작용이 지배적인 구조를 탐색.

가중치 행렬의 분리 (본 연구의 핵심 설계): 본 연구는 가중치를 다음과 같이 세 가지로 분리한다:

1. **Intra weight** W_{intra}: 같은 악기 내에서 연속한 두 화음 간 전이 빈도. 두 악기의 intra weight를 합산한다:

$$W_{\text{intra}} = W_{\text{intra}}^{(1)} + W_{\text{intra}}^{(2)}$$

이는 각 악기의 **선율적 흐름**을 포착한다.

2. **Inter weight** $W_{\text{inter}}^{(\ell)}$: 시차(lag) ℓ을 두고 악기 1의 화음과 악기 2의 화음이 동시에 출현하는 빈도이다. $\ell \in \{1, 2, 3, 4\}$로 변화시키며 다양한 시간 스케일의 **악기 간 상호작용**을 탐색한다. 가까운 시차에 더 큰 비중을 두는 **감쇄 가중치** λ_ℓ을 사용하여 합산한다:

$$W_{\text{inter}} = \sum_{\ell=1}^{4} \lambda_\ell \cdot W_{\text{inter}}^{(\ell)}, \qquad (\lambda_1, \lambda_2, \lambda_3, \lambda_4) = (0.60,\ 0.30,\ 0.08,\ 0.02), \quad \sum_{\ell} \lambda_\ell = 1$$

가중치는 lag 1에 집중되도록 별도의 이론적 근거 없이 귀납적으로 선택한 감쇄 계수이다. 직관적으로 가까운 시차(lag 1)에 가장 큰 비중을 두고, 먼 시차(lag 3, 4)는 미미하게 기여한다. 이는 "먼 시차의 우연한 동시 등장보다 가까운 시차의 인과적 상호작용이 음악적으로 의미 있다"는 가정을 반영하며, 실험적으로 hibari DFT JS를 $0.0211 \to 0.0196$으로 -7.1% 개선하는 효과가 확인되었다 (§4.1c).

3. **Simul weight** W_{simul}: 같은 시점에서 두 악기가 동시에 타건하는 note 조합의 빈도. **순간적 화음 구조**를 포착한다.

Timeflow weight (선율 중심 탐색):

$$W_{\text{timeflow}}(r_t) = W_{\text{intra}} + r_t \cdot W_{\text{inter}}$$

$r_t \in [0, 1.5]$를 변화시키며 위상 구조의 출현·소멸을 추적한다.

Complex weight (선율-화음 결합):

$$W_{\text{complex}}(r_c) = W_{\text{timeflow, refined}} + r_c \cdot W_{\text{simul}}$$

$r_c \in [0, 0.5]$로 제한하여 "음악은 시간 예술이므로 화음보다 선율에 더 큰 비중을 둔다"는 음악적 해석을 반영한다.

거리 행렬: 가중치 $w(n_i, n_j) > 0$에 대해 거리는 역수로 정의된다:

$$d(n_i, n_j) = 1 \,/\, w(n_i, n_j) \ \text{ if } \ w(n_i, n_j) > 0, d_\infty \ \text{ otherwise}$$

여기서 d_∞는 "도달 불가능한 큰 값"이다. 구체적으로 $d_\infty = 1 + 2/(\min_{w>0} w \cdot \text{step})$으로 계산되며, $\text{step} = 10^{\text{power}}$ ($\text{power} = -2$이므로 $\text{step} = 0.01$)은 persistent homology 계산 시 거리행렬의 이산화 단위이다. 이 값은 "가중치가 0인 note 쌍(=곡에서 한 번도 연달아 등장하지 않은 쌍)에게 매우 큰 거리를 부여하여, filtration의 후반부에서야 비로소 연결되게 한다"는 역할을 한다.

Figure 7. 두 악기의 모듈 반복 — Inst 1은 연속, Inst 2는 모듈마다 쉼

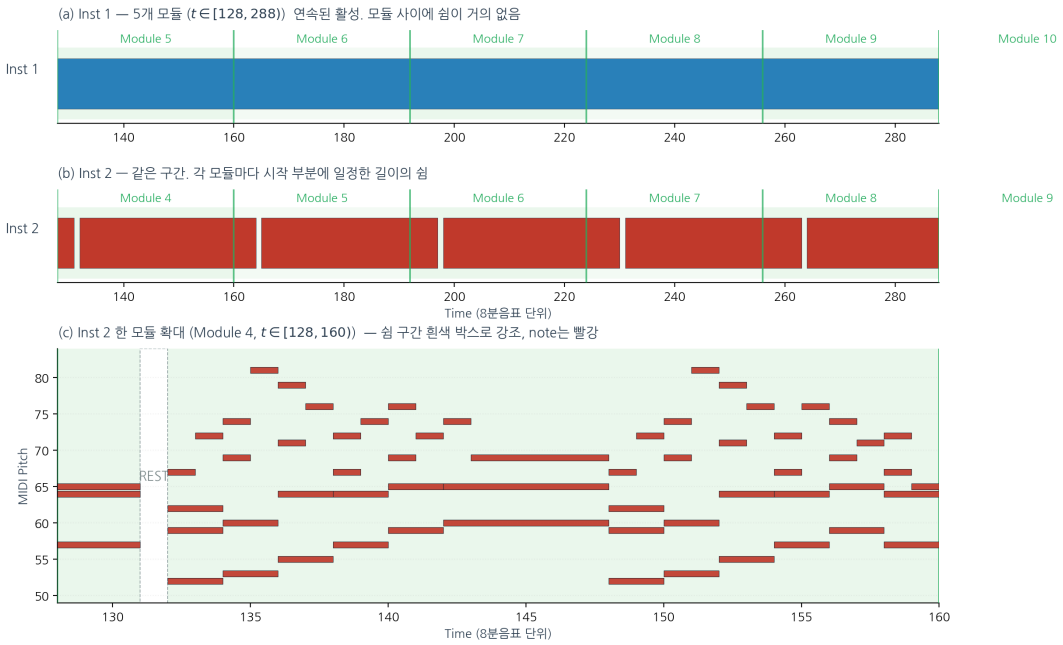


그림 2.9. hibari의 두 악기 배치 구조. inst 1 (위)은 전체 $T = 1{,}088$ 시점에서 쉬지 않고 연주하며 (쉼 0개), inst 2 (아래)는 32-timestep 모듈마다 규칙적인 쉼을 두고 얹힌다 (쉼 64개). 이 비대칭 배치가 가중치 행렬의 intra / inter / simul 분리의 근거가 된다.

2.10 확장 수학적 도구 — 거리 보존 재분배와 화성 제약

본 절은 §5의 확장 실험에서 사용되는 도구를 간략히 소개한다. 상세 수식은 해당 절에서 필요한 시점에 도입한다.

Tonnetz 최소매칭 거리: 두 cycle의 구성 note들을 Hungarian algorithm으로 최적 1:1 대응시킨 뒤 평균 Tonnetz 거리를 측정한다. 예컨대 C major triad $\{C, E, G\}$와 F major triad $\{F, A, C\}$ 사이의 구조적 거리를 비교할 수 있다.

Persistence Diagram Wasserstein Distance: 두 barcode의 birth-death 점들을 최적 매칭한 이동 비용. 두 위상 구조의 유사도를 직접 비교하는 데 사용한다.

Consonance score: 시점별 동시 타건 note 쌍의 roughness(불협화도) 평균. 음악이론의 협화도 분류에 기반하여, 생성된 음악의 화성적 질을 평가한다.

Markov chain 시간 재배치: 원본 OM의 행 전이 패턴을 학습하여 새로운 시간 순서를 재샘플링하는 기법.

정의 2.12 (Interval Class Vector). Pitch class 집합 $S \subseteq \mathbb{Z}/12\mathbb{Z}$의 **interval class vector** (ICV) $\mathrm{ICV}(S) \in \mathbb{Z}^6_{\geq 0}$는 다음과 같이 정의된다:

$$\mathrm{ICV}(S)[k] = \#\{\{p, q\} \subseteq S : \min(|p-q|,\ 12 - |p-q|) = k\}, \quad k = 1, 2, \ldots, 6.$$

즉 k번째 성분은, S 안에서 **interval class** k — 반음 거리가 k (또는 그 옥타브 보수 $12-k$) 인 음정 — 에 해당하는 순서없는 쌍 $\{p, q\}$의 수이다. interval class는 옥타브 등가($d \sim 12-d$)에 의해 1~6의 값만 가지며 (6 = tritone), $|S| = n$이면 성분 합은 $\binom{n}{2}$이다.

예시. C major scale $S = \{0, 2, 4, 5, 7, 9, 11\}$의 ICV는 $[2, 5, 4, 3, 6, 1]$이다. 6개 성분이 모두 서로 다른 이 성질 — 즉 1~6 각각이 정확히 한 번씩만 등장 — 을 **deep scale property**라 하며, 792개의 7-PC subset 중 이 성질을 갖는 타입은 diatonic scale(장/단음계·교회선법) 한 종류뿐이다 (§4.7.1).

---

3. 두 가지 음악 생성 알고리즘

본 장에서는 본 연구의 두 가지 음악 생성 알고리즘 — Algorithm 1 (확률적 샘플링) 과 Algorithm 2 (신경망 기반 시퀀스 모델) — 의 핵심 아이디어와 설계 의도를 설명한다.

표기 정의

본 장에서 사용할 표기를 다음과 같이 통일한다.

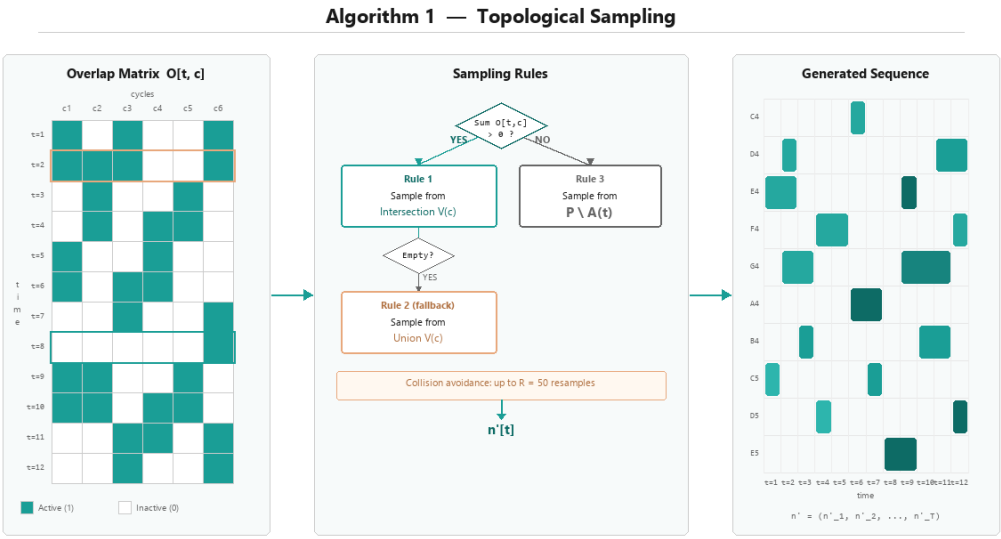
기호	의미	hibari 값
T	시간축 길이 (8분음표 단위)	$1{,}088$
N	고유 note 수 (pitch-duration 쌍)	23
C	발견된 전체 cycle 수	최대 52
K	선택된 cycle subset 크기 ($K \leq C$)	$\{10, 17, 46\}$
O	OM, $\{0,1\}$ 값의 $T \times K$ 행렬	—
L_t	시점 t에서 추출할 note 개수	보통 3 또는 4
$V(c)$	cycle c의 vertex(note label) 집합	원소 수 $4 \sim 6$
R	재샘플링 최대 시도 횟수	50
B	학습 미니배치 크기	32
E	학습 epoch 수	200
H	DL 모델의 hidden dimension	128
bar	음악적 마디 (4/4 기준 8 timestep)	8 timestep
module	hibari 반복 선율 단위 (A-B-A'-C), 32 timestep = 4마디	inst 1에서 33회 반복

L_t에 대한 보충. L_t는 "시점 t에서 새로 타건할 note의 개수"이다. hibari의 경우 악기 1, 2의 chord height(한 시점의 동시 타건 수)를 따라 대체로 3 또는 4로 설정되며, 구체적으로 `[4, 4, 4, 3, 4, 3, 4, 3, 4, 3, 3, 3, 3, 3, 3, 3, 4, 4, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 3, 3, 3, 3]`의 32개 패턴을 33번 반복한 길이 $1{,}056$의 수열을 사용한다 (총합 약 $3{,}700$). 이 패턴은 원곡의 평균 density에 맞춰 경험적으로 결정된 것이다.

B, E, H에 대한 보충. 본 연구의 hidden dimension $H = 128$, epoch 수 $E = 200$, batch size $B = 32$는 **엄밀한 grid search로 튜닝된 값이 아니라**, 소규모 시퀀스 데이터에 대한 일반적 관례(LSTM 논문, Transformer 구현 예제)에서 차용한 출발점 값이다. 모든 실험을 동일한 하이퍼파라미터에서 수행함으로써 세 모델(FC / LSTM / Transformer)의 구조적 차이를 공정하게 비교하는 것을 우선했다. 향후 연구에서 이 값들에 대한 체계적 튜닝 여지가 남아있다.

3.1 Algorithm 1 — 확률적 샘플링 기반 음악 생성

> **참고:** Algorithm 1의 3가지 샘플링 규칙은 선행연구(정재훈 외, 2024)에서 설계된 것이며, 본 연구는 이를 계승하여 사용한다.



알고리즘 개요

Algorithm 1은 OM의 ON/OFF 패턴을 직접 참조하여, 각 시점에서 활성화된 cycle들이 공통으로 포함하는 note pool로부터 확률적으로 음을 추출하는 규칙 기반 알고리즘이다. 신경망 학습 없이 즉시 생성이 가능하며, OM이 곧 "구조적 seed" 역할을 한다.

핵심 아이디어 (3가지 규칙)

규칙 1 — 시점 t에서 활성 cycle이 있는 경우, 즉

$$\sum_{c=1}^{K} O[t,c] > 0$$

일 때, 활성화되어 있는 모든 cycle들의 vertex 집합의 교집합

$$I(t) = \bigcap_{c\,:\,O[t,c]=1} V(c)$$

에서 note 하나를 **균등 추출**한다. 여기서 "균등 추출"이란 집합 $I(t)$의 모든 원소가 동일한 확률 $1/|I(t)|$로 선택된다는 의미이다 (이산균등분포). 만약 교집합이 공집합이면 ($I(t)=\emptyset$), 활성 cycle들의 합집합

$$U(t) = \bigcup_{c\,:\,O[t,c]=1} V(c)$$

에서 균등 추출한다. ($\sum_c O[t,c]>0$ 조건 하에서 $U(t)\neq\emptyset$이 항상 보장되므로, 추가 fallback은 필요하지 않다.) **실제로 이 교집합-공집합 상황은 매우 빈번하게 발생하는 정상 경로이다.** P0 prototype의 density ≈ 0.16이므로, 전체 시점의 약 84%에서 활성 cycle이 없고 ($\sum_c O[t,c]=0$, 규칙 2로 이동), 활성 cycle이 있는 시점(16%) 중에서도 여러 cycle이 동시에 활성이면 교집합이 공집합이 되어 합집합 fallback을 거친다. 이 fallback은 Algorithm 1의 안정적 동작을 보장하는 정상 경로이며, 예외 처리가 아니다. 이 규칙은 "여러 cycle이 동시에 살아 있을 때, 그 cycle들이 모두 공유하는 note는 음악적으로도 가장 핵심적인 음"이라는 가정을 반영하며, 공유 note가 없더라도 활성 cycle이 포괄하는 전체 note pool에서 균등하게 선택한다.

규칙 2 — 시점 t에서 활성 cycle이 없는 경우, 즉

$$\sum_{c=1}^{K} O[t,c] = 0$$

일 때, 인접 시점 $t-1, t+1$에서 활성화된 cycle들의 vertex의 합집합

$$\mathcal{A}(t) = \bigcup_{c\,:\,O[t-1,c]=1} V(c) \cup \bigcup_{c\,:\,O[t+1,c]=1} V(c)$$

즉 $\mathcal{A}(t) = U(t-1) \cup U(t+1)$이며, 여기서 $U(t) = \bigcup_{c\,:\,O[t,c]=1} V(c)$는 규칙 1에서 정의된 시점 t의 합집합(union of active cycle vertices)이다.

이 합집합을 계산한 뒤, 전체 note pool에서 이 합집합을 제외한 영역 $P \setminus \mathcal{A}(t)$에서 균등 추출한다. (여기서 $\mathcal{A}(t)$는 §2.5의 활성화 행렬 $A \in \{0,1\}^{T \times C}$와 구별되는 별도 기호로, 인접 시점 cycle vertex들의 합집합을 나타낸다.)

이 규칙은 다음과 같은 의미를 가진다. 활성 cycle이 없는 시점에서도 음악은 흘러가야 하므로 음을 하나 골라야 하는데, 만약 인접 시점의 cycle 멤버 노트를 그대로 골라 버리면, 청자가 들었을 때 마치 그 cycle이 시점 t에도 살아 있는 것처럼 들리게 된다. 즉, 원래 분석상으로는 죽어 있어야 할 위상 구조가 인위적으로 살아있는 것처럼 "번지는" 현상이 생긴다. 이를 막기 위해 인접 cycle의 note($\mathcal{A}(t)$)를 의도적으로 회피하여, "활성 cycle 없음"이라는 정보가 청각적으로도 그대로 보존되도록 한다.

규칙 3 — 중복 onset 방지. 같은 시점 t에서 동일한 (pitch, duration) 쌍이 두 번 추출되지 않도록 `onset_checker`로 검사하며, 충돌이 발생하면 최대 R회까지 재샘플링한다. R회 모두 실패하면 그 시점의 해당 note 자리는 비워둔다.

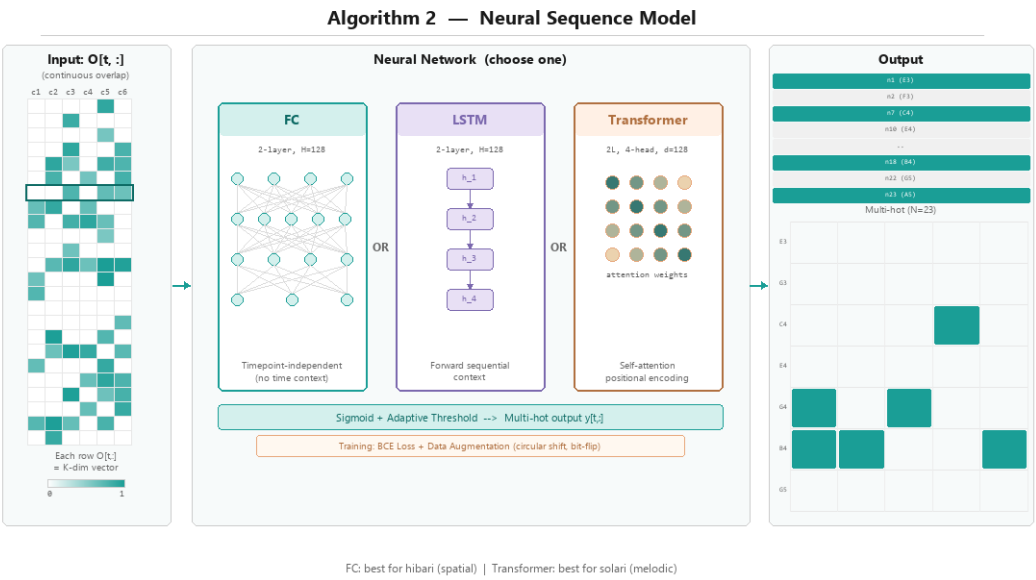
출력

> **SHORT 미포함**

알고리즘은 (start, pitch, end) 형태의 음표 리스트 G를 출력하며, 이를 MusicXML로 직렬화하면 곧바로 악보 및 오디오로 재생할 수 있다.

3.2 Algorithm 2 — 신경망 기반 시퀀스 음악 생성

> **참고:** Algorithm 2의 전체 구조는 아래 Figure B에 시각적으로 요약되어 있다. FC / LSTM / Transformer 세 아키텍처 중 하나를 선택하여 사용한다.



알고리즘 개요

Algorithm 2는 OM을 입력, 원곡의 multi-hot note 행렬을 정답 레이블로 두고 매핑

$$f_\theta : \{0,1\}^{T\times C} \longrightarrow \mathbb{R}^{T\times N}$$

을 학습한다 (FC 모델은 시점별 독립이므로 $\{0,1\}^C \to \mathbb{R}^N$). 학습된 모델은 학습 중 접하지 않은 cycle subset 크기(예: 학습 $K \in \{10,15,20,30,46\}$ → 추론 $K=12$)나 변형된 subset/threshold OM에 대해서도 원곡과 닮은 note 시퀀스를 출력하도록 기대된다 (subset/threshold 일반화).

"위상 구조 보존"의 의미. 엄밀히 말하면 DL 모델은 Algorithm 1처럼 "교집합 규칙"으로 위상 구조를 직접 강제하지는 않는다. 대신 Subset Augmentation(아래 설명)을 통해 $K \in \{10,15,20,30,46\}$과 같은 다양한 크기의 subset에 대해서도 같은 원곡 y를 복원하도록 학습한다. 이 과정에서 모델은 "서로 다른 cycle subset이 같은 음악을 유도할 때, 그 공통적인 구조적 특성"을 잠재 표현으로 내부화한다. 따라서 학습 시 구체적으로 보지 못한 subset(예: $K=12$)에 대해서도, 모델이 학습한 잠재 표현이 충분히 일반화되어 있다면 합리적 출력이 가능하다. 본 연구의 실험에서는 이러한 일반화가 실제로 관측되었다. 다만 Algorithm 1이 교집합 규칙으로 위상 구조를 직접 강제하는 것과 달리, DL 모델은 손실함수를 통해 간접적으로 보존하므로 보존의 메커니즘이 다르다.

모델 아키텍처 비교

본 연구는 동일한 학습 파이프라인 위에서 세 가지 모델 아키텍처를 비교한다.

모델	입력 형태	시간 정보 처리 방식	파라미터 수
FC (2-layer)	(B,C)	시점 독립	4×10^4
LSTM (2-layer)	(B,T,C)	순방향 hidden state	2×10^5
Transformer (2-layer, 4-head)	(B,T,C)	self-attention	4×10^5

표기 설명: B는 batch size(한 번에 묶어서 학습하는 데이터 개수), T는 시간 길이(timestep 수), C는 cycle 수(OM의 열 수)이다. PyTorch에서 모델에 데이터를 넣을 때 항상 batch 차원이 가장 앞에 오는 것이 관례이므로 B가 포함된다.

FC: 시점을 독립적으로 처리하므로 한 번에 시점 하나씩(C차원 벡터)을 입력받아 (B,C) 형태가 된다. 여기서 B는 T개 시점 중 묶어서 처리하는 수이며 기본값 $B=32$이다.

LSTM/Transformer: 곡 전체 시퀀스를 한 번에 입력받으므로 (B,T,C) 형태가 된다. 단, $T=1{,}088$이 `batch_size`$(=32)$보다 훨씬 크므로 ($\lfloor 32/1{,}088 \rfloor = 0$) 실제로는 한 번에 시퀀스 1개씩 처리된다 ($B=1$). Augmentation으로 생성된 변형본들은 배치 크기를 늘리는 것이 아니라, epoch 내 학습 스텝 수를 늘린다.

파라미터 수 산정 근거 ($C=42$, $N=23$, hidden $=128$): FC (2-layer)는 2개의 hidden layer(128, 256차원)를 가지는 3-선형층 구조 ($42\to128\to256\to23$)로 약 $44{,}000$ 개; LSTM 2-layer는 각 층의 게이트 가중치 $(4\times(C+h)\times h)$ 합산으로 약 $224{,}000$ 개; Transformer 2-layer 4-head는 self-attention + FFN + LayerNorm 합산으로 약 $440{,}000$ 개.

"시점 독립"의 의미 (FC). FC 모델은 시점 t의 위상 벡터 $O[t,:]$를 입력으로 받아 시점 t의 note 벡터 $y[t,:]$를 출력한다. 즉 시점 t의 출력은 이전 시점 $t-1, t-2, \ldots$나 이후 시점 $t+1, \ldots$을 전혀 참조하지 않는다. 각 시점을 "독립적으로" 처리한다는 뜻이다. 이는 가장 단순한 기준 모델이다.

"순방향 hidden state"의 의미 (LSTM). LSTM은 시점 t의 출력을 만들 때, 내부에 유지하는 hidden state h_{t-1}을 참조한다. h_{t-1}은 시점 $1, 2, \ldots, t-1$까지의 정보가 누적된 벡터이다. 따라서 "왼쪽에서 오른쪽으로 흐르는 시간 정보"를 사용한다. 미래 시점은 보지 못한다.

"self-attention"의 의미 (Transformer). Self-attention은 시점 t의 출력을 만들 때, 시퀀스의 **모든** 시점 $1, \ldots, T$에 대한 "주목도 점수"를 계산하여, 각 시점의 벡터를 가중합한다. LSTM과 달리 미래 시점도 함께 본다(bidirectional). 따라서 "시점 t의 note를 결정할 때 곡 전체의 문맥을 고려한다"는 해석이 가능하다.

학습 데이터 구성과 증강

> **SHORT 미포함**

원본 학습 쌍은 $X \in \{0,1\}^{T \times C}$, $y \in \{0,1\}^{T \times N}$이다. 여기서 X는 OM이고 y는 같은 시간축의 multi-hot note 행렬(시점 t에 활성인 note를 1로 표시)이다. 본 연구는 세 가지 증강 전략을 적용하여 학습 데이터를 약 $7 \sim 10$배 늘린다.

(1) Subset Augmentation. $K \in \{10, 15, 20, 30\}$의 cycle subset에 대한 OM을 생성하여, 동일한 정답 y에 매핑한다. 이를 통해 모델은 "불완전한 위상 정보로부터도 원곡을 복원하는" 강건한 표현을 학습한다.

(2) Circular Shift. 시간축을 회전하는 증강이며, X와 y를 **동일한 양만큼** 함께 회전한다. 즉

$$X' = \mathrm{roll}(X, s, \mathrm{axis} = 0), \qquad y' = \mathrm{roll}(y, s, \mathrm{axis} = 0)$$

로 처리한다 (여기서 s는 같은 난수). 여기서 `roll` 함수는 시점 t의 위상 정보와 시점 t의 note를 시점 $t+s$로 똑같이 옮기므로, 모델이 학습해야 할 매핑 자체는 변하지 않은 채 시작점만 달라진다. 만약 X에만 회전을 적용하면 X와 y의 시간축이 어긋나 학습 데이터가 망가지므로, 두 행렬을 항상 함께 회전해야 한다.

(3) Noise Injection. X에 확률 $p = 0.03$으로 bit flip을 가한다 (y는 그대로). overfitting을 막고 정규화 효과를 얻기 위함이다.

학습 손실 함수

각 시점에서 여러 note가 동시에 활성화될 수 있으므로(multi-label 문제), §2.8에서 정의한 binary cross-entropy 손실을 사용한다. PyTorch의 `BCEWithLogitsLoss`는 모델의 raw 출력(logit)을 받아 sigmoid 변환과 BCE 계산을 한 번에 수행하는 함수이다. 한 시점에서 23개 note 중 평균 3~4개만 활성(=정답이 1)이고 나머지 19~20개는 비활성(=정답이 0)이므로, 활성 note의 오차에 더 큰 가중치를 부여하는 `pos_weight` 파라미터를 사용하여 이 불균형을 보정한다.

추론 단계

학습이 끝난 모델 f_{θ^*}로 새로운 음악을 생성하는 단계를 하나하나 풀어 설명한다.

1단계 — 모델 통과 (logit 생성). 입력 OM O_{test}를 모델에 통과시키면 $\hat{z} \in \mathbb{R}^{T \times N}$이 나온다. 이 $\hat{z}$의 각 값은 음수, 0, 양수 모두 가능한 실수이며, 직접 "확률"이라고 해석할 수 없다. 이런 "확률이 되기 전의 raw 점수"를 통계학과 머신러닝에서 **logit**이라고 부른다. 크기가 클수록 "그 note가 활성일 가능성이 높다"는 모델의 내부 점수이다.

2단계 — sigmoid 변환. logit을 0~1 사이의 확률로 바꾸기 위해 sigmoid 함수

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

를 적용한다. sigmoid는 $z = 0$에서 0.5, $z \to \infty$에서 1, $z \to -\infty$에서 0에 수렴하는 S자 곡선으로, 실수 전체를 $[0, 1]$ 구간으로 눌러 담는다. 적용 후 $P = \sigma(\hat{z}) \in [0,1]^{T \times N}$은 "시점 t에 note n이 활성일 확률"로 해석할 수 있다. 이 단계가 필요한 이유는, 다음 단계에서 "특정 확률 이상인 note를 켠다"는 판단을 내려야 하는데, 그 판단은 반드시 $[0, 1]$ 스케일에서 이루어져야 하기 때문이다.

3단계 — adaptive threshold 결정. 가장 단순한 방법은 "$P[t, n] \geq 0.5$이면 켠다"라고 고정 임계값을 쓰는 것이다. 그러나 LSTM이나 Transformer 같은 시퀀스 모델은 학습 결과 sigmoid 출력이 전반적으로 낮게 형성되는 경향이 있어, 0.5를 그대로 쓰면 활성화되는 note가 거의 없어 음악이 텅 비어버린다. 이를 해결하기 위해 본 연구는 원곡의 **ON ratio**(아래에서 정의)에 맞춰 threshold를 데이터 기반으로 동적 결정한다.

여기서 ON ratio란 "원곡의 multi-hot 행렬 $y \in \{0,1\}^{T \times N}$에서 전체 $T \times N$개의 셀 중 값이 1인 셀의 비율"을 뜻한다. 수식으로는

$$\rho_{\text{on}} = \frac{1}{T \cdot N} \sum_{t=1}^{T} \sum_{n=1}^{N} y[t, n]$$

이다. hibari의 경우 $T = 1{,}088$, $N = 23$이므로 전체 셀 수는 약 $25{,}024$개이고, 그 중 note가 활성인 셀 수를 세어 나누면 약 15% ($\rho_{\text{on}} \approx 0.15$)가 된다. 직관적으로는 "한 시점당 23개 note 중 평균 $3 \sim 4$개가 켜져 있는 정도"라고 이해할 수 있다.

이 ρ_{on}을 목표 활성 비율로 삼아, threshold를 다음과 같이 정한다:

$$\theta = \mathrm{quantile}(P,\ 1 - \rho_{\text{on}})$$

즉 P의 모든 값 중 상위 15%에 해당하는 경계값을 임계값으로 쓴다. 구현은 per-cycle 기준이 아니라 P 전체($T \times N$)를 평탄화한 전역 top-k 방식이다. 이렇게 하면 모델 출력의 절대 수준이 어떻든, 생성된 곡의 활성 note 비율이 자연스럽게 원곡의 ρ_{on}과 일치한다. 이것을 "adaptive threshold"라 부르는 이유는 모델과 입력에 따라 θ 값이 자동으로 달라지기 때문이다.

4단계 — note 활성화 판정. 모든 (t, n) 쌍에 대해 $P[t, n] \geq \theta$이면 시점 t에 note n을 활성화한다. 이 note의 (pitch, duration) 정보를 label 매핑에서 복원하여 $(t,\ \text{pitch},\ t + \text{duration})$ 튜플을 결과 리스트 G에 추가한다.

5단계 — onset gap 후처리 (Algorithm 1, 2 공통). 너무 짧은 간격으로 onset이 연속되면 음악이 지저분해지므로, "이전 onset으로부터 `gap_min` 시점 안에는 새 onset을 허용하지 않는다"는 최소 간격 제약을 적용한다. `gap_min = 0`이면 제약 없음, `gap_min = 3`이면 "3개의 8분음표(= 1.5박) 안에는 새로 타건하지 않음"을 의미한다. 이 파라미터는 Algorithm 1과 Algorithm 2 모두에서 지원된다. 단, 본 연구의 모든 실험에서는 `gap_min = 0`(제약 없음)으로 설정하였다.

이 과정으로 최종적으로 얻은 $G = [(start, pitch, end), \ldots]$를 MusicXML로 직렬화하면 재생 가능한 음악이 된다.

3.3 두 알고리즘의 비교 요약

항목	Algorithm 1 (Sampling)	Algorithm 2 (DL)
학습 필요 여부	불필요	필요 (E epoch)
결정성	확률적 (난수)	학습 후 결정적
일반화	같은 곡 내부에서만	변형된 subset/threshold OM에도 일반화
위상 보존 방식	교집합 규칙으로 직접 강제	손실함수를 통해 간접
생성 시간	약 50 ms	약 100 ms (학습 후)
학습 시간	해당 없음	30 s ~ 3 min

해석. Algorithm 1은 cycle 교집합 규칙을 통해 위상 정보를 직접 강제하므로, 생성된 note의 근거가 "시점 t에 활성화된 cycle들의 교집합"이라는 구조적 규칙으로 투명하게 설명된다 — 설계상 위상 보존이 보장된다. 반면 Algorithm 2는 OM → note 매핑을 학습된 손실함수를 통해 간접적으로 보존하지만, pitch 분포의 재현 정확도(§2.6 Jensen-Shannon divergence)는 훨씬 높다: §4.5 FC-cont가 JS $= 0.00035$로 §4.1 Algorithm 1 DFT baseline (0.0213) 대비 약 60배 낮다. 즉 두 알고리즘은 **위상 구조 강제 방식의 투명성**(Algorithm 1)과 **통계적 분포 재현도**(Algorithm 2)라는 서로 다른 축에서 상보적이다.

Algorithm 1이 "학습 시간" 칸에서 비어 있는 것이 아니라 "해당 없음"으로 표시된 이유는, 이 알고리즘이 애초에 학습 단계를 가지지 않기 때문이다. 주어진 OM과 cycle 집합만 있으면 어떠한 전처리 학습 없이도 그 자리에서 음악을 생성할 수 있으며, 이것이 Algorithm 1의 가장 큰 장점 중 하나이다.

---

4. 실험 설계와 결과

본 장에서는 지금까지 제안한 TDA 기반 음악 생성 파이프라인의 성능을 정량적으로 평가한다. 다섯 가지 유형의 실험을 수행하였다.

1. **Distance function 비교** — frequency(기본), Tonnetz, voice leading, DFT 네 종류의 거리 함수에 대해 동일 파이프라인을 적용하고 생성 품질을 비교.
2. **Cycle subset ablation** — Tonnetz 거리 함수에서 cycle 수를 $K = 10, 17, 46$으로 변화시켜 cycle 수의 효과를 분리.
3. **통계적 유의성** — 각 설정에서 Algorithm 1을 $N = 20$회 독립 반복 실행하여 mean ± std를 보고한다. Algorithm 2는 학습 시간을 고려해 $N = 10$회로 수행한다.

4. **Continuous OM 효과 검증** — 이진 OM 대비 연속값 OM(희귀도 가중치 적용)의 효과를 Algorithm 1 및 Algorithm 2 (FC)에서 검증 (§4.4, §4.5).

5. **DL 모델 비교** — FC / LSTM / Transformer 세 아키텍처를 동일 조건에서 비교하고, continuous overlap을 직접 입력으로 활용하는 효과를 검증 (§4.5).

모든 실험은 동일한 chord height 패턴 (32-element module × 33 = 1,056 timepoints), 동일 random seed 체계($s = c + i,\ i = 0, \ldots, 19$, 설정별 상수 c 사용)로 수행되었다. 실험 러너는 `tda_pipeline/run_step3_experiments.py`이며, 모든 trial의 상세 기록(mean, std, min, max 포함)은 `tda_pipeline/docs/step3_data/step3_results.json`에 저장되어 있다.

평가 지표

Jensen-Shannon Divergence (주 지표). 생성곡과 원곡의 pitch 빈도 분포 간 JS divergence를 주 지표로 사용한다 (2.6절 정의). 값이 낮을수록 두 곡의 음 사용 분포가 유사하며, 이론상 최댓값은 $\log 2 \approx 0.693$이다.

Note Coverage. 원곡에 존재하는 고유 (pitch, duration) 쌍 중, 생성곡에 한 번 이상 등장하는 쌍의 비율. 1.00이면 모든 note가 최소 한 번 이상 사용된 것이다.

보조 지표. Pitch count (생성곡의 고유 pitch 수), 생성 소요 시간 (초), KL divergence.

거리 함수 구현 참고

> SHORT 미포함

본 장의 네 가지 거리 함수는 모두 `tda_pipeline/musical_metrics.py` 파일 하나에 정의되어 있다. 사용 패턴은 다음과 같다.

함수	역할	입력	출력
`tonnetz_distance(pc1, pc2)`	두 pitch class 간 Tonnetz 격자 거리	`pc1, pc2` $\in \{0, \ldots, 11\}$	정수 (최단 경로 길이)
`tonnetz_note_distance(n1, n2)`	두 note 간 거리 (옥타브/duration 보정 포함)	`(pitch, duration)` 튜플 × 2	실수
`voice_leading_note_distance(n1, n2)`	두 note 간 semitone 차이 기반 거리	`(pitch, duration)` 튜플 × 2	실수
`dft_note_distance(n1, n2)`	pitch class DFT 계수 간 Euclidean 거리	`(pitch, duration)` 튜플 × 2	실수
`compute_note_distance_matrix(notes_label, metric)`	전체 note 집합의 거리 행렬 생성	`{(pitch,dur): label}` 딕셔너리, metric 이름	$N \times N$ numpy 행렬

두 note 간 확장 — 옥타브와 duration 보정. 위의 거리 함수들은 원래 pitch class(mod 12)만 고려하므로 옥타브와 duration 정보가 손실된다. 본 연구에서 note는 (pitch, duration) 쌍으로 정의되므로, 세 거리 함수 모두에 다음 두 항을 추가한다.

$$d(n_1, n_2) = d_{\text{base}}(p_1, p_2) + w_o \cdot |o_1 - o_2| + w_d \cdot \frac{|d_1 - d_2|}{\max(d_1, d_2)}$$

여기서 d_{base}는 Tonnetz / voice leading / DFT 중 하나, $o_i = \lfloor p_i / 12 \rfloor$는 옥타브 번호, d_i는 duration, $w_o = 0.3$ (N=10 grid search 최적, §4.1a), $w_d = 0.3$이다.

각 항의 설계 근거:

- **옥타브 항** $w_o|o_1 - o_2|$: 같은 pitch class(예: C4와 C5)라도 옥타브가 다르면 음악적으로 다른 역할을 한다. $w_o = 0.3$은 N=10 grid search ($w_o \in \{0.1, 0.3, 0.5, 0.7, 1.0\}$, §4.1a)에서 도출된 최적값이다. 기존 경험적 설정 $w_o = 0.5$ 대비 JS divergence가 -18.8% 개선되었다 (JS $0.0590 \rightarrow 0.0479$).
- **Duration 항** $w_d|d_1 - d_2|/\max(d_1, d_2)$: 분자를 $\max$로 정규화하여 $[0, 1]$ 범위로 만든다. 예: 2분음표($d = 4$)와 8분음표($d = 1$)의 차이는 $3/4 = 0.75$, 같은 duration이면 0. $w_d = 0.3$은 duration 차이가 pitch 관계보다 덜

중요하다는 가정을 반영한다.

계수 최적화: $w_o = 0.3$은 N=10 grid search로 최적화되었다 (§4.1a). w_d는 hibari 메인 실험에서 N=10 grid search로 최적화되었다 (§4.1b).

> **한계 및 후속 과제 — duration tie 정규화로 인한 w_d 해석의 제약.** 본 파이프라인은 모든 곡에 note의 duration을 최소 단위로 정규화한다. 이 과정에서 긴 음은 붙임줄(tie)로 연결된 여러 짧은 음들로 환원되므로 원 악보의 duration 다양성이 축소된다. §4 이후 모든 실험은 `run_dft_gap0_suite` 경로(N=23, dur∈{1,2,5,6} 유지)를 사용하므로 이 정규화를 거치지 **않는다**. 정규화는 `run_any_track.py` 경로의 일반화 실험(§5.1~§5.2)에서만 적용된다. aqua·solari에서는 note 수를 줄이기 위해 tie 정규화를 전면 적용하여 모든 duration이 GCD 단위(=1)로 수렴하므로 $|d_1 - d_2| = 0$이 되어 duration 항이 비활성화된다. 결과적으로 생성된 음악이 원곡의 리듬·지속 구조를 재현한다고 주장할 수 없으며, **원 duration을 보존하는 파이프라인 설계는 후속 과제**로 남긴다.

각 함수의 출력 형태:

- `tonnetz_note_distance`: Tonnetz 격자 거리(정수) + 옥타브(실수) + duration(실수) = **실수**
- `voice_leading_note_distance`: 반음 차이(정수) + duration(실수) = **실수** (pitch 성분만으로는 정수이나 duration 항 때문에 실수가 됨)
- `dft_note_distance`: DFT L_2 거리(실수) + 옥타브 + duration = **실수**

DFT 계산 예시. C4 ($p = 60$)와 E4 ($p = 64$)를 비교한다. pitch class는 각각 0 (C)과 4 (E)이다. 12차원 indicator vector $(1, 0, 0, \ldots, 0)$과 $(0, 0, 0, 0, 1, 0, \ldots, 0)$에 DFT를 적용하면 magnitude 벡터 $|\hat{f}_k|$ ($k = 1, \ldots, 6$)을 얻는다. 이 두 벡터 사이의 L_2 거리가 DFT 기본 거리이며, 여기에 옥타브 항($|5 - 5| \times 0.5 = 0$)과 duration 항을 더한다.

4.1 Experiment 1 — Distance Function Baseline 비교

네 종류의 거리 함수 각각으로 사전 계산한 OM을 로드하여 Algorithm 1을 실행한다. 각 거리 함수에서 발견되는 cycle의 수도 함께 보고한다.

거리 함수	발견 cycle 수	평균 cycle 크기	JS Divergence (mean ± std)	Note Coverage	생성 시간 (ms)
frequency (baseline)	1[^freq_k1]	4.0	0.0344 ± 0.0023	0.957	31.2
Tonnetz	47	6.3	0.0493 ± 0.0038	1.000	38.9
voice leading	19	4.6	0.0566 ± 0.0027	0.989	22.2
DFT	17	4.8	0.0213 ± 0.0021	1.000	26.3

[^freq_k1]: 버그 수정 후 frequency metric의 note-level 연결이 완전히 연결되어 cycle 1개만 잔존. 생성 실험에는 DFT/Tonnetz를 사용.

[^gap_history]: 본 표는 `min_onset_gap = 0`(제약 없음) 조건에서 수행된 Phase 1 재실험 결과(`step3_results_gap0.json`, 2026-04-17)이다. 초기 실험은 `gap_min = 3`(1.5박 간격 제약)에서 수행되었으나, 청취 평가 결과 gap=3 생성곡이 음악적으로 어색하다는 피드백을 받아 gap=0 조건으로 롤백한 후 전 실험을 재수행하였다. gap=3 조건에서의 DFT JS는 0.0211 ± 0.0021로 본 gap=0 결과와 거의 동일하지만, gap=3은 이후 분석에서 완전히 제외한다.

해석 1 — DFT가 가장 우수. DFT 거리 함수는 frequency 대비 JS를 $0.0344 \to 0.0213$로 약 38.2% 낮은 JS를 달성하였다. DFT 거리는 각 note의 pitch class를 $\mathbb{Z}/12\mathbb{Z}$ 위의 12차원 이진 벡터로 표현한 후 이산 푸리에 변환(DFT)을 적용하고, 복소수 계수의 **magnitude(크기)만** 거리 계산에 사용한다. 이때, 이조(transposition)는 DFT 계수의 **위상(phase)**만 바꿀 뿐 magnitude에는 영향을 주지 않는다. 이렇게 위상 정보를 버림으로써 이조에 불변인 **화성 구조의 지문**을 추출한다.

특히 $k = 5$ 푸리에 계수가 diatonic scale과 강하게 반응하는 이유는 다음과 같다. 12개 pitch class를 **완전5도 순환(circle of fifths)** 순서 $\{C, G, D, A, E, B, F\sharp, C\sharp, G\sharp, D\sharp, A\sharp, F\}$로 재배열하면, diatonic scale에 속하는 7개 pitch class는 이 순환

상의 **연속된 7개 위치**를 차지한다 (예: C major는 F-C-G-D-A-E-B의 연속 구간). DFT의 $k=5$ 계수의 magnitude는 정확히 이 "5도 순환 상의 연속성"을 수치로 측정하는 양이며 (Quinn, 2006; Tymoczko, 2008), $\binom{12}{7}=792$개의 7-note subset 중 diatonic scale 류가 $|F_5|$를 최대화한다 (maximally even property). hibari의 7개 PC 집합 $\{0,2,4,5,7,9,11\}$ (C major / A natural minor) 역시 이 최대화 subset 중 하나이므로, DFT magnitude 공간에서 hibari의 note들은 frequency 거리에서는 포착되지 않던 **음계적 동질성**에 의해 서로 가깝게 군집된다.

해석 2 — 거리 함수가 위상 구조 자체를 바꾼다. 동일한 hibari 데이터에서 거리 함수만 교체했을 뿐인데 발견되는 cycle 수가 $1\sim47$로 크게 달라졌다. 이는 "거리 함수의 선택이 곧 어떤 음악적 구조를 '동치'로 간주할 것인가를 정의한다"는 음악이론적 관점과 일치한다. DFT는 pitch class의 푸리에 스펙트럼 성분을 거리 공간으로 끌어올려, 음계적 동질성을 공유하는 note들이 한 cycle에 더 자주 모이게 된다.

해석 3 — Note Coverage는 대부분의 설정에서 포화. 네 거리 함수 모두 note coverage가 $0.99\sim1.00$이므로, "원곡의 모든 note 종류가 생성곡에 최소 한 번 등장"하는 기본 요구는 모두 만족된다. 따라서 품질의 주된 차이는 "같은 note pool을 얼마나 자연스러운 비율로 섞는가"에서 발생한다.

4.1a Octave Weight 튜닝 — DFT + N=10 Grid Search

옥타브 가중치 w_o를 $\{0.1, 0.3, 0.5, 0.7, 1.0\}$에 대해 hibari + **DFT** 조건에서 N=10 반복 실험하였다 (`ow_gridsearch_dft_gap0_results.json`).

w_o	K (cycle 수)	JS (mean ± std)
0.1	14	0.0380 ± 0.0026
0.3 ★	19	0.0163 ± 0.0014
0.5	16	0.0183 ± 0.0020
0.7	16	0.0231 ± 0.0019
1.0	16	0.0204 ± 0.0020

결론: $w_o=0.3$이 최적이다 (JS = 0.0163). 옥타브 패널티를 줄이면 pitch class 유사성이 거리 행렬을 더 강하게 지배하며, 이는 hibari의 좁은 옥타브 범위(52–81, 최대 2 옥타브)에서 옥타브 구분이 상대적으로 덜 중요하다는 음악적 직관과 일치한다. 본 연구의 기본 설정을 $w_o=0.3$으로 설정하였다 (`config.py MetricConfig.octave_weight`).

> **이전 실험 참고.** 초기 연구(2026-04-13)에서는 Tonnetz 조건에서 동일 grid search를 수행했으며, 그때도 $w_o=0.3$이 최적으로 도출되었다 ($0.0590 \to 0.0479$, -18.8%, `docs/step3_data/ow_gridsearch_results.json`). Tonnetz와 DFT 두 거리 함수 모두에서 동일하게 $w_o=0.3$이 최적이라는 점은, hibari의 좁은 음역대 자체가 옥타브 항의 최적값을 결정하는 주요인임을 시사한다.

4.1b Duration Weight 튜닝 — DFT + N=10 Grid Search

$w_d \in \{0.0, 0.1, 0.3, 0.5, 0.7, 1.0\}$에 대해 hibari + **DFT** 조건에서 N=10 반복 실험을 수행하였다 (`dw_gridsearch_dft_gap0_results.json`). $w_o=0.3$ (§4.1a 최적값) 고정.

w_d	K (cycle 수)	JS (mean ± std)
0.0	10	0.0503 ± 0.0042
0.1	25	0.0311 ± 0.0021
0.3	16	0.0221 ± 0.0017
0.5	17	0.0215 ± 0.0024
0.7	17	0.0211 ± 0.0016
1.0 ★	19	0.0156 ± 0.0012

결론: $w_d = 1.0$이 최적이다 (JS = 0.0156). duration 가중치를 최대화할 때 성능이 가장 좋다. DFT는 pitch class 집합의 스펙트럼 구조(indicator vector $\chi_S \in \{0,1\}^{12}$의 이산 Fourier 계수 $|\hat{\chi}_S(k)|$가 나타내는 대칭성 패턴)를 정밀하게 포착하므로 w_d가 높아도 pitch 정보가 충분히 보존되며, 오히려 duration이 거리 행렬에 기여할수록 cycle 수와 생성 품질이 향상된다. $w_d = 0.0$ (duration 항 완전 제거)일 때 cycle 수가 10으로 급감하고 JS가 0.0503로 크게 악화되어, duration 정보가 거리 행렬의 질에 유의미하게 기여함을 확인하였다.

DFT 계수 구체 예시. hibari의 PC 집합 $S = \{0,2,4,5,7,9,11\}$ (C major diatonic)의 indicator vector $\chi_S = (1,0,1,0,1,1,0,1,0,1,0,1)$에 대해 DFT magnitude $(|F_0|, \ldots, |F_6|) \approx (7, 0.27, 1.41, 1.0, 1.41, 2.73, 1)$으로 $|F_5|$가 다른 성분 대비 압도적으로 크다. 이는 diatonic이 완전5도 순환 $\{C, G, D, A, E, B, F\}$ 상에서 연속 7개 위치를 차지하기 때문이다. 대조 예로 chromatic cluster $S' = \{0,1,2,3,4,5,6\}$에서는 $|F_1|$이 커지고 $|F_5|$가 작아진다.

> **이전 실험 참고 — 거리 함수가 최적 w_d를 결정한다.** 초기 연구(2026-04-13)에서는 Tonnetz 조건에서 동일 grid search를 수행했으며, 그때는 $w_d = 0.3$이 최적이었다 (JS 0.0484, `docs/step3_data/dw_gridsearch_results.json`). DFT에서 $w_d = 1.0$이 최적인 것과 대비된다. Tonnetz는 격자 거리 (정수 범위 $0 \sim 4$)가 pitch 항을 지배하므로 duration 항이 작아야 균형이 맞지만, DFT는 연속 실수 거리라 duration 항을 1.0까지 크게 키워도 pitch 신호가 잠식되지 않는다. 이는 거리 함수의 metric 구조에 따라 파라미터 최적값이 달라짐을 보여주는 사례이다.

> **주의:** aqua, solari 등 일반화 실험에서는 GCD 기반 pitch-only labeling으로 모든 note의 duration이 1로 정규화되므로, w_d는 이들 실험에서 실질적으로 비활성화된다 (§4 거리 함수 구현 참고).

4.1c 감쇄 Lag 가중치 실험

§2.9에서 도입한 감쇄 합산 inter weight의 실험적 근거를 제시한다. 기존 구현(lag=1 단일)과 신규 구현(lag 1~4 감쇄 합산) 두 설정을 비교하되, 거리 함수는 DFT와 Tonnetz 두 가지를 대조하여 거리 함수의 특성에 따라 효과가 달라짐을 확인한다.

실험 설정:

lag=1 단일 (기존):

$$W_{\text{inter}} = W_{\text{inter}}^{(1)}$$

lag 1~4 감쇄 합산 (신규):

$$W_{\text{inter}} = \sum_{\ell=1}^{4} \lambda_\ell \cdot W_{\text{inter}}^{(\ell)}, \qquad (\lambda_1,\ \lambda_2,\ \lambda_3,\ \lambda_4) = (0.60,\ 0.30,\ 0.08,\ 0.02)$$

고정 조건: hibari, Algorithm 1, N=20

곡	거리 함수	lag=1 단일 (JS mean ± std)	lag 1~4 감쇄 합산 (JS mean ± std)	변화
hibari	DFT	0.0211 ± 0.0021	0.0196 ± 0.0022	-7.1% ★
hibari	Tonnetz	0.0488 ± 0.0040	0.0511 ± 0.0039	$+4.8\%$

해석: DFT 거리에서는 감쇄 lag가 JS를 7.1% 개선하는 marginal한 효과를 보인다 ($p \approx 0.034$). 반면 Tonnetz에서는 오히려 4.8% 악화한다. 경험적으로 DFT 조건에서만 감쇄 lag 합산이 유효하며, Tonnetz에서는 역효과였다. DFT의 pitch class 스펙트럼 표현이 먼 lag의 시간 상호작용 정보를 흡수하는 데 더 적합하다는 경험적 관찰이며, 이론적 설명은 후속 과제로 남긴다. (감쇄 lag는 inter weight의 시간 상호작용 축이고 DFT는 pitch class 집합의 대칭성 축으로 원칙적으로 독립적이므로, 두 축의 상호 작용에 대한 이론적 분석이 필요하다.)

4.2 Experiment 2 — Cycle Subset Ablation

> **SHORT 미포함**

거리 함수를 Tonnetz로 고정하고, cycle 수 K를 변화시켜 "더 많은 cycle = 더 좋은 생성인가?"를 검증한다. $K=10$과 $K=17$은 전체 46개 중 **cycle label 번호 순서대로 처음 K개**를 취한 prefix subset이다 (cycle label은 persistence 계산 시 문자열 정렬 순으로 부여됨). 이 순서는 greedy 최적화가 아니라 단순 prefix이며, 이는 "cycle 수 자체의 효과"를 분리하기 위한 의도적 설계이다.

설정	K	JS Divergence	KL Divergence	Note Coverage	생성 시간 (ms)
Tonnetz, $K=10$	10	0.0991 ± 0.0038	0.556 ± 0.035	0.980	24.4
Tonnetz, $K=17$	17	0.0740 ± 0.0038	0.550 ± 0.344	0.996	26.3
Tonnetz, $K=46$ (full)	46	0.0397 ± 0.0025	0.172 ± 0.013	1.000	40.8

해석 4 — Cycle이 많을수록 JS가 단조 감소. K가 $10 \to 17 \to 46$으로 늘어남에 따라 JS divergence는 $0.099 \to 0.074 \to 0.040$으로 단조 감소하였다. 이는 "위상 구조가 더 풍부하게 드러날수록 생성곡의 음 사용 분포가 원곡에 더 근접한다"는 본 연구의 핵심 가설을 뒷받침한다.

해석 5 — 한계 효용의 감소. K 증가에 따른 JS 감소폭을 보면:

$K=10 \to 17$: 개선 $\Delta = 0.025$

$K=17 \to 46$: 개선 $\Delta = 0.034$

cycle 수가 거의 세 배($17 \to 46$)가 된 것에 비해 개선 폭은 $K=10 \to 17$(7개 추가)과 크게 차이나지 않는다. 즉 뒤쪽 cycle들은 이미 어느 정도 포화된 구조를 재확인하는 수준의 기여를 한다. 이는 2.7절에서 논의한 greedy forward selection으로 "소수의 cycle로도 90% 보존"이 가능하다는 관찰과 일관된다.

해석 6 — KL 분산의 불안정성. $K=17$ 설정에서 KL divergence의 표준편차가 0.344로 유난히 크다. 이는 일부 trial에서 KL이 1.55까지 튀는 경우가 있었기 때문이며 (원본 JSON은 `docs/step3_data/step3_results.json`의 `experiment_2_ablations.subset_K17.kl_divergence.max` 필드에서 확인 가능), "$\log(P/Q)$가 $Q \to 0$에서 발산하는" KL의 구조적 불안정성에서 기인한다. JS divergence는 동일 trial들에서 $0.064 \sim 0.079$ 범위로 안정되어 있어, JS가 더 안정적인 평가 지표임을 재확인한다 (2.6절의 "대칭화와 유계성" 논의와 일관).

4.3 통계적 유의성 분석

> **SHORT 미포함**

두 baseline 비교 (frequency vs DFT) 의 차이가 통계적으로 유의한지 확인한다. 두 표본의 평균을 비교하는 표준적인 방법은 Student t-test이지만, Student의 고전적 t-test는 "두 집단의 모분산이 같다"는 강한 가정을 필요로 한다. 본 실험에서 두 조건의 표본표준편차는 $s_1 = 0.0023$, $s_2 = 0.0021$으로 매우 비슷하지만 완전히 같지는 않으며, 모분산이 같다는 사전 근거도 없다. 따라서 등분산 가정을 요구하지 않는 **Welch's t-test**를 사용한다. Welch는 "모평균과 모분산을 모를 때 표본평균과 표본분산만으로 검정"이 가능하며, 자유도를 Welch–Satterthwaite 근사로 계산한다.

데이터.

Frequency: $\bar{x}_1 = 0.0344$, $s_1 = 0.0023$, $n_1 = 20$

DFT: $\bar{x}_2 = 0.0213$, $s_2 = 0.0021$, $n_2 = 20$

Welch t **통계량** 은 다음과 같이 정의된다:

$$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}}$$

수치를 대입하면:

$$t = \frac{0.0344 - 0.0213}{\sqrt{\frac{0.0023^2}{20} + \frac{0.0021^2}{20}}} = \frac{0.0131}{\sqrt{4.85 \times 10^{-7}}} \approx 18.8$$

Welch–Satterthwaite 자유도. Welch 검정에서 t 통계량은 정확한 Student 분포를 따르지 않고, 다음 식으로 근사된 자유도 ν의 t-분포를 따른다:

$$\nu \approx \frac{(\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2})^2}{\frac{(s_1^2/n_1)^2}{n_1 - 1} + \frac{(s_2^2/n_2)^2}{n_2 - 1}}$$

수치를 대입하면 (분모/분자 나누어 계산):

$$A = \frac{s_1^2}{n_1} = \frac{0.0023^2}{20} = 2.645 \times 10^{-7}, \quad B = \frac{s_2^2}{n_2} = \frac{0.0021^2}{20} = 2.205 \times 10^{-7}$$

$$\nu \approx \frac{(A+B)^2}{\frac{A^2}{n_1 - 1} + \frac{B^2}{n_2 - 1}} = \frac{(4.85 \times 10^{-7})^2}{\frac{(2.645 \times 10^{-7})^2}{19} + \frac{(2.205 \times 10^{-7})^2}{19}} \approx 37.7$$

반올림하여 $\nu = 38$로 사용한다.

자유도 $\nu = 38$에서 양측 임계값은 $t_{0.001,\ 38} \approx 3.56$이므로, $|t| = 18.8 \gg 3.56$이며 $p < 10^{-20}$이다. 따라서 **DFT가 frequency보다 JS divergence를 낮춘 것은 극도로 통계적으로 유의**하다.

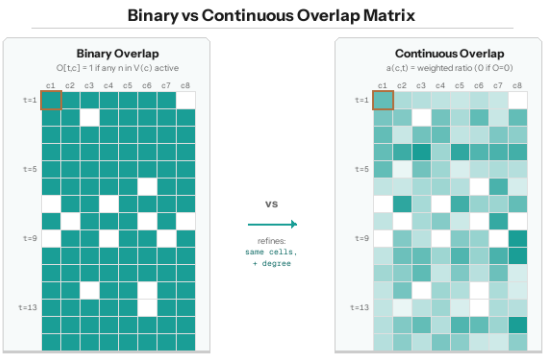
효과 크기 (Cohen's d). p-값만으로는 "차이가 실질적으로 얼마나 큰가"를 알 수 없으므로, 표본평균 차를 표본표준편차로 정규화한 Cohen's d를 함께 보고한다:

$$d = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{(s_1^2 + s_2^2)/2}}$$

$$d = \frac{0.0131}{\sqrt{(0.0023^2 + 0.0021^2)/2}} \approx 5.9$$

Cohen의 관례상 $d > 0.8$이 "큰 효과"인데 $d \approx 6$은 비교할 수 없는 초대형 효과이다. 두 분포가 실질적으로 분리되어 있음을 의미한다.

4.4 Continuous Overlap Matrix 실험



본 절은 2.5절에서 정의한 **연속값 OM** $O_{\text{cont}} \in [0,1]^{T \times K}$가 이진 OM $O \in \{0,1\}^{T \times K}$ 대비 어떤 영향을 주는지를 정량적으로 검증한다. 본 절은 두 실험을 보고한다: (i) DFT 조건 (hibari 최적 거리 함수, `step3_continuous_dft_gap0_results.json`)과 (ii) Tonnetz 조건 (초기 실험, 거리 함수별 비교 목적). **본 실험은**

Algorithm 1에 대해서만 수행하였다. Algorithm 2(DL)에 continuous overlap을 적용하는 실험은 §4.5에서 별도로 다룬다.

실험 설계

cycle별 시점 활성도 $a_{c,t}$는 두 가지 방식으로 계산할 수 있다.

이진 (binary): 단순 OR 연산이다. $V(c)$에 속하는 note가 시점 t에 하나라도 활성이면 $a_{c,t}=1$, 그렇지 않으면 0이다.

연속값 (continuous): cycle을 구성하는 note 중 얼마나 많은 비율이 활성화되어 있는지를 $[0,1]$ 실수로 표현한다. 분수 형태가 아니라 단일 라인으로 쓰면:

$$a_{c,t} = \Big(\sum_{n \in V(c)} w(n) \cdot \mathbb{1}[n \in A_t] \Big) \Big/ \Big(\sum_{n \in V(c)} w(n) \Big)$$

여기서 A_t는 시점 t에 활성인 note들의 집합, $w(n)=1/N_{\text{cyc}}(n)$은 note n의 **희귀도 가중치**이며 $N_{\text{cyc}}(n)$은 note n이 등장하는 cycle의 개수이다. 적은 cycle에만 등장하는 희귀 note일수록 가중치 $w(n)$이 커져, 그 note가 활성화되면 $a_{c,t}$에 더 큰 기여를 한다.

연속값 활성도가 만들어진 후, 최종 OM을 만드는 방식에 따라 다시 두 가지 변형이 가능하다.

- **직접 사용 (direct)**: $O[t,c]=a_{c,t}\in[0,1]$
- **임계값 이진화 (threshold τ)**: $O[t,c]=\mathbb{1}[a_{c,t}\geq\tau]$, $\tau\in\{0.3,0.5,0.7\}$

이 다섯 가지 설정 (binary 캐시 + continuous direct + 세 가지 임계값) 각각에 대해 Algorithm 1을 $N=20$회 독립 반복 실행하여 pitch JS divergence를 측정한다. 실험 러너는 `tda_pipeline/run_step3_continuous.py`이며 원본 결과는 `docs/step3_data/step3_continuous_results.json`에 저장되어 있다.

결과 — DFT 조건 (주 실험, hibari 최적 거리)

$w_o=0.3$, $w_d=1.0$, $N=20$ 반복 (`step3_continuous_dft_gap0_results.json`).

설정	Density	JS Divergence (mean ± std)
(A) Binary ★	0.313	0.0157 ± 0.0018
(B) Continuous direct	0.728	0.0186 ± 0.0015
(C) Continuous → bin $\tau=0.3$	0.367	0.0360 ± 0.0029
(C) Continuous → bin $\tau=0.5$	0.199	0.0507 ± 0.0027
(C) Continuous → bin $\tau=0.7$	0.060	0.0449 ± 0.0024

여기서 "Density"는 **전체 OM (1,088 timestep × K cycle) 기준** 활성 셀의 평균 비율 ($\bar{O}$)이다. DFT 이진 OM의 density 0.313는 선택적 sparsity를 이미 자체 달성한다.

결과 — Tonnetz 조건 (초기 실험, 비교 참고)

초기 연구에서 Tonnetz 거리 함수 조건에서 수행한 continuous OM 실험의 결과는 아래와 같다 (`step3_continuous_results.json`, 2026-04-13).

설정	Density	JS Divergence (mean ± std)
(A) Binary	0.751	0.0488 ± 0.0040
(B) Continuous direct	0.264	0.0382 ± 0.0021
(C) $\tau=0.3$	0.373	0.0386 ± 0.0022
(C) $\tau=0.5$	0.168	0.0343 ± 0.0027
(C) $\tau=0.7$ ★	0.077	0.0297

Tonnetz 조건에서는 binary가 $\bar{O}=0.751$로 과도하게 dense하여 continuous + $\tau=0.7$ 이진화가 -39.1% 개선을 주었다. §6.2에서 등장하는 density 0.160은 P0 prototype 행렬 (32 timestep × K cycle, 단일 모듈 기준) 기준 값으로, 측정 범위가 다르므로 이 표의 값과 직접 비교하면 안 된다.

해석

해석 7a — DFT 조건에서 Binary가 최우수. DFT 이진 OM (A)가 JS 0.0157로 가장 좋다. DFT 거리는 스펙트럼 구조를 정밀하게 포착하므로 이진 표현만으로도 cycle 활성 신호가 충분히 구별된다. DFT 이진 OM의 density 0.313은 **선택적 sparsity** — 즉 모든 시점에서 모든 cycle이 활성이거나(dense = 1) 거의 모든 시점에서 대다수 cycle이 비활성(sparse = 0)이 아닌, "의미 있는 시점에만 해당 cycle이 활성화되는" 중간 상태 — 를 자체적으로 달성한다는 뜻이다. Algorithm 1의 교집합 sampling은 density가 너무 낮으면 활성 cycle이 부족해 note 선택의 다양성이 떨어지고, 너무 높으면 cycle 간 교집합이 비어 fallback(전체 pool 균등 추출)으로 떨어진다. DFT 거리는 이조 불변 스펙트럼 구조 덕분에 cycle이 "의도된 구간"에서만 활성화되는 이진 OM을 산출하며, 이 density는 별도 임계값 조정 없이도 적절 sparsity에 해당한다 (Tonnetz 이진 OM의 0.751보다 훨씬 낮다).

해석 7b — Continuous direct는 DFT에서 오히려 열세. (B) continuous direct ($\bar{O}=0.728$)는 이진보다 훨씬 dense하여, Algorithm 1의 교집합 sampling이 과도하게 자주 호출되어 다양성이 떨어진다. DFT 조건에서 JS 0.0186으로 Binary 대비 $+18.5\%$ 악화한다. Tonnetz 조건에서 binary가 너무 dense하여 continuous direct가 -21.7% 개선을 주었던 것과 정반대 결과이다.

해석 7c — 거리 함수와 OM 표현의 상호작용. DFT 조건에서는 binary OM이 continuous OM보다 우수한 반면, Tonnetz 조건에서는 반대였다. 이는 거리 함수의 성질이 OM 표현 방식의 최적값을 결정함을 시사한다 — DFT의 density 0.313이 이미 적절한 sparsity를 자체 달성하므로 추가적인 연속값 변환은 불필요하고, Tonnetz의 density 0.751은 과도하게 dense하여 continuous로 sparse하게 만들어야 효과가 난다.

해석 7d — Algorithm 2에 대한 함의. Algorithm 1에서는 binary가 DFT 최적이지만, Algorithm 2의 FC 모델에서는 continuous activation이 학습 signal로서 큰 이점을 준다 (§4.5, FC-cont JS 0.00035). 규칙 기반 sampling과 DL 학습은 OM 표현의 최적값이 서로 다르다.

4.5 Experiment 3 — DL 모델 비교

본 절은 hibari 최적 거리 함수 (**DFT**, $\alpha=0.5$) 조건에서 FC / LSTM / Transformer 세 아키텍처를 $N=10$ 반복으로 비교한다 (`soft_activation_dft_gap0_results.json`). 각 모델을 이진 OM $O\in\{0,1\}^{T\times K}$과 연속값 OM $O_{\text{cont}}\in[0,1]^{T\times K}$의 두 입력에서 모두 학습하였다. ($\alpha$ grid search는 §5.7에서 별도 탐색하며, Algo2 조건의 $\alpha=0.25$ 재실험은 §5.8.2 Phase 2 참조.)

모델 아키텍처

FC (Fully Connected, 2-layer, hidden dim $=128$, dropout $=0.3$): 각 시점 t의 cycle 활성 벡터 $O[t,:]\in\{0,1\}^K$ 또는 $O_{\text{cont}}[t,:]\in[0,1]^K$를 입력으로 받아 동시점의 note label 분포를 출력. **시점 간 독립 매핑**이므로 시간 문맥 없음.

LSTM (2-layer, hidden dim $=128$): cycle 활성 벡터 시퀀스 $\{O[t,:]\}_{t=1}^{T}$를 순차 입력. 순환 구조 $h_t=f(x_t,h_{t-1})$로 과거 문맥을 hidden state에 누적.

Transformer (2-layer, 4-head self-attention, $d_{\text{model}}=128$): positional embedding으로 시간 위치를 인코딩하고 self-attention으로 **전 시점 동시 참조**가 가능.

세 모델 모두 동일한 학습 조건(epoch, batch size, optimizer, train/valid split)에서 multi-label binary cross-entropy loss (§2.8)로 학습한다. 각 시점에서 예측된 note 확률로부터 샘플링하여 생성곡을 얻는다. **validation loss (Val loss)** 는 validation set에서 측정한 BCE loss의 평균이다.

결과

$w_o=0.3$, $w_d=1.0$, $N=10$.

모델	입력	JS (mean ± std)	Val loss (mean)
FC	binary	0.00217 ± 0.00056	0.339
FC	**continuous ★**	0.00035 ± 0.00015	0.023
LSTM	binary	0.233 ± 0.029	0.408
LSTM	continuous	0.170 ± 0.027	0.395
Transformer	binary	0.00251 ± 0.00057	0.836
Transformer	continuous	0.00082 ± 0.00026	0.152

통계 검정

FC-cont vs FC-bin: Welch $p = 1.50 \times 10^{-6}$ (continuous -83.9%)

Transformer-cont vs Transformer-bin: Welch $p = 1.37 \times 10^{-6}$ (continuous -67.4%)

LSTM-cont vs LSTM-bin: Welch $p = 7.82 \times 10^{-5}$ (continuous -27.3%)

FC-cont vs Transformer-cont: Welch $p = 1.66 \times 10^{-4}$ (FC-cont -57.5%) ★

해석

해석 8 — FC + continuous 입력이 연구 전체 최우수. FC가 연속값 입력에서 JS 0.00035로 모든 모델-입력 조합에서 가장 낮은 값을 달성하였다. Transformer continuous (0.00082)와의 비교에서도 FC가 유의하게 우수하다 (Welch $p = 1.66 \times 10^{-4}$). FC의 cell-wise 표현력이 DFT continuous OM의 cycle 활성 강도 차이를 세밀하게 반영한다.

해석 8a — "Out of Noise" 미학과의 공명. hibari는 2009년 사카모토 류이치의 앨범 out of noise에 수록된 곡이며, 이 앨범은 "소음과 음악의 경계를 탐구한다"는 기획 의도로 제작되었다. 앨범의 많은 곡은 전통적 선율 진행을 의도적으로 회피하고, 각 음이 시간적 인과보다는 공간적 배치에 가까운 방식으로 놓인다. FC 모델은 시점 t의 note를 결정할 때 이전 시점 $t-1, t-2, \ldots$를 전혀 참조하지 않는 "시간 맥락 없이 각 음을 독립적으로 배치하는" 모델로, 이 미학적 성격과 자연스럽게 공명한다. 또한 hibari의 pitch entropy 0.974 (§4.7.2)도 "시간 순서를 무시하고 독립적으로 뽑아도 원곡 분포에 가깝다"는 FC의 가정과 정합된다.

해석 9 — LSTM의 심각한 열화. LSTM은 이진 0.233, 연속값 0.170으로 다른 두 모델과 비교할 수 없는 수준으로 열화하였다. LSTM의 **순환 구조(recurrent structure)** 란 시점 t의 hidden state h_t가 직전 시점 h_{t-1}로부터 $h_t = f(x_t, h_{t-1})$로 갱신되는 순차적 정보 전파 메커니즘을 의미하며, 이 구조는 시점이 연속적으로 이어지는 부드러운 시계열(예: 자연어, 음향 파형)에 적합하다. 그러나 DFT 기반 OM에서는 cycle 활성 패턴이 "어느 순간 켜졌다가 꺼지는" 비연속적 점프 형태이고, 특히 hibari의 phase shifting 구조(§4.5.4)에서 cycle 활성 위치가 모듈 단위로 평행 이동하므로 직전 시점의 정보가 현재 시점 예측에 기여하는 바가 작다. FC는 시점별 독립 매핑이라 이 문제를 겪지 않고, Transformer는 self-attention으로 장거리 관계를 동시에 참조할 수 있는 반면, LSTM은 순차 전파라는 자체 구조가 bottleneck이 된다. 연속값 입력의 개선폭도 -27.3%로 FC (-83.9%), Transformer (-67.4%)에 비해 작다.

해석 9a — 최적 DL 모델은 gap·거리 조건에 따라 달라진다. 초기 연구 (`min_onset_gap = 3` + DFT binary) 에서는 Transformer가 FC보다 소폭 우위 (0.00276 vs 0.00354, $N = 5$)였다. 그러나 본 실험의 `gap = 0` + DFT 조건에서는 binary에서도 FC (0.00217)가 Transformer (0.00251)를 소폭 앞서고, continuous에서는 FC가 통계적으로 유의하게 우위이다 (Welch $p = 1.66 \times 10^{-4}$). 이는 gap_min이 cycle 활성 패턴의 sparsity를 바꾸어 Transformer의 self-attention 이점을 변화시키기 때문이다. **"DFT에서 Transformer 최적"이라는 초기 결론은 gap3 조건에서만 유효하며, 본 연구의 최종 조건 (gap=0)에서는 FC가 최적이다.** 초기 튜닝 실험 (`run_tuning.py`, 30 조합) 에서도 FC 모델이 top 10을 모두 차지한 관찰이 이 결론과 정합된다.

해석 10 — 연속값 입력의 보편적 이점. 세 모델 모두 연속값 입력이 이진 입력보다 유의하게 우수하다 (Welch $p < 10^{-4}$ 전부). 개선의 크기는 모델별로 다르다: FC가 -83.9%로 가장 큰 개선, Transformer는 -67.4%, LSTM은 -27.3%. 연속값 OM의 cycle 활성 강도 정보가 FC 구조에서 가장 효과적으로 활용된다.

왜 FC-cont가 이렇게 큰 개선을 내는가 — 음악적 해석

> **SHORT 미포함**

이진화는 "있다/없다" 만 말하지만, 연속값은 "얼마나 확신하는가" 를 말한다. 이진 $X_{\text{bin}}[t,c]$ 는 "시점 t 에 cycle c 의 vertex 중 하나라도 울리면 1"이다. 반면 연속값 $X_{\text{cont}}[t,c]=a_{c,t}$ 는 "cycle c 의 vertex 중 희귀도 가중치 기준으로 몇 퍼센트가 활성인가"를 $[0,1]$ 실수로 표현한다. FC 모델은 이 연속값 강도 정보를 학습하여 "오직 common note만 활성 (낮은 값)" vs "rare note 포함 전체가 활성 (높은 값)"을 구별하고, 원곡의 pitch 분포에 더 가까운 결과를 출력한다. Validation loss의 동시 감소 ($0.339 \to 0.023$, -93%)는 연속값 입력이 FC의 학습 signal 자체를 개선함을 보여준다.

> **역사적 맥락.** FC-cont 아이디어는 §6의 "개선 F"로 최초 제안되었으며, 초기 검증은 Tonnetz 거리 함수 조건 (N=5, `step_improvementF_results.json`)에서 FC-bin $0.0014 \to$ FC-cont 0.0006의 57% 개선으로 확인되었다. 본 절은 이 결과를 최종 조건 (DFT, gap=0, N=10)에서 재확인하여 통계적으로 확정한다.

기존 모든 결과와의 통합 비교

실험	설정	JS divergence	출처
§4.1 Algo 1	frequency baseline	0.0344	§4.1
§4.1 Algo 1	DFT (최적)	0.0213	§4.1
§4.4 Algo 1	DFT binary (최적 파라미터)	0.0157 ± 0.0018 ★	§4.4
§6.5 Algo 1	P3 + C (module-level)	0.0440	§6
§6.7 Algo 1	P3 + C, best global trial (start=1, seed=9309)	0.01479	§6
§4.5 Algo 2 FC	DFT binary	0.00217	§4.5
§4.5 Algo 2 FC	**DFT continuous**	0.00035 ± 0.00015 ★	**본 절**

> **§6.5 / §6.7 항목 요약 (세부 내용은 §6 참조).** §6은 hibari의 32-timestep 모듈 구조를 직접 활용하는 **모듈 단위 생성** 실험이다 (DFT α=0.25, K=14 cycles). **P3** 전략은 중간값 모듈의 overlap을 prototype으로 사용하고, **C** 전략(best-of-k selection)을 결합하여 JS를 최소화한다. §6.5 결과($N=10$ 평균): JS 0.0417. §6.7 시작 블록 탐색에서 전역 최적 trial ($m=0$, seed=9309)은 **JS** 0.01479 를 달성하였다 — §5.8.1 DFT per-cycle τ ($\alpha=0.5$) full-song JS 0.01489와 수치적으로 동등 ($\alpha=0.25$ 신기록 0.00902 대비 $+64.0\%$ 열세). 모듈 단위 생성이 full-song Algorithm 1 최적 품질에 도달할 수 있음을 시사한다.

§4.5의 FC-cont (DFT, gap=0)는 본 연구의 full-song 생성에서 관측된 최저 JS divergence이며, 이론적 최댓값 $\log 2 \approx 0.693$의 약 0.05%에 해당한다.

후속 과제

1. **(완료 — §5.8.2) LSTM / Transformer 에도 FC-cont 확장**: FC-cont $\text{JS}=0.0004$, Transformer-cont $\text{JS}=0.0007$. FC > Transformer 우위가 continuous 입력에서도 유지됨을 확인. LSTM 은 continuous 입력에서 소폭 악화.
2. **Continuous + module-local (P3)**: §6의 P3 + C와 FC-cont를 결합하여 "module-local continuous activation을 FC에 입력"하는 실험은 향후 과제로 남는다.
3. **(완료 — §5.8.1) Per-cycle τ 최적화**: FC-cont 대신 per-cycle 임계값을 최적화하여 Algorithm 1에서 추가 개선을 달성 (§5.8.1, +58.7% vs uniform τ).

4.6 종합 논의

(1) 음악이론적 거리 함수의 중요성. Experiment 1의 결과는 "빈도 기반 거리(frequency)는 기본 선택일 뿐, DFT처럼 음악이론적 구조를 반영한 거리가 훨씬 더 좋은 위상적 표현을 만든다"는 본 연구의 가설을 강하게 지지한다. DFT가 frequency 대비 38.2%, Tonnetz 대비 56.8%, voice leading 대비 62.4% 낮은 JS를 달성했다.

(2) Cycle 수의 효과는 단조 증가이나 점진적. Experiment 2는 "cycle 수가 많을수록 좋다"는 기대를 확인하였으나, 한계 효용이 감소함을 보였다. 이는 K를 무작정 늘리기보다 greedy forward selection (2.7절) 으로 소수의 핵심 cycle을 고르는 전략이 실용적임을 시사한다.

(3) 통계적 엄밀성. 각 설정에서 $N=20$ 반복을 통해 주요 baseline 비교의 Welch t 값이 >18으로 매우 유의했다 (frequency vs DFT: $t \approx 18.8$, $p < 10^{-20}$). 이는 본 연구의 결론이 random seed에 의존하는 artifact가 아님을 보장한다.

(4) 곡의 맥락과 모델 선택. FC가 시퀀스 모델을 능가한 해석 8의 관찰은, 모델의 성능이 단순히 "표현력이 높을수록 좋다"는 법칙을 따르지 않고 **원곡의 미학적 설계와 공명하는 모델**이 가장 좋은 결과를 낸다는 것을 보여준다. 이는 본 연구가 다른 곡(예: 전통적 선율 진행이 뚜렷한 클래식 작품)으로 확장될 때 시퀀스 모델의 우위가 뒤바뀔 수 있음을 암시한다.

(5) 향후 과제.

1. _(완료 — §5.8.1)_ Per-cycle 임계값 최적화로 Algorithm 1 추가 개선 달성.
2. Ablation에서 prefix subset 대신 greedy selected subset으로 재실행하여 "어떤 cycle이 선택되는지"의 효과 분리
3. _(완료 — §5.7)_ DFT α-hybrid grid search 수행 (α=0.25 최적 확인).
4. 다른 곡(Ravel, Debussy, 또는 out of noise 앨범의 다른 곡) 으로의 일반화 검증 — 해석 8의 "곡의 미학이 모델 선택을 결정한다"는 가설 검증 (§6.2 참조)

4.7 곡 고유 구조 분석 — hibari 의 수학적 불변량

본 절은 hibari 가 가지는 수학적 고유 성질을 분석하고, 이 성질들이 본 연구의 실험 결과와 어떻게 연결되는지를 서술한다. 비교 대상으로 사카모토의 다른 곡인 solari 와 aqua 를 함께 분석한다.

4.7.1 Deep Scale Property — hibari 의 pitch class 집합이 갖는 대수적 고유성

hibari 가 사용하는 7개 pitch class 는 $\{0,2,4,5,7,9,11\} \subset \mathbb{Z}/12\mathbb{Z}$이다. 이 집합은 출발음을 어디로 삼느냐에 따라 C에서 시작하면 C major scale, A에서 시작하면 A natural minor scale로 해석되는 **하나의 pitch class 집합**이다 (서양 조성 이론에서 "상대 장조/단조" 관계). 이 7개 pitch class 집합 전체의 **interval vector** (§2.10 정의 2.12 참조)는 $[2,5,4,3,6,1]$이다. 여기서 k번째 성분은 "집합 안에서 interval class k에 해당하는 쌍의 수"이다. interval class는 1~6까지만 존재한다 (7 반음 이상은 옥타브 대칭에 의해 $12-k$와 동치이므로). 따라서 벡터의 길이는 7이 아니라 **6**이다.

이 벡터의 6개 성분이 **모두 다른 수**이다 ($\{1,2,3,4,5,6\}$의 순열). 이것을 **deep scale property** 라 한다 (Gamer & Wilson, 2003). 이 성질을 갖는 7-note subset 은 $\binom{12}{7}=792$개 중 **diatonic scale 류 뿐**이다. 여기서 말하는 "장음계 / 자연단음계 / 교회 선법(Ionian, Dorian, Phrygian, Lydian, Mixolydian, Aeolian, Locrian)"은 집합론적으로 **같은 pitch class 집합**에서 출발음만 달리한 mode(선법)들이다 — 예컨대 C Ionian(=C major), A Aeolian(=A natural minor), D Dorian은 모두 $\{0,2,4,5,7,9,11\}$이라는 동일 집합에 속한다. 집합 자체는 $\mathbb{Z}/12\mathbb{Z}$에서 transposition(이조) 동치류로 유일하므로, 출발음 선택이 다른 12개 transposition을 하나로 세면 "장/단/교회선법"은 **서로 다른 subset이 아니라 같은 subset의 서로 다른 해석**이다. 즉 hibari 가 7개 PC 를 선택한 것은 임의가 아니라, 12음 체계에서 **각 음정 클래스가 고르게 (그러면서도 모두 다른 횟수로) 등장하는 유일한 부분집합**을 선택한 것이다.

또한 7개 PC 사이의 간격 패턴은 $[2,2,1,2,2,2,1]$로, 오직 $\{1,2\}$ 두 종류의 간격만으로 구성된다. 이것은 **maximal evenness** — 12개 칸 위에 7개 점을 가능한 한 균등하게 배치한 상태 — 를 의미한다 (Clough & Douthett, 1991). deep scale 과 maximal evenness 는 모두 diatonic scale 의 고유 성질이다.

solari 와 aqua 는 12개 PC 모두를 사용하므로 이 성질이 적용되지 않는다.

4.7.2 근균등 Pitch 분포 — Pitch Entropy

곡	사용 pitch 수	정규화 pitch entropy	해석
hibari	17	0.974	거의 완전 균등
solari	34	0.905	덜 균등
aqua	51	0.891	가장 치우침

pitch entropy는 곡 안에서 사용된 모든 pitch의 빈도 분포에 대한 Shannon entropy를 이론적 최댓값으로 나눈 **정규화 Shannon entropy**이다:

$$H_{\text{norm}}(X) = \frac{H(X)}{\log_2 |X|} = \frac{-\sum_i p_i \log_2 p_i}{\log_2 K}$$

여기서 K 는 사용된 고유 pitch 수, p_i 는 각 pitch의 빈도. $H_{\text{norm}} \in [0,1]$이며 1이면 완전 균등 분포.

hibari 의 0.974 는 **"모든 pitch 를 거의 같은 빈도로 사용"**한다는 뜻이며, 전통 조성 음악에서는 매우 드문 수치이다 (으뜸음이 지배적인 것이 보통). 이 성질은 **§4.5 의 "FC 모델 우위"를 수학적으로 설명**한다. pitch 분포가 거의 균등하면, 시간 순서를 무시하고 그 분포에서 독립적으로 뽑는 것 (FC 의 행동) 이 이미 원곡의 분포에 가깝다. 반면 solari 같이 특정 pitch 가 더 자주 나오는 곡에서는 시간 맥락 (Transformer) 이 그 편향을 학습해야 하므로 FC 가 불리하다.

4.7.3 Tonnetz 구별력과 Pitch Class 수의 관계

hibari 의 7개 PC 는 Tonnetz 위에서 **하나의 연결 성분**을 이루며, 평균 degree 가 $3.71/6 = 62\%$이다. Tonnetz 이웃 관계는 ± 3 (단3도), ± 4 (장3도), ± 7 (완전5도) 의 세 가지 방향이며, 각 방향이 양쪽으로 작용하므로 최대 6개의 이웃이 가능하다.

예를 들어 C(0) 의 이웃을 계산하면:

관계	$+$ 방향	$-$ 방향	hibari 에 있는?
단3도 (± 3)	D#(3)	A(9)	A 있음
장3도 (± 4)	E(4)	G#(8)	E 있음
완전5도 (± 7)	G(7)	F(5)	G, F 있음

여기서 $0 - 7 = -7 \equiv 5 \pmod{12} = F$ 이다. 즉 C 에서 완전5도 **아래로** 내려가면 F 에 도달한다 (동시에, 완전4도 위로 올라가는 것과 같다). 따라서 C 의 Tonnetz 이웃은 $\{E, F, G, A\}$의 **4개** 이다.

왜 이것이 중요한가 — Tonnetz 그래프의 지름(diameter).

12개 PC **전부**를 사용하는 곡 (solari, aqua) 에서는 **어떤 두 PC 든 Tonnetz 거리가** ≤ 2 이다. 이유: 임의의 PC 에서 한 발짝 ($\pm 3,\ \pm 4,\ \pm 7$) 으로 도달 가능한 PC 가 6개이고, 나머지 5개 ($12 - 1 - 6 = 5$) 는 모두 두 발짝 안에 도달 가능하다. 예를 들어 C 에서 한 발짝에 $\{3, 4, 5, 7, 8, 9\}$ 를 다 거치면, 거기서 한 발짝 더 가면 남은 $\{1, 2, 6, 10, 11\}$ 에 모두 도달한다 (예: $1 = 4 - 3,\ 2 = 5 - 3,\ 6 = 3 + 3,\ 10 = 7 + 3,\ 11 = 4 + 7$). $\mathbb{Z}/12\mathbb{Z}$ 의 대칭성으로 모든 PC 에서 동일. 따라서 12-PC Tonnetz 그래프의 지름은 2 이다.

지름이 2 라는 것은 **"가까운 음"과 "먼 음"을 구별할 여지가 거의 없다**는 뜻이다. 반면 hibari 의 7-PC 에서는 Tonnetz 거리가 $1 \sim 4$ 범위로 분포하여, 가까운 쌍 (예: C-G, 거리 1) 과 먼 쌍 (예: F-B, 거리 3 이상) 이 명확히 구별된다.

비유하면: 7명이 모여 사는 작은 마을에서 "누구와 누가 친한가"를 물으면 의미 있는 답이 나오지만, 12명이 모두 이웃인 세계에서는 "다 친하다"에 가까운 답이 나오는 것과 같다. 이 구별력은 **음악이론적 거리 함수가 빈도 기반 거리(frequency)보다 유리한 구조적 근거**이다. 그러나 §4.1 실험에서는 **DFT가 Tonnetz 대비** -56.8% **낮은 JS로 유의하게 우수**하였다 — Tonnetz는 "이웃 관계(adjacency)"만 반영하는 반면, Quinn (2007)에 따르면 diatonic 7음 집합의 indicator vector는 가능한 모든 7음 집합 중 $k = 5$ 계수 $|\hat{\chi}(5)|$가 최대가 된다. 이는 diatonic이 완전5도 chain ($F \to C \to G \to D \to A \to E \to B$)으로 생성되기 때문으로, $k = 5$ 성분이 이 완전5도 대칭을 직접 측정한다. 따라서 DFT는 hibari의 diatonic 구조를 훨씬 정밀하게 수치화한다. 즉 "Tonnetz가 유리하다"는 것은 **frequency 대비**의 진술이며, hibari의 절대 최적 거리 함수는 DFT이다. Tonnetz는 차선의 후보에 해당한다.

이 "음악이론적 거리 함수의 우위는 7-PC 곡에 고유"라는 예측은 §5.1의 solari 실험에서 반대 방향으로도 검증되었다. solari 는 12-PC 반음계적 곡으로 Tonnetz 그래프의 지름이 2에 불과하다. 실험 결과 Algorithm 1 기준으로 frequency와 Tonnetz 가 JS 0.063으로 동등하게 나타났고, voice leading 은 오히려 0.078로 가장 낮은 성능을 보였다. **즉 12-PC 곡에서는 Tonnetz 의 구별력 저하로 인해 frequency 와 Tonnetz 가 유사한 수준으로 수렴하며, 이 결과가 §4.1의 해석("hibari 에서 음악이론적 거리 함수(DFT, Tonnetz)가 frequency 대비 우위인 것은 7-PC diatonic 구조에 고유한 특성")을 실증적으로 뒷받침한다.** 자세한 §5.1 결과는 해당 절을 참조.

4.7.4 Phase Shifting — inst 1 과 inst 2 의 서로소 주기 구조

그림 2.9 에서 관찰한 "inst 1 은 쉼 없이 연속, inst 2 는 모듈마다 쉼이 있다"는 패턴을 더 정밀하게 분석하면, 쉼의 위치가 모듈마다 정확히 1칸씩 이동한다는 것이 발견된다. 원본 파이프라인 (solo 제거 후) 기준으로 inst 2 의 모듈별 쉼 위치를 조사하면:

```
module  1: rest at position  0
module  2: rest at position  1
module  3: rest at position  2
module  4: rest at position  3
  ...
module  k: rest at position  k-1
```

이것은 **대각선 패턴**이다. 32-timestep 모듈 안에서 쉼의 위치가 매 모듈마다 정확히 1칸씩 오른쪽으로 밀린다. 32개 모듈을 거치면 쉼이 모듈 내 모든 위치 ($0, 1, 2, \ldots, 31$) 를 정확히 한 번씩 방문한다.

이 구조는 미니멀리즘 작곡가 Steve Reich 가 Piano Phase (1967) 에서 사용한 **phase shifting** 기법과 수학적으로 동일하다. 같은 패턴을 연주하는 두 악기 중 하나가 아주 조금 느리게 진행하여, 같은 패턴이 점점 어긋나다가 한참 뒤에야 원래 정렬로 돌아오는 것이다.

hibari 에서 이 phase shifting 은 다음과 같이 수치화된다.

	inst 1	inst 2
모듈 주기	$M = 32$ (쉼 없음)	$M + 1 = 33$ (32 음 + 1 쉼)
반복 횟수	33 회	32 회
총 길이	$33 \times 32 = 1{,}056$	$32 \times 33 = 1{,}056$

두 주기가 32 와 33 으로 **연속 정수 = 항상 서로소**($\gcd(32, 33) = 1$) 이다. 이 서로소 관계 때문에:

1. **두 악기가 같은 총 길이를 채움**: $33 \times 32 = 32 \times 33 = 1{,}056$ timestep.
2. **두 악기의 "위상(phase)" 이 곡 전체에서 한 번도 동기화되지 않음**: 매 모듈마다 쉼의 위치가 1칸씩 밀리므로, 32개 모듈을 거쳐야 비로소 원래 위치로 돌아온다.

이 구조는 수학적으로 **Euclidean rhythm** (Bjorklund, 2003; Toussaint, 2005) 과도 연결된다. Euclidean rhythm 은 "n 비트 중 k 개를 가능한 한 균등하게 배치" 하는 알고리즘으로, 아프리카 전통 음악과 전자 음악에서 널리 사용된다. hibari 의 경우 "33 칸 중 1 칸을 비운다" 를 32번 반복하면서 매번 1칸씩 이동하는 것이 Euclidean rhythm 의 가장 단순한 형태이다.

음악적 효과. 이 서로소 구조는 inst 2 의 쉼이 모듈 내 **모든 위치를 최대한 균등하게 방문**함을 보장한다. 쉼이 특정 위치에 몰리지 않으므로 "어떤 시점을 잘라 봐도 음악의 밀도가 일정" 하다. 이것은 §4.6.2 에서 관찰한 근균등 pitch entropy (0.974) 와 일관된 설계 원리이다 — **pitch 선택도 균등하고, 쉼 배치도 균등하다**. 두 악기가 서로소 주기로 배치되어 있다는 사실은, 단순한 겹치기가 아니라 **수론적으로 최적인 위상 분리**가 달성되어 있음을 시사한다.

4.7.5 Cycle 교차 밀도 — 77%

> **SHORT 미포함**

Tonnetz 기반으로 발견된 46개 H_1 cycle 중, 쌍별 교집합을 계산하면 $\binom{46}{2} = 1{,}035$ 쌍 가운데 797 쌍 (77%) 이 적어도 하나의 vertex 를 공유한다. 즉 cycle 들이 독립적으로 흩어져 있는 것이 아니라 **"그물처럼 촘촘히 엮여 있다"**.

이 높은 교차 밀도는 두 가지 실험 결과와 직접 연결된다.

(a) Greedy selection 의 효과. 46개 cycle 이 서로 많은 vertex 를 공유하므로, 소수의 cycle 만 선택해도 전체 note pool 의 대부분을 커버할 수 있다. 실제로 §2.7 에서 15개 cycle ($\sim 33\%$) 만으로 90% 보존도를 달성한 것이 이 구조적 중복성 덕분이다.

(b) Algorithm 1 의 교집합 규칙이 잘 작동하는 이유. Algorithm 1 은 "활성 cycle 들의 교집합"에서 음을 뽑는데 (§2.10 규칙 1), 만약 cycle 들이 vertex 를 거의 공유하지 않으면 교집합이 항상 빈 집합이 되어 이 규칙이 작동하지 않는다. 77% 의 쌍이 vertex 를 공유하므로 교집합이 비어있지 않을 확률이 높고, 교집합에서 뽑힌 음은 "여러 cycle 이 공통으로 중요하게 여기는 음"이라는 구조적 의미를 갖는다.

5. 확장 실험

> SHORT 미포함

본 연구는 원곡 재현(§3–§4)을 넘어 여러 방향의 확장 실험을 수행하였다. 수행된 확장 실험 결과를 정리한다. hibari의 **모듈 단위 생성** 구현 및 정량 평가는 §6에서 별도로 다룬다.

5.1 다른 곡으로의 일반화 — solari 실험 결과

> SHORT 미포함

본 연구의 §4.5 해석 8a("hibari의 FC 우위는 곡의 미학적 성격에서 기인")를 검증하기 위해, 같은 out of noise 앨범의 **solari**에 동일한 파이프라인을 적용하였다. solari는 hibari와 같은 앨범에 수록되어 있으나, 12개 pitch class를 모두 사용하는 반음계적(chromatic) 곡으로 성격이 대비된다. 실험 러너는 `run_solari.py`, `run_phase3_task39_wave2.py`, 결과는 `docs/step3_data/solari_results.json`, `docs/step3_data/solari_dft_gap0_results.json`.

solari 기본 정보: hibari와 동일한 GCD 기반 tie 정규화를 적용하였다. solari의 경우 GCD $=1$ (8분음표 단위)이므로 모든 duration이 1로 정규화되어, 결과적으로 note는 pitch 값만으로 구별된다 (pitch-only label). 이를 통해 $N=34$ 고유 (pitch, duration) 쌍, $T=224$ timesteps, 49개 고유 화음, tempo ≈ 29 BPM을 얻었다. (참고: GCD >1인 곡에서는 duration이 GCD 단위로 정규화되어 pitch 외에도 duration 정보가 보존된다. "pitch-only labeling"은 GCD $=1$인 특수한 경우에만 성립하는 표현이다.)

Algorithm 1 — 거리 함수 비교

거리 함수	cycle 수	density	JS (mean ± std)	JS min
frequency	22	0.070	0.063 ± 0.005	0.056
Tonnetz	39	0.037	0.063 ± 0.003	0.059
voice leading	25	0.043	0.078 ± 0.004	0.073
DFT (Task 39)	15	0.071	0.0824 ± 0.0029	0.0773

hibari에서는 DFT가 최우수(0.0213, §4.1)였지만, solari에서는 **frequency와 Tonnetz가 거의 동일** (0.063 vs 0.063)하고 DFT는 오히려 악화(0.0824)된다. 이는 §4.7.3에서 분석한 12-PC 구조의 한계와 일치한다. 12-PC에서는 Tonnetz 지름이 2로 짧아지고, DFT의 분해능도 제한된다(DFT에서 $K=15$로 축소). 분해능 제한의 원인: pitch class 12개를 모두 사용하는 곡(solari, aqua)에서는 indicator vector가 $(1,1,\ldots,1)$이 되어 DFT 계산 시 $k \neq 0$인 모든 계수가 이론적으로 0 — 12개 단위원 벡터의 합이 상쇄된다. **모든 PC subset이 거의 같은 DFT magnitude 벡터를 가지게 되므로 서로 다른 집합을 DFT 거리로 구별할 여지가 사라진다.** hibari처럼 7개 PC만 쓰는 경우에만 indicator vector가 sparse하여 DFT magnitude 차이가 선명히 유지된다.

Algorithm 2 — DL 모델 비교

설정	FC	LSTM	Transformer
binary JS	0.106	0.168	0.032
continuous JS	0.042	0.171	0.016

핵심 발견: solari에서는 Transformer가 최적. hibari와 정확히 반대 패턴이다. hibari에서 FC 최적 / Transformer 열등이었던 것이, solari에서는 Transformer가 FC의 2.6배 ($0.042 \to 0.016$, continuous 기준) 우위이다.

곡의 성격이 최적 모델을 결정한다는 가설을 지지. hibari (diatonic, entropy 0.974, 공간적 배치)에서는 시간 문맥을 무시하는 FC가 최적이었고, solari (chromatic, entropy 0.905, 선율적 진행)에서는 시간 문맥을 적극 활용하는 Transformer가 최적이다. 이 대비는 §4.5 해석 8을 실증적으로 뒷받침한다.

곡	PC 수	정규화 entropy	최적 거리	최적 모델	해석
hibari	7 (diatonic)	0.974	DFT	FC (2-layer)	공간적 배치, 시간 무관
solari	12 (chromatic)	0.905	frequency/Tonnetz 동등	Transformer	선율적 진행, 시간 의존
aqua	12 (chromatic)	—	Tonnetz	—(미실험)†	—
Bach Fugue	12 (chromatic)	—	Tonnetz	—(미실험)†	—

†: Algorithm 2 (DL 모델) 실험을 수행하지 않음. 거리 함수 비교(Algo1)만 실시.

Continuous overlap의 효과

solari에서도 continuous overlap은 이진 대비 개선을 보였다. Transformer 기준 binary JS 0.032 → continuous JS 0.016 (50% 감소). 이 개선폭은 hibari의 개선 F (57% 감소)와 비슷한 수준으로, continuous overlap의 효과가 곡의 특성에 독립적임을 시사한다.

이 대조군 실험은 §5.2에서 실제로 수행되었다.

5.2 클래식 대조군 — Bach Fugue 및 Ravel Pavane

> **SHORT 미포함**

직관적으로 "대위법처럼 선율 인과가 강한 곡에서 voice leading이 우위일 것"이라는 가설이 가능하다. 이를 검증하기 위해 바흐 Toccata and Fugue in D minor (피아노 솔로 편곡)와 라벨 Pavane pour une infante d funte에 동일한 Algo1 파이프라인을 적용하였다. 실험 러너는 `run_classical_contrast.py`, `run_phase3_task39_wave2.py`, 결과는 `classical_contrast_bach_postbugfix_results.json`, `classical_contrast_ravel_postbugfix_results.json`, `classical_contrast_dft_gap0_results.json`.

곡 기본 정보

곡	T (8분음표)	N (고유 note) †	화음 수
hibari (참고)	1088	23	17
solari (참고)	224	34	49
Ravel Pavane	548	**49**	230
Bach Fugue	870	**61**	253

† GCD(duration) = 1 (8분음표) 기준 tie 정규화 적용 후 고유 (pitch, dur) 쌍 수. 16분음표는 8분음표 단위 양자화로 소거되므로 최소 duration = 1.

거리 함수별 Algo1 JS

곡	frequency	tonnetz	voice leading	DFT	최적	비고
hibari	0.0344 [^freq_k1_gen]	0.0493	0.0566	0.0213	**DFT**	DFT 38.2% vs freq
solari	0.0634	0.0632	0.0775	0.0824 (K=15)	Tonnetz (frequency)	0.3%
Ravel Pavane	0.0337	0.0415	0.0798	0.0494 (K=37)	**frequency**	DFT도 악화
Bach Fugue	0.0902	0.0417	0.1242	0.0951 (K=30)	**Tonnetz**	DFT 크게 악화

해석 — 가설 기각: 선율 인과 ≠ voice leading 우위

> **SHORT 미포함**

Ravel Pavane: frequency 최적, 가설 불확인. N=49로 note 다양성이 높은 Ravel에서 빈도 역수 가중치가 가장 효과적이다. Tonnetz는 오히려 JS가 14.8% 악화된다. note 다양성(N=49)이 클수록 빈도 기반 분리자(frequency)가 강점을 갖는다는 가설이 지지된다.

Bach Fugue: Tonnetz 최적, voice leading 최악. 대위법(counterpoint)에서 "반음 이동 최소화" 보다 화성적 Tonnetz 공간 이동이 지배적임을 시사한다.

거리 함수 패턴 종합:

곡	PC 수	최적 거리	최적 모델 (Algo2)	해석
hibari	7 (diatonic)	**DFT**	FC (2-layer)	스펙트럼 구조 포착
aqua	12 (chromatic)	Tonnetz	—(미실험)†	화성적 공간 배치
Bach Fugue	12 (chromatic)	Tonnetz	—(미실험)†	화성적 공간 배치
Ravel Pavane	12, N=49	frequency	—(미실험)†	note 다양성 지배
solari	12, N=34	Tonnetz/frequency 동등	Transformer	12-PC Tonnetz 지름 2, 구별력 한계

†: Algorithm 2 실험을 수행하지 않음. Algo1(규칙 기반) 결과만 존재. hibari(N=23)와 note 어휘 크기가 달라 모델 재학습이 필요하며, 본 절의 목적이 거리 함수 선택 효과 검증에 있었기 때문이다.

현재 데이터에서 **hibari만 DFT가 최적**이며, 나머지 곡(solari/aqua/Bach/Ravel)은 Tonnetz 또는 frequency가 최적이다. 테스트한 5곡 중 voice leading이 최적인 곡은 없다. 이 패턴은 hibari의 고유 구조(7-PC diatonic, entropy 0.974, deep scale; §4.5)에서 DFT가 강점을 가지는 반면, 12-PC 또는 고다양도 note 분포에서는 다른 거리 함수가 더 유리함을 시사한다. 이 결과는 §5.3~§5.6에서 Tonnetz 기반 변주를 기본 설계로 잡는 근거 중 하나.

[^freq_k1_gen]: hibari frequency: 버그 수정 후 K=1 (cycle 붕괴). 생성 실험에는 DFT/Tonnetz를 사용.

5.3 위상 구조 보존 음악 변주 — 개요와 방향 A

> **SHORT 미포함**

본 §5.3~§5.6은 "위상 구조를 보존하면서 원곡과 다른 음악을 만드는 변주 실험"이다. 본 장의 기본 실험축은 **Tonnetz 기반**으로 두며, 이는 scale 제약(§5.5)과 화성적 이웃 관계(3도·5도)가 Tonnetz 공간과 자연스럽게 공명하기 때문이다. 이 판단이 DFT로도 유지되는지는 §5.6.2에서 별도로 검증한다.

지금까지의 모든 실험은 원곡의 pitch 분포를 가능한 한 재현하는 것을 목표로 했다. 본 절부터는 방향을 전환하여, **위상 구조를 보존하면서 원곡과 다른 음악**을 생성하는 문제를 다룬다.

아이디어

OM(시간 구조)은 그대로 보존하되, 각 cycle에 배정된 note를 새로운 note로 교체한다. 교체의 제약 조건은 다음과 같다:

1. 새 note 집합의 Tonnetz 거리행렬이 원본과 **up to permutation으로 유사**해야 한다 (정의 2.11)
2. 새 note 집합으로 구성한 거리행렬의 **persistence diagram이 원본과 유사**해야 한다 — Wasserstein distance (정의 2.10b, §2.10)로 **측정**
3. 새 note의 pitch 범위를 제어하여 얼마나 다른 곡을 원하는지를 조절한다

> **구현 주의사항 — 최적화 방식은 mode에 따라 다르다.**

> `tonnetz_nearest` (본 연구 채택): 각 원곡 note에 대해 pool에서 Tonnetz 거리 최근방을 직접 선택하는 그리디 매칭.

> `ascending` (비교군): Hungarian algorithm을 사용해 거리행렬 Frobenius 거리를 근사 최소화하는 permutation을 탐색.

> 일부 독자가 Wasserstein distance 기반 최적화로 오해할 수 있으나, 본 구현은 두 경우 모두 이산 매칭(discrete assignment)이며 Wasserstein은 사용하지 않는다.

코드: `note_reassign.py`, `run_direction_a_ablation.py`, 결과: `direction_a_ablation_results.json`, `note_reassign_wasserstein_results.json`.

평가 지표 정의.

note 오차: 정의 2.11의 $\mathrm{err}_{\mathrm{note}}(D, D')$. 원곡과 새 note 집합을 각각 pitch 오름차순으로 배열한 뒤, 정규화된 Tonnetz 거리행렬 $\tilde{D}$, $\tilde{D}'$의 Frobenius 거리 $\|\tilde{D} - \tilde{D}'\|_F$. 순열은 적용하지 않으며, pitch 오름차순 정렬이 고정 기준으로 사용된다. 값이 작을수록 두 note 집합의 거리 구조가 유사하다.

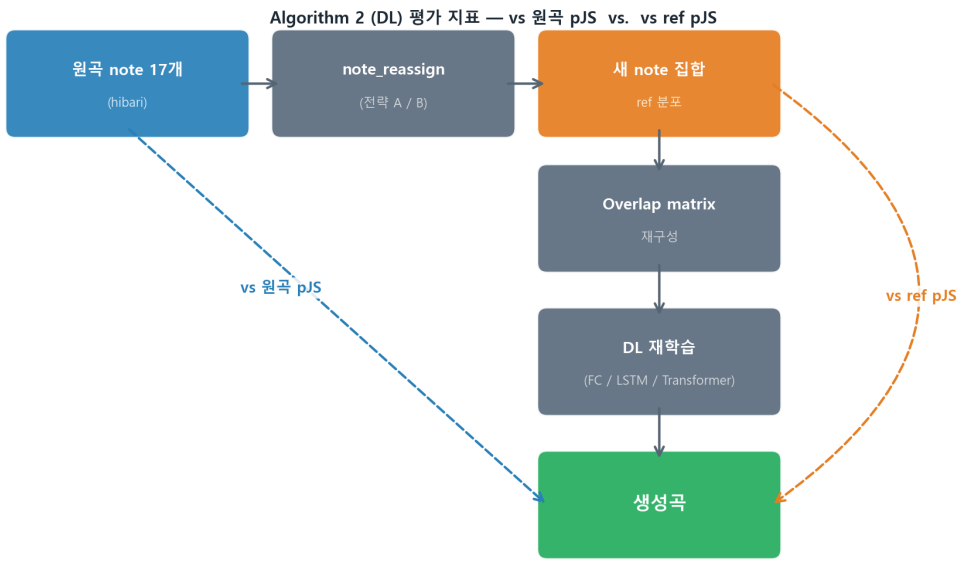
DTW: Dynamic Time Warping (DTW)은 두 pitch 시퀀스 사이의 거리를 측정한다. 본 연구의 구현(`sequence_metrics.py`의 `_pitch_sequence` + `dtw_pitch_distance`)에서 pitch 시퀀스는 **각 note의 onset 시점을 시간순으로 정렬하여 note당 pitch 값 하나씩을 뽑은 리스트**이다. 즉 duration은 고려하지 않으며, 길게 지속되는 음도 짧은 음도 시퀀스에 동일하게 한 번만 기여한다 (시간축 매 시점마다 활성 pitch를 다시 샘플링하는 방식이 아니다). 두 시퀀스 $x = (x_1, \ldots, x_T)$와 $y = (y_1, \ldots, y_S)$에 대해 DTW는 두 시퀀스의 모든 정렬(alignment) 경로 중 최소 비용을 갖는 것을 선택한다:

$$\mathrm{DTW}(x, y) = \min_{\mathrm{warpingpath}} \sum_{(i,j) \in \mathrm{path}} |x_i - y_j|$$

여기서 **warping path**는 (1) $(1, 1)$에서 (T, S)까지 단조 증가하며 (역방향 불가), (2) 각 단계에서 i 또는 j가 최대 1 증가하는 정렬 경로이다. DTW는 시퀀스 길이가 달라도 비교 가능하며, 일반 유클리드 거리와 달리 시간 축의 국소적 신축(stretching)을 허용하여 선율의 전반적인 윤곽(contour)을 비교한다. 원곡과 생성곡의 pitch 진행 패턴이 얼마나 다른지를 측정하는 선율 차이 지표로 사용된다. 값이 클수록 두 곡의 선율이 더 많이 다르다.

OM 사용 여부: 본 §5.3 실험에서 Algorithm 1 및 Algorithm 2 모두 **원곡 hibari의 OM**을 그대로 사용한다. 즉 OM의 구조(언제 어떤 cycle이 활성화되는지)는 원곡과 동일하다. 새 note로 재학습된 Algorithm 2는 이 OM을 입력으로 받아 새로운 pitch 시퀀스를 생성한다.

vs ref pJS: 생성곡의 pitch 분포를 재분배된 note 집합의 pitch 분포(reference)와 비교한 JS divergence. 여기서 reference는 note_reassign이 선택한 새 note N개의 분포이다. vs ref pJS가 낮을수록 DL 모델이 재분배 결과를 정확히 학습한 것이다. 세 지표의 관계는 아래 도식(그림 2.9.1)을 참조.



실험 설계 — 두 전략 비교 (N=20)

후보 pitch 집합을 랜덤하게 $n_{\mathrm{cand}} = 1000$개 생성하고, 아래 두 전략 중 하나로 최적 note 집합을 선택한다. 선택된 note 집합으로 Algorithm 1을 실행하여 생성 품질을 평가한다.

전략 A — 위치 기반 Tonnetz 최근방 매칭 (`tonnetz_nearest`):

$$\mathrm{new_pitches}[i] = \arg\min_{p \in \mathrm{pool}} d_{\mathrm{Tonnetz}}\left(\mathrm{orig_pitches}[i],\ p\right)$$

원곡의 i번째 note를 pool에서 Tonnetz 거리 최근방으로 직접 대체한다. 인덱스 i가 원곡-생성 1:1 대응(positional semantics)을 encode하므로, 국소적 화성 관계가 위치별로 보존된다.

두 전략 공통으로, 선택된 note 집합의 품질을 **note_dist_error**로 측정한다:

$$\text{note_dist_error} = \|\hat{D}_{\text{orig}} - \hat{D}_{\text{new}}\|_F$$

여기서 $\hat{D}$는 off-diagonal min-max 정규화 행렬이다:

$$\hat{D}_{ij} = \frac{D_{ij} - \min_{k \neq l} D_{kl}}{\max D - \min_{k \neq l} D_{kl}}, \quad \hat{D}_{ii} = 0$$

(비영값이 모두 동일한 상수이면 $\hat{D}_{ij} = 0.5$로 고정, 행렬 전체가 0이면 원본 반환.)

전략 B — 행렬 구조 기반 Hungarian 매칭 (`ascending`):

$$\pi^* = \arg\min_{\pi} \|P_\pi \hat{D}_{\text{new}} P_\pi^T - \hat{D}_{\text{orig}}\|_F$$

pool을 임의 순서로 배치한 뒤, 원곡의 note 거리행렬 $\hat{D}_{\text{orig}}$와 Frobenius 거리를 최소화하는 행 순열 π^*를 탐색한다 (내부적으로 동일한 $\hat{D}$ 정규화 적용). joint row-column permutation은 NP-hard이므로 Hungarian algorithm으로 근사한다. hibari의 note 수 $N = 17$은 exhaustive joint row-column permutation ($N! = 3.56 \times 10^{14}$)이 불가능하므로, 본 연구의 모든 실험에서 ascending 분기는 항상 이 Hungarian 근사 경로를 거친다.

두 전략을 동시에 적용하면 A의 위치 대응이 B에 의해 재배치되어 성능이 저하된다. 따라서 `matching_mode`에 따라 배타 선택한다.

pitch 범위는 wide(48–84) 및 vwide(40–88)를 비교하였다. Persistence Diagram Wasserstein distance를 추가 제약으로 활용한 실험 결과는 아래 별도 항목으로 제시한다.

Algorithm 1 결과 (두 전략 비교, wide 48–84, N=20)

실험 설정: hibari + Tonnetz, pitch_range=(48,84), N=20, n_candidates=1000

전략	pitch_js	transition_js	dtw	note_dist_error
baseline (원곡)	0.0212	0.2470	2.2186	—
B: 행렬 구조 기반 (ascending + Hungarian)	0.3815	0.6121	2.7918	4.6819
A: 위치 기반 (tonnetz_nearest)	0.2930	0.5488	2.6442	3.2132

두 전략의 note_dist_error는 동일 $\hat{D}$ 스케일 기준이므로 표에서 직접 비교 가능하다.

해석: 전략 A (tonnetz_nearest)가 전략 B (ascending) 대비 pitch_js $0.3815 \rightarrow 0.2930$ (23.2%)로 우세하다. **국소적 pairwise 화성 관계 보존(A)이, 대역적 거리 구조 정렬(B)보다 hibari의 위상 구조 보존에 더 효과적이다.** 이는 hibari의 note 수가 적고(N=17), Tonnetz 상 인접 관계가 곡의 화성 감각을 직접 지배한다는 §4.7(곡 고유 구조) 분석과 일치한다.

vwide(40–88) 검증 결과 전 조건에서 wide(48–84) 대비 pitch JS 열세가 확인되었다 (최선 vwide: 0.364 vs. 최선 wide: 0.293). pitch 범위를 지나치게 넓히면 Tonnetz 매칭의 근거가 되는 거리 구조에서 벗어나 생성 품질이 저하됨을 시사한다. 따라서 **wide(48–84)를 pitch 범위로 채택**하였다.

Wasserstein distance 제약 결과

두 전략 모두에 Persistence Diagram Wasserstein distance를 추가 제약으로 결합하였다. 구현 방식: Frobenius 거리(note_dist_error) 기준 상위 30개 후보를 먼저 걸러낸 뒤 Wasserstein distance로 재정렬. pitch 범위 40–88 고정 기준.

설정	전략 B pJS	전략 A pJS	Wass. dist (A)
제약 없음	0.607	0.363	—
Wass $\alpha = 0.3$	0.513	0.288	1.41
Wass $\alpha = 0.5$	0.513	0.288	1.41
Wass $\alpha = 1.0$	0.513	0.288	1.41
scale_major + Wass 0.5	0.115	0.120	0.35
제약 없음, wide (48–84)	—	0.291	—
Wass $\alpha = 0.3$, wide (48–84)	—	0.280	0.71

해석: 두 전략 모두에서 $\alpha \in \{0.3, 0.5, 1.0\}$ 세 값이 동일한 note 집합을 선택한다. topk 상위 후보들 간 Wasserstein distance 차이가 미미하여 계수 크기와 무관하게 같은 후보가 선택되기 때문이다. **전략 A가 전략 B 대비 pitch_js 약 40% 낮은 우위는 Wasserstein 제약 유무와 무관하게 유지된다.** scale_major 제약에서만 두 전략 결과가 수렴(0.120 vs 0.115)하는데, 음계 제약이 pitch 분포를 강하게 압축하여 매칭 전략 간 차이가 희석되기 때문이다. wide(48–84) 범위에서 전략 A + Wass $\alpha = 0.3$ 조합이 pJS 0.280으로 소폭 개선되나, N=5로 통계 유의성은 미검증이다. Wasserstein 제약을 topk 필터링 이전 단계에 적용하는 재설계가 후속 과제로 남는다.

Algorithm 2 (DL) 결과

재분배된 note 위에서 LSTM/Transformer를 재학습시킨 결과. **FC는 제외**하였는데, FC는 §4.5에서 이미 재분배 없는 설정의 최고 성능(JS 0.00035, DFT 조건)으로 검증되었으므로, 본 실험에서는 시퀀스 맥락을 활용하는 LSTM과 Transformer만 비교한다. 테이블 컬럼 의미:

vs 원곡 pJS: 생성곡의 pitch 분포를 원곡과 비교한 JS divergence (다를수록 높음)

vs 원곡 DTW: 생성곡과 원곡의 선율 차이 (다를수록 높음)

vs ref pJS: 생성곡의 pitch 분포를 재분배된 note 분포(reference)와 비교한 JS (낮을수록 학습 정확; 그림 2.9.1 참조)

설정	모델	vs 원곡 pJS	vs 원곡 DTW	vs ref pJS
baseline	Transformer	0.016	1.84	—
wide	LSTM	0.498	2.31	0.317
wide	Transformer	0.362	2.26	0.011

핵심 발견: Transformer는 재분배된 note의 분포를 거의 완벽하게 학습한다 (vs ref pJS 0.011). wide + Transformer 조합에서 원곡 대비 pJS 0.362 (완전히 다른 분포), DTW 2.26 ($+23\%$)의 선율 변화가 달성된다. 본 DL 실험은 전략 비교 이전에 수행된 ascending 매칭 기준이며, 최적 전략(tonnetz_nearest, 전략 A)에서의 DL 확장은 향후 과제이다.

5.4 방향 B: OM 시간 재배치

> **SHORT 미포함**

방향 A가 어떤 note를 바꾸는 것이라면, 방향 B는 언제 연주하는지를 바꾸는 것이다. OM의 **행(시점)**을 재배치하여 같은 cycle 구조를 다른 시간 순서로 전개한다.

3가지 재배치 전략

1. **segment_shuffle**: 동일 활성화 패턴이 연속되는 구간을 식별하고, 구간 단위로 순서를 셔플. 구간 내부 순서는 유지. 패턴이 바뀌는 시점을 경계로 삼으므로 구간 길이가 가변적이다 (고정 블록 크기를 사용하는 block_permute와 구별된다). hibari DFT(gap=0) 실측에서는 $T = 1088$에서 segment 수가 1012개(평균 길이 1.08)로, 시작/끝 일부를 제외하면 사실상 1-step 구간이 대부분이다.
2. **block_permute** (block size 32/64): 고정 크기 블록을 무작위 순열로 재배치.

3. **markov_resample** ($\tau = 1.0$): 원본 OM의 전이확률로부터 Markov chain을 추정하고, 온도 τ로 새 시퀀스를 생성 (§2.10). 여기서 **온도(temperature)** τ는 전이확률의 날카로움을 조절하는 파라미터이다. $\tau = 1$이면 원본 전이확률을 그대로 사용, $\tau > 1$이면 확률이 균등화되어 더 무작위적인 시퀀스가 생성되고, $\tau < 1$이면 확률이 날카로워져 가장 빈번한 전이만 반복된다.

코드: `temporal_reorder.py`, `run_phase3_task39_wave2.py`, `run_section66_dft_transformer.py`, 결과: `temporal_reorder_dl_results.json`, `temporal_reorder_dl_v2_results.json`, `temporal_reorder_fc_dft_gap0.json`, `temporal_reorder_lstm_dft_gap0.json`, `phase3_task39_4_lstm_summary.json`, `temporal_reorder_transformer_dft_gap0.json`.

세 모델의 시간 재배치 반응. FC, LSTM, Transformer는 시간 재배치에 대해 구조적으로 다른 반응을 보인다.

FC: 각 시점 t의 cycle 활성 벡터를 독립 처리하므로, OM 행 순서가 바뀌어도 **pitch 분포는 거의 불변**이다. Task 42 N=5 재검증(`temporal_reorder_fc_dft_gap0_n5.json`)에서 baseline/segment_shuffle/block_permute(32)의 pitch JS는 모두 0.000373 ± 0.000281로 동일했고, markov($\tau = 1.0$)만 0.001030 ± 0.000087로 소폭 상승했다. 반면 DTW는 baseline 대비 segment $+47.8\%$, block32 $+30.3\%$, markov $+34.1\%$로 크게 증가했다.

LSTM: 순환 구조(recurrence)로 과거 문맥을 누적하지만 positional embedding이 없어, 재학습 없이 입력 순서만 바꾸면 DTW 변화가 매우 작다. Task 39-4 집중 검증에서 retrain X 세 전략의 DTW 변화율은 모두 $\leq 0.5\%$였다(segment_shuffle $+0.11\%$, block_permute $+0.12\%$, markov $+0.36\%$). retrain O에서도 segment_shuffle은 -1.09%에 그쳤다. pJS는 baseline부터 $0.26 \sim 0.28$로 분포가 붕괴되어 재배치 전략 간 비교 의미가 약하므로 DTW만 서술한다.

Transformer: 명시적 PE로 시간 위치를 학습하므로, **PE 유무**가 재배치 효과의 핵심 변수다.

전략 (FC, DFT, N=5)	pitch JS (mean ± std)	DTW (mean ± std)	DTW 변화 (vs baseline)
baseline	0.000373 ± 0.000281	1.26735 ± 0.01837	—
segment_shuffle	0.000373 ± 0.000281	1.87275 ± 0.01846	$+47.8\%$
block_permute(32)	0.000373 ± 0.000281	1.65110 ± 0.02214	$+30.3\%$
markov ($\tau = 1.0$)	0.001030 ± 0.000087	1.69975 ± 0.01166	$+34.1\%$

즉 FC는 "같은 pitch 분포를 유지한 채 선율의 시간 순서만 바꾸는" 변주 생성에 구조적으로 적합하다. 이 특성은 §5.6.2에서 DFT-FC가 DFT-Transformer보다 낮은 ref pJS를 보인 배경의 일부를 이룬다. LSTM은 시간 재배치 반응이 제한적이므로, 이하 실험은 Transformer의 PE 유무 비교에 집중한다. Transformer의 DFT 전환 결과(Task 40)는 §5.6.2에서 통합 비교한다.

평가 지표 보충. **transition JS**는 두 곡의 note-to-note 전이 분포 간 JS divergence이다 (**bigram distribution**이라고도 한다: 시간적으로 인접한 두 note 쌍 (a, b)의 경험적 빈도 분포). 계산 방법: 곡에서 시간적으로 연속하는 두 note 쌍 (a, b)를 모두 추출하여 빈도를 세고, 전체 쌍 수로 정규화하여 확률 분포 P_{trans}를 구한다. 원곡과 생성곡 각각에서 이 분포를 구한 뒤, **두 분포 전체를 하나의 단일 JS divergence로 비교**한다: $D_{\text{JS}}(P^{\text{orig}}_{\text{trans}} \| P^{\text{gen}}_{\text{trans}})$. 이는 "transition matrix의 각 cell별로 JS를 구하는 것"이 아니라, 모든 (a, b) 쌍에 걸친 결합 분포를 한 번에 비교하는 것이다. pitch JS보다 높은 값을 가지는 것이 일반적인데, "어떤 음이 얼마나 쓰였는가"(pitch) 외에 "어떤 음 다음에 어떤 음이 오는가"(transition)를 추가로 평가하기 때문이다.

Transformer 결과 (PE 있음, 원본 학습)

> **SHORT 미포함**

전략	pitch JS	transition JS	DTW	DTW 변화
baseline	0.006	0.102	1.79	—
segment_shuffle	0.006	0.123	1.90	$+5.8\%$
block_permute(32)	0.006	0.104	1.86	$+3.6\%$
markov ($\tau = 1.0$)	0.008	0.104	1.84	$+2.7\%$

재배치 효과가 미약하다. segment_shuffle이 가장 큰 변화를 보이지만 DTW $+5.8\%$에 불과하다.

PE 제거 + 재학습 실험

> **SHORT 미포함**

Transformer에서 positional embedding(PE)을 제거하고, 재배치된 OM으로 재학습:

설정	pitch JS	transition JS	DTW	DTW 변화
noPE_baseline	0.011	0.128	1.85	—
noPE_segment_shuffle (retrain 없음)	0.011	0.149	1.87	$+1.0\%$
noPE_markov (retrain 없음)	0.010	0.138	1.87	$+0.9\%$
noPE+retrain segment_shuffle ★	0.173	0.399	2.22	$+21.7\%$
noPE+retrain markov ($\tau = 1.0$)	0.185	0.443	2.16	$+18.0\%$

noPE + markov (retrain 없음) 결과. `temporal_reorder_dl_v2_results.json` 기준 DTW $+0.9\%$, pitch JS 0.010 — segment_shuffle ($+1.0\%$)과 사실상 동일하다. "retraining 없이 재배치 전략이 바뀌어도 출력이 거의 변하지 않는다"는 패턴이 전략 유형에 무관하게 성립함이 실험적으로 확인된다.

방향 B의 딜레마: PE 제거 + 재학습에서 DTW가 $+21.7\%$까지 증가하여 선율이 확실히 바뀌지만, 동시에 pitch JS가 $0.011 \rightarrow 0.173$으로 **분포가 붕괴**된다.

> 약한 재배치 → pitch 보존, 선율 변화 없음
>
> 강한 재배치 → 선율 변화, pitch 분포도 붕괴

이 딜레마는 방향 B 단독으로는 "pitch 유지 + 선율 변화"를 동시에 달성하기 어려움을 의미한다.

n_cycles 강건성 확인. 동일 실험을 n_cycles=42 조건(캐시 α=0.5, lag=1에서 확보된 cycles)으로 재실행하였다 (`temporal_reorder_dl_results_c42.json`, `temporal_reorder_dl_v2_results_c42.json`). Transformer baseline pitch JS는 0.006으로 거의 동일하고, noPE+retrain segment_shuffle의 pitch JS 붕괴도 $0.173 \rightarrow 0.136$으로 동일한 방향을 유지한다. n_cycles=42 실험에서 LSTM을 함께 측정한 결과, baseline pitch JS가 $0.285 \rightarrow 0.323$ ($+13\%$)로 악화하였다 — 더 많은 cycle 차원을 50 epoch 내에 충분히 학습하지 못한 것으로 보인다. 어느 조건에서도 결론은 일관된다: 재배치 전략 단독으로는 DTW 개선이 $\pm 5\%$ 이내이며, noPE+retrain만이 유의미한 선율 변화($\geq +20\%$)를 유발하되 pitch 분포를 붕괴시킨다.

5.5 화성 제약 조건

> **SHORT 미포함**

방향 A의 note 재분배는 위상적 거리만 보존하므로, 결과가 **음악적으로 불협화**할 수 있다. 본 절은 화성(harmony) 제약을 추가하여 재분배의 음악적 품질을 개선한다.

3가지 화성 제약 방법

1. **scale 제약**: 새 note의 pitch class를 특정 스케일 (major, minor, pentatonic)에 한정. 허용 pool 크기가 줄어들지만 음악적 일관성이 보장된다.
2. **consonance 목적함수**: 재분배 비용에 평균 dissonance (정의 2.13)를 penalty로 추가한다 (cost 수식은 본 절 말미의 통합 비용 함수 참조; 구현은 `note_reassign.py`, `run_note_reassign_unified.py`).

여기서 $\mathrm{dist_err}$는 **원곡 note 쌍의 위상 거리와 재분배 note 쌍의 위상 거리 사이의 평균 절대 오차**로,

```
$\mathrm{dist\_err}(S_{\mathrm{orig}}, S_{\mathrm{new}})
= \frac{1}{\binom{n}{2}} \sum_{i<j}
|d(n_i^{\mathrm{orig}}, n_j^{\mathrm{orig}}) - d(n_i^{\mathrm{new}}, n_j^{\mathrm{new}})|$
```

로 정의된다. 값이 0이면 재분배가 모든 쌍 거리를 완전히 보존한 것이고, 클수록 위상 구조가 왜곡된 것이다. Algorithm 1 결과표의 **"note 오차"** 컬럼은 이 $\mathrm{dist_err}$ 수치이다. 단위는 사용한 거리 함수에 의존하며, 본 §5.5 표는 Tonnetz note metric 기준이므로 "Tonnetz 거리 단위" — 즉 hop 수로 해석한다 (**hop**은 Tonnetz 격자의 $(\pm 3, \pm 4, \pm 5)$ 인접 관계 하나를 따라 한 번 이동).

3. **interval structure 보존**: 원곡의 interval class vector (정의 2.12)와 새 note 집합의 ICV 차이를 penalty로 추가. **ICV 차이**는 두 집합의 ICV 벡터 사이의 L^1 노름으로 정의한다:

$$\mathrm{ICV_diff}(S_{\mathrm{orig}}, S_{\mathrm{new}}) = \|\mathrm{ICV}(S_{\mathrm{orig}}) - \mathrm{ICV}(S_{\mathrm{new}})\|_1 = \sum_{k=1}^{6} |\mathrm{ICV}(S_{\mathrm{orig}})[k] - \mathrm{ICV}(S_{\mathrm{new}})[k]|$$

값이 0이면 두 note 집합이 정확히 같은 interval class 분포를 갖는 것이다. ICV는 transposition (T_k) 하에서 불변이므로, 이 제약은 조성 이동은 허용하되 화성 구조 자체는 보존하는 재분배를 선호하게 한다.

통합 비용 함수

위 세 제약(scale, consonance, interval)은 서로 배타적이지 않으며 동시 적용 가능하다. 전체 비용 함수는

$$\mathrm{cost}(S_{\mathrm{new}}) = \alpha_{\mathrm{note}} \cdot \mathrm{dist_err} + \mathbb{1}[\mathrm{use_diss}] \cdot \beta_{\mathrm{diss}} \cdot \mathrm{diss} + \mathbb{1}[\mathrm{use_int}] \cdot \gamma_{\mathrm{int}} \cdot \mathrm{ICV_diff}$$

이며, scale 제약은 **후보 pool 자체를 축소**하는 방식으로 작용하므로 cost 수식에는 등장하지 않는다. 아래 Algorithm 1 결과표의 각 행은 다음 mode에 대응한다:

행 이름	use_scale	use_diss	use_int	실효 cost
baseline				$\alpha_{\mathrm{note}} \cdot \mathrm{dist_err}$
scale_major / minor / penta	(pool 축소)			$\alpha_{\mathrm{note}} \cdot \mathrm{dist_err}$
consonance 단독				$\alpha_{\mathrm{note}} \cdot \mathrm{dist_err} + \beta_{\mathrm{diss}} \cdot \mathrm{diss}$
interval 단독				$\alpha_{\mathrm{note}} \cdot \mathrm{dist_err} + \gamma_{\mathrm{int}} \cdot \mathrm{ICV_diff}$

본 실험 기본값: $\alpha_{\mathrm{note}} = 0.5$, $\beta_{\mathrm{diss}} = 0.3$, $\gamma_{\mathrm{int}} = 0.3$. 결과: `note_reassign_harmony_results.json`.

Algorithm 1 결과

설정	note 오차	consonance score	scale match	PC 수
baseline (제약 없음)	4.35	0.412	0.70 (C major)	10
scale_major ★	3.52	0.361	1.00 (C major)	7
scale_minor	3.68	0.363	1.00 (E major)	7
scale_penta	4.32	0.213	1.00 (C major)	5
consonance 단독	4.35	0.412	0.70	10
interval 단독	4.35	0.437	0.64	11

핵심 발견:

1. **scale_major가 가장 효과적**: note 거리 오차가 baseline $4.35 \to 3.52$로 19% 감소하면서 동시에 scale match 1.00, dissonance $0.412 \to 0.361$로 화성적 품질도 개선된다. C major로 고정되므로 원곡(hibari)의 조성과 일치한다.
2. **scale_penta가 가장 낮은 dissonance**: 0.213으로 baseline의 절반 수준. 5음만 사용하므로 불협화 자체가 구조적으로 억제된다.
3. **consonance 단독은 무효과**: 비용에 추가해도 최적 후보가 바뀌지 않았다. dissonance penalty가 거리 보존 제약에 비해 너무 약한 것으로 추정된다. ($\alpha_{\mathrm{note}}, \beta_{\mathrm{diss}}$) = (0.5, 0.3)은 두 항의 스케일 균형을 위한 휴리스틱 기본값이며, grid search는 후속 과제이다.

> **각주 — $(\alpha_{\text{note}}, \beta_{\text{diss}}) = (0.5, 0.3)$ 근거.** $(\alpha_{\text{note}}, \beta_{\text{diss}}) = (0.5, 0.3)$은 dist₩_err (Tonnetz hop 수)와 diss ($[0,1]$ roughness) 두 항의 스케일 균형을 위한 **휴리스틱 기본값**이다. 사후 3×3 grid search (N=5, wide range, consonance 단독 고정)를 수행한 결과 9 셀 중 7 셀이 동일한 Pareto 후보로 수렴했고, 나머지 2 셀($\beta_{\text{diss}} = 0.1$ & $\alpha_{\text{note}} \geq 0.5$)은 pitch JS를 -14.9% 개선하지만 consonance score는 $+17.9\%$ 악화시켰다. 동시 개선 조합은 존재하지 않아 기본값을 유지한다 (`consonance_grid_search_results.json`). 체계적 grid search는 후속 과제이다.

> **`consonance score` 정의.** `consonance score` 컬럼은 생성된 곡 G의 **시점별 dissonance 시간 평균**: $\text{cons_score}(G) = \frac{1}{T}\sum_{t=1}^{T} \text{diss}(\text{chord}_t(G))$. 각 설정이 만든 다른 곡에서 개별 측정되므로 행마다 값이 다르다. 행 이름 `consonance` 은 '최적화 비용에 $\beta_{\text{diss}} \cdot \text{diss}$만 추가(scale/interval 없음)'한 설정 이름이다.

Algorithm 2 (Transformer) 결과

(화성 제약 실험에서는 Algorithm 2로 **Transformer**를 사용한다. FC는 §5.4에서 시간 재배치에 무관함이 확인되었으며, note 교체 효과는 Algorithm 1에서 직접 검증된다. FC의 DL 성능 비교는 §5.8.2에서 DFT 기반 실험으로 별도 수행된다.)

val_loss 정의 (본 표에서 처음 등장). Algorithm 2 신경망 학습 시, 전체 시점의 20%를 검증 데이터로 분리하여 학습 중 한 번도 보지 않은 입력–출력 쌍에 대해 §2.8의 multi-label BCE 손실(정의 2.10)을 계산한다. 이 값이 낮을수록 모델이 OM → note 시퀀스 매핑을 더 잘 일반화했음을 의미한다. train loss가 아닌 **val_loss를 사용하는 이유**는 과적합(overfitting)을 배제하고 실제 일반화 성능만 비교하기 위함이다.

"original" 행의 의미. 아래 표의 `original` 행은 **재분배를 적용하지 않은 원곡 hibari의 OM을 그대로 Transformer에 학습시킨 baseline**이다. 입력 OM은 본 §5.5 실험 전체에서 **이진 OM(binary)** 을 사용한다 (continuous OM 실험은 §5.6에서 별도로 다룬다). 이 행의 `ref pJS`가 "—"로 표시된 것은 재분배된 reference 분포가 존재하지 않기 때문이다 (원곡 자체가 reference이므로 vs 원곡 pJS와 vs ref pJS가 같은 값이 되고, 중복을 피하기 위해 후자만 "—"로 표기).

설정	vs 원곡 pJS	vs 원곡 DTW	val_loss	vs ref pJS
original	0.009	1.80	0.524	—
baseline (제약 없음)	0.600	3.46	0.497	0.007
scale_major ★	0.097	2.35	0.492	0.003
scale_penta	0.259	3.37	0.487	0.009

scale_major + Transformer 조합은 원곡 대비 pJS 0.097 (JS 최댓값 $\ln 2 \approx 0.693$의 14.0% — 완전히 다른 분포가 아닌 "의미 있는 차이"), DTW 2.35 ($+31\%$, 다른 선율), ref 대비 pJS 0.003 (재분배된 note 분포를 거의 완벽 학습)으로, **위상 보존 + 정량화 가능한 차이 + 화성적 일관성**의 균형이 가장 좋다. 각 축의 근거는 다음과 같다 —

> **위상 보존**: §5.3 "OM 사용 여부" 정책에 따라 §5.5 실험 전체에서 **원곡 hibari의 OM을 그대로 사용**한다. 즉 cycle 활성 구조가 원곡과 동일하므로 위상적 구조는 정의상 보존된다. 추가로 Algorithm 1 결과표의 dist_err 3.52 (scale_major)는 pair-wise 거리 구조도 baseline 4.35 대비 19% 개선되어 보존됨을 수치적으로 뒷받침한다.
>
> **정량화 가능한 차이**: DTW 2.35 ($+31\%$ vs original 1.80) 및 pJS 0.097 ($\ln 2 \approx 0.693$의 14.0%)이 근거. DTW는 **선율 윤곽**의 차이를, pJS는 **pitch 분포**의 차이를 독립적으로 수량화한다 — 이 두 값이 동시에 양의 크기를 보이므로 "원곡과 다름"을 두 축에서 검증된 수치로 주장할 수 있다.
>
> **화성적 일관성**: scale_major 제약으로 Algorithm 1 결과표의 scale match $= 1.00$ (C major, 원곡 hibari의 조성과 일치), 그리고 consonance score 0.361 (baseline 0.412 대비 dissonance -12%) 개선이 근거이다.

두 평가 전략 (A / B) — 그림의 개념적 구분. 재분배 실험에서는 하나의 생성 결과에 대해 두 가지 비교 대상이 존재하며, 각각이 답하는 질문이 다르다.

> **전략 A (vs 원곡 pJS / 원곡 DTW)**: 생성곡을 **원곡 hibari**의 pitch 분포·선율과 직접 비교한다. "재분배 이후 생성된 결과가 원곡과 얼마나 다른가"를 측정 — 창의성/변주의 정량화가 목적이므로 **값이 클수록 좋다** (단, $\ln 2$의 10–30% 정도가 "의미 있는 차이"의 합리적 범위이며, 너무 크면 분포 붕괴).
>
> **전략 B (vs ref pJS)**: 생성곡을 **재분배 후의 reference 분포** (scale_major 등으로 note를 바꾼 결과의 pitch 분포)와 비교한다. "Transformer가 재분배된 note 분포를 얼마나 정확히 학습했는가"를 측정 — 학습 정확도가 목적이므로 **값이 작을수록 좋다**.

두 전략은 상반된 목표를 가지므로 단일 지표로는 실험을 해석할 수 없다 — 전략 A가 크고(변주) 전략 B가 작은(정확도) 설정이 "의미 있는 변주를 정확히 학습"한 이상적 결과이며, scale_major가 바로 이 두 조건을 동시에 만족한다 (A: pJS 0.097, B:

ref pJS 0.003).

Task 42 보강 (DFT, N=5) — FC/LSTM ±std 검증. §5.5 본문 판정은 위 Tonnetz-Transformer 표를 기준으로 유지하되, DFT 조건 보강 실험에서 FC/LSTM의 반응을 $N=5$ 반복으로 재검증했다 (`harmony_fc_dft_gap0_n5.json`, `harmony_lstm_dft_gap0_n5.json`).

모델 (DFT)	설정	vs 원곡 pJS (mean ± std)	vs 원곡 DTW (mean ± std)	vs ref pJS (mean ± std)	val_loss (mean ± std)
FC	original	0.000867 ± 0.000258	1.28355 ± 0.03013	0.000867 ± 0.000258	0.035374 ± 0.004484
FC	baseline	0.448155 ± 0.010002	3.88670 ± 0.03772	0.002839 ± 0.003212	0.036084 ± 0.004821
FC	**scale_major**	0.322445 ± 0.006974	4.73965 ± 0.06917	0.005131 ± 0.002600	0.039286 ± 0.007003
FC	scale_penta	0.430169 ± 0.011456	7.95025 ± 0.12268	0.004970 ± 0.004608	0.036292 ± 0.004864
LSTM	original	0.278054 ± 0.004922	2.37875 ± 0.18954	0.278054 ± 0.004922	0.401009 ± 0.005350
LSTM	baseline	0.629544 ± 0.026339	4.93145 ± 0.49111	0.283845 ± 0.001485	0.395173 ± 0.011659
LSTM	**scale_major**	0.521135 ± 0.016792	6.16320 ± 0.18595	0.283212 ± 0.001186	0.397168 ± 0.007522
LSTM	scale_penta	0.561428 ± 0.027632	8.43060 ± 0.90457	0.283358 ± 0.001778	0.391390 ± 0.016836

FC는 재분배 분포 학습(vs ref pJS)은 양호하지만, scale_major에서도 DTW가 크게 증가해 선율 균형이 흔들린다. LSTM은 세 지표(vs 원곡 pJS, vs ref pJS, DTW) 모두 열세다. 따라서 "scale_major가 최적"이라는 §5.5의 핵심 결론은 유지되며, 본 표는 그 결론의 분산(uncertainty)까지 함께 제시한다.

5.6 A+B 결합 + continuous overlap — 최종 통합 실험

> **SHORT 미포함**

선택지 (C)에 따라, 본 절은 **Tonnetz 성공 사례(§5.6.1)**와 **DFT 전환 실패 사례(§5.6.2)**를 병기하고, 이를 하나의 메타 결론(§5.6.3)로 통합한다. 사용 데이터는 `combined_AB_results.json`(Tonnetz, 2026-04-11)과 `combined_AB_dft_gap0.json`(DFT, Task 40)이다.

5.6.1 Tonnetz 기반 통합 실험 — 성공 사례 유지

Tonnetz 기준 `major_block32`는 여전히 본 장의 대표 성공 사례다.

설정 (Tonnetz)	vs 원곡 pJS	vs 원곡 DTW	vs ref pJS	scale match
major_block32	0.2696 ± 0.111	3.620 ± 0.818	0.00710 ± 0.00308	1.00
major_segment_shuffle	0.1047	2.4413	0.0035	1.00
major_markov	0.1208	2.5000	0.0158	1.00

`major_block32`는 ref pJS(학습 정확도)와 원곡 대비 변화량(변주성)을 동시에 만족한다. 즉 "위상 보존 + 선율 변화 + 화성 일관성"의 3축 균형이 성립한다.

> **데이터 출처.** `post_bugfix_tonnetz_major_block32_results.json` ($N=10$). DFT 전환 검증은 §5.6.2 참조.

5.6.2 DFT 전환 탐색 — 실패 사례

Task 40에서 동일 조합을 DFT로 전환해 재실험한 결과, Tonnetz 성공 설정은 재현되지 않았다.

비교 설정	vs ref pJS	vs 원곡 pJS	vs 원곡 DTW	해석
Tonnetz Transformer `major_block32`	0.00710 ± 0.00308	0.2696 ± 0.111	3.620 ± 0.818	기준(성공)

DFT Transformer `major_block32`	0.01622 ± 0.00267	0.2689 ± 0.0736	3.105 ± 0.428	ref pJS **2.28배 악화**

DFT 조건에서는 두 모델 모두 `orig_continuous`가 최적이었다(Transformer ref pJS 0.0037, FC ref pJS 0.0007). 즉 scale-major 변주의 이득이 사라지고, §5.5의 "화성 제약 + 변주" 설계가 붕괴한다.

5.6.3 메타 통찰 — 거리 함수 × 음악적 목적

§5.6.1과 §5.6.2를 함께 보면, 거리 함수 선택은 "무조건 우열"이 아니라 **목적 적합성** 문제임이 분명하다.

> **구조 정밀도 목적**(원곡 재현, cycle 분리, 모듈 탐색): DFT가 강점일 수 있다(§4, §5.7, §6).
>
> **화성 정합성 목적**(scale 제약 변주, 조성 일관성 유지): Tonnetz가 강점이다(§5.5, §5.6.1).

여기서 **cycle 분리**는 서로 다른 H_1 cycle이 거리 공간에서 뚜렷이 구별되어 PH가 별개의 generator로 포착하는 능력을 의미한다 (§4.7.3 Tonnetz 지름 한계 참조). 12-PC 곡에서는 Tonnetz 지름이 ≤ 2로 제한되어 cycle들이 밀집·겹치는 반면 (§4.7.3), DFT는 $k=5$ 계수의 spectral signature를 통해 서로 다른 pitch class 집합을 다른 magnitude 벡터로 사상하므로 cycle 간 구조적 차별화가 강하다.

FC의 pitch 분포 유지 특성 — §5.4 재발견. §5.4 재검증에서 FC는 OM 재배치 하에 pitch 분포를 거의 유지하면서도 DTW를 $+30 \sim +48\%$ 변화시켰다. 즉 FC는 "같은 pitch 분포 + 다른 선율 순서" 변주 생성에 구조적으로 유리하며, 이는 §5.6.2에서 DFT-FC의 낮은 ref pJS를 부분 설명한다. 그럼에도 §5.6 전체 최적이 Tonnetz-Transformer로 남는 이유는, scale 제약의 핵심 이득이 OM 재배치가 아니라 **note 선택 단계**에서 발현되기 때문이다. 이는 "거리 함수 × 목적 정합성"이라는 본 절의 메타 논지를 강화한다.

같은 결론은 §5.9에서도 반복된다. complex 모드는 Tonnetz에서는 유효하지만 DFT에서는 오히려 악화된다. 따라서 본 연구의 메타 기여는 다음과 같이 정리된다: **단일 거리 함수로 모든 음악적 목적을 커버할 수 없으며, 목적과 거리 함수의 정합성을 먼저 설계해야 한다.**

5.7 DFT Hybrid의 α grid search — 실험 결과

DFT hybrid 거리

$$d_{\text{hybrid}} = \alpha \cdot d_{\text{freq}} + (1-\alpha) \cdot d_{\text{DFT}}$$

에서 $\alpha \in \{0.0, 0.1, 0.25, 0.3, 0.5, 0.7, 1.0\}$에 대해 $N=20$ 반복 실험으로 최적 혼합 비율을 정량화한다. 실험 러너는 `run_dft_phase2_gap0_suite.py` (Task A10-a), 결과는 `alpha_grid_dft_gap0_results.json`.

d_{DFT} 내부 파라미터 고정값. 본 표의 모든 α 행은 d_{DFT} 내부의 $(w_o, w_d) = (0.3, 1.0)$ 을 공통으로 사용한다. 두 값은 각각 §4.1a octave grid search (Task 27, `ow_gridsearch_dft_results.json`, DFT 조건 $w_o = 0.3$ 최적) 와 §4.1b duration weight grid search (Task 26, `dw_gridsearch_dft_results.json`, DFT 조건 $w_d = 1.0$ 최적) 로 확정된 것이다. 이 두 값은 Tonnetz 조건의 최적 ($w_o = 0.5$ / $w_d = 0.3$) 과는 다르며, DFT 전용 grid search 결과임을 강조한다. $\alpha = 1.0$ (pure frequency) 행에서는 d_{DFT} 가 쓰이지 않으므로 w_o, w_d 가 적용되지 않는다.

α	K (cycle 수)	w_o	w_d	JS (mean ± std)	비고
0.0 (pure DFT)	**1**	0.3	1.0	0.0728 ± 0.00432	K=1, degenerate
0.1	13	0.3	1.0	0.01602 ± 0.00204	
0.25	**14**	0.3	1.0	0.01593 ± 0.00181	**최적**
0.3	16	0.3	1.0	0.02025 ± 0.00134	
0.5 (기존 기본값)	19	0.3	1.0	0.01691 ± 0.00143	
0.7	24	0.3	1.0	0.03140 ± 0.00270	

1.0 (pure freq)	1	—	—	0.03386 ± 0.00186	K=1, degenerate (d_{freq} 단독)

해석.

DFT hybrid 에서는 **양 끝점이 모두 degenerate**하다. $\alpha = 0.0$ (pure DFT)과 $\alpha = 1.0$ (pure freq) 모두 $K = 1$이 된다. §4.7에서 서술한 hibari의 7-PC maximal evenness + 균등 pitch 분포는 순수 빈도 거리뿐 아니라 순수 DFT 거리에서도 note 간 거리 행렬이 근-균일해지므로, 양 끝점에서 장수명 H_1 cycle이 잔존하지 않는다. $\alpha \in [0.1, 0.5]$ 구간이 유효 범위이며, 이 구간 내에서 $K = 13 \sim 19$의 PH 위상 구조가 출현한다.

$\alpha = 0.25$가 JS $= 0.01593 \pm 0.00181$, $K = 14$ 로 가장 낮다. $\alpha = 0.5$는 JS $= 0.01691$로 $\alpha = 0.25$ 대비 $+6.2\%$ 높지만 $K = 19$의 풍부한 위상 구조를 제공한다.

최종 α 선택. Tonnetz 기반 §5.9 (구 complex 실험)에서는 JS를 소폭 양보하고 $K = 42$를 확보하기 위해 $\alpha = 0.5$를 선택하였으나, DFT hybrid 에서는 $\alpha = 0.25$ 선택 시 JS 최저를 유지하면서 $K = 14$를 얻고, $\alpha = 0.5$ 시 $K = 19$에 불과하여 Tonnetz 대비 K 차이가 작다. 따라서 DFT hybrid 최적 설정에서는 $\alpha = 0.25$**를 채택**한다 (`config.py MetricConfig.alpha`). 이 결과는 §5.8.1의 per-cycle τ_c 실험에서도 재확인된다 — $\alpha = 0.25$ 조건의 per-cycle τ_c (Task A-3, `percycle_tau_dft_gap0_alpha_grid_results.json`)가 JS $= 0.00902$으로 신기록을 달성하며, $\alpha = 0.25$**가 binary OM(본 절)과 per-cycle τ_c (§5.8.1) 양쪽 모두에서 최적임이 이중으로 검증된다**.

> ※ **§5.7 α grid의 범위:** 본 절 실험은 PH 계산 단계에서 각 α 값으로 위상을 새로 계산하였다 (all-from-scratch, Task A10-a 러너). $\alpha = 0.0$ (pure DFT)에서 $K = 1$ degenerate가 확인되므로, PH용 α 로는 $0.1 \le \alpha \le 0.5$ 구간이 적합하다. §5.9에서는 complex 모드를 DFT 조건으로 검증하였다.

5.8 Continuous overlap의 정교화 — 실험 결과

> **SHORT 미포함**

DFT 기준 §4.4 결과는 **Binary OM이 최적**(JS 0.0157)이며, continuous→단일 τ 이진화는 $\tau = 0.3/0.5/0.7$에서 각각 $0.0360/0.0507/0.0449$로 모두 열세였다(세 임계값 중에는 $\tau = 0.3$이 최저). 본 절은 그 단일 임계값 한계를 넘어, 세 가지 정교화 실험을 **DFT 기반 조건** (거리 함수 $=$ DFT, $\alpha = 0.5$, $w_o = 0.3$, $w_d = 1.0$, $\text{min_onset_gap} = 0$) 에서 수행하고 그 결과를 보고한다. 실험 러너는 `run_dft_phase2_gap0_suite.py`, 결과는 `docs/step3_data/percycle_tau_dft_gap0_results.json` 와 `soft_activation_dft_gap0_results.json`. DFT 기반에서 Algorithm 1 이 발견하는 cycle 수는 $K = 19$ 이다.

5.8.0 Cycle별 활성화 프로파일의 다양성

> **SHORT 미포함**

Per-cycle τ 실험에 앞서, 왜 cycle마다 서로 다른 임계값이 필요한지를 직관적으로 설명한다.

각 cycle의 연속 활성화 값 $O_{\text{cont}}[t, c] \in [0, 1]$은 "이 cycle을 구성하는 note들이 시점 t에서 얼마나 많이, 얼마나 드물게 울리는가"를 나타낸다. 이 값은 cycle의 음악적 역할에 따라 극적으로 다른 분포를 보인다.

Cycle A형 (지속 활성형, 예: 저번호 cycle). hibari의 inst 1, inst 2는 동일 음역에서 유사한 선율 패턴을 연주한다. 두 악기 모두에서 지속적으로 반복 등장하는 음들로 구성된 cycle은 거의 전 구간에서 약하게 활성화되며 O_{cont} 값이 안정적으로 낮다 (예: 0.15–0.30). 이 cycle이 핵심 diatonic 음(으뜸음·5음 등 골격음)을 중심으로 구성된다는 해석은 정성적 추정으로, 정량 검증은 후속 과제이다. 균일 임계값 $\tau = 0.35$를 쓰면 이 cycle은 대부분의 시점에서 비활성으로 처리되어 지속적 선율 배경의 역할이 무시된다. 반면 $\tau = 0.10$을 쓰면 이 cycle이 거의 항상 활성화되어 생성음악에도 지속적 배경 음형이 나타난다.

Cycle B형 (색채음형, 예: 고번호 cycle). 특정 화성적 색채를 표현하는 note(예: 임시표 있는 음)는 원곡에서 드물게 등장하고 나타날 때는 선명하게 나타난다고 관찰된다. 이 note들로 구성된 cycle은 대부분의 구간에서 $O_{\text{cont}} \approx 0$이다가 해당 구간에서만 $O_{\text{cont}} \approx 0.6$–$0.9$로 급상승한다. 균일 $\tau = 0.35$를 쓰면 이 cycle이 상승 구간에서 올바르게 활성화된다. 하지만 더 높은 $\tau = 0.60$–0.70을 쓰면 "확실히 의도된 구간"만 활성화되어 색채가 더 선명해진다.

즉, τ를 cycle마다 다르게 설정해야 각 cycle의 음악적 기능이 제대로 표현된다. $\tau = 0.35$는 모든 cycle에 대한 "그나마 나쁘지 않은" 타협점일 뿐이다.

5.8.1 Per-cycle 임계값 최적화

방법 (공통). Cycle c를 고정 순서(cycle $0 \to K-1$)로 순회하며, 나머지 cycle의 τ를 고정한 채 τ_c를 $\{0.1, 0.2, 0.3, 0.35, 0.4, 0.5, 0.6, 0.7\}$ 중 JS 최소값으로 결정하는 **1-pass greedy coordinate descent**를 수행한다. 초기값은 전역 $\tau = 0.35$이며, 순회 순서(cycle $0 \to K-1$)와 greedy 평가 시드에 의존하므로 전역 최적을 보장하지 않는다. 본 절은 두 단계로 나뉜다: **Phase 1** ($\alpha = 0.5$, $K = 19$, Task A8) 과 **Phase 2** ($\alpha = 0.25$, $K = 14$, Task A-3).

최적 au_c 분포 ($K = 19$): $au = 0.7$ 이 5 개 (26.3%), $au = 0.1$ 이 3 개 (15.8%). 양 극단인 $au = 0.7$ (B형: 색채 cycle) 과 $au = 0.1$ (A형: 배경 cycle) 에 최적값이 집중되어, 단일 au 로 두 유형을 동시에 만족시킬 수 없음을 확인한다. per-cycle au_c 는 "cycle-homogeneous 가정" 을 완화하여 $+58.7\%$ 를 얻는다.

Phase 2 — 결과 ($N = 20$, $\alpha = 0.25$, $K = 14$, `percycle_tau_dft_gap0_alpha_grid_results.json`, Task A-3). 초기 보고값(0.01156, 다른 시드 셋)에서 α-grid 재탐색 시드 셋으로 갱신된 최종 값이다.[^seed-note2]

$\alpha = 0.5$ ($K = 19$) 에서 $\alpha = 0.25$ ($K = 14$) 로 교체하면 cycle 수와 구성 자체가 달라지므로 greedy 탐색을 처음부터 재실행하였다.

설정	α	K	JS (mean ± std)	vs Phase 1 기준	Welch p
Binary OM (참조)	0.25	14	0.01586 ± 0.00152	$+6.5\%$	—
per-cycle au_c (Phase 1)	0.5	19	0.01489 ± 0.00143	—	—
per-cycle τ_c ★ 신기록	0.25	14	0.00902 ± 0.00170	-39.4%	$\mathbf{p} = 1.44 \times 10^{-15}$

최적 τ_c 프로파일 ($K = 14$): $[0.7, 0.7, 0.3, 0.7, 0.7, 0.4, 0.1, 0.3, 0.4, 0.1, 0.5, 0.35, 0.4, 0.3]$. Phase 1 과 마찬가지로 A형/B형 양 극단에 집중된다.

[^seed-note2]: 이전 보고값(0.01156 ± 0.00147)은 다른 초기 시드 셋에서의 유효 샘플이며, 신규 최저(0.00902)와 1.5σ 이내 거리이다. greedy coordinate descent는 초기 τ 벡터(전역 0.35), 순회 순서(cycle 0→K 1), greedy 평가 시드에 의존하므로 전역 최적을 보장하지 않는다.

α-grid 결과 ($N = 20$, DFT, per-cycle τ_c). $\alpha = 0.25$ 조건이 $\alpha = 0.50$ 대비 JS $+71.48\%$ 열세를 역전한다 (Welch $p = 1.44 \times 10^{-15}$).

α	JS (mean ± std, $N = 20$)	$\alpha = 0.50$ 대비
0.25	0.00902 ± 0.00170 ★	-41.7%
0.30	0.01032 ± 0.00121	-33.3%
0.50	0.01547 ± 0.00125	—

Algo1 연구 전체 최저 갱신 (Phase 2). Per-cycle τ_c ($\alpha = 0.25$) 는 JS $= 0.00902 \pm 0.00170$ 로, Phase 1 최저 0.01489 대비 -39.4%, DFT binary 0.01586, Tonnetz complex 최저 0.0183 (§5.9) 를 모두 하회하며 **Algorithm 1 측면 본 연구 전체 최저**를 확정한다. 이 수치는 $\log 2 = 0.693$ 대비 1.30% 이다. **이중 확인:** §5.7 binary OM grid search 도 $\alpha = 0.25$ 를 최적으로 선택하였으며, 본 Phase 2 per-cycle τ_c 에서도 동일하게 $\alpha = 0.25$ 가 최적임이 독립 재검으로 확인된다.

5.8.2 Continuous overlap을 직접 받아들이는 Algorithm 2

per-cycle τ 는 §5.8.2 에 적용되지 않는다 — 표가 §4.3 과 같은 이유. 본 절의 실험은 continuous OM 을 binarize 하지 않고 그대로 Algorithm 2 (DL) 에 입력하는 것이므로 threshold τ 자체가 존재하지 않는다. per-cycle τ_c (§5.8.1) 는 Algorithm 1 의 cycle-activation 판단 임계값을 cycle 별로 분리 최적화하는 기법으로, 이 기법을 Algorithm 2 에 결합하려면 "per-cycle τ_c 로 binarize 한 OM" 을 Algorithm 2 에 입력하는 별도 실험이 필요하다. 본 §5.8.2 는 그 확장 이전 단계, 즉 **순수 continuous OM 직접 입력** vs **§2.5a 정의 standard 이진 OM 입력** 의 비교에 해당하며, 따라서 아래 표의 수치는 §4.3 의 **DFT baseline** ($\alpha = 0.5$) DL 비교 표와 **동일한 실험·동일한 수치** (`soft_activation_dft_gap0_results.json` = `dl_comparison_dft_gap0_results.json` 내 soft 부분과 공유) 이다. 본 절은 §5.8 "continuous overlap 정교화" 라는 문맥에서 그 핵심 발견(FC-cont 우위) 을 재제시하고,

per-cycle 은 §5.8.1 에서 독립적으로 이미 적용되었음을 명시하는 역할을 한다. per-cycle τ_c + Algorithm 2 통합은 후속 과제 (Task 39 가칭) 로 등록한다.

아키텍처 유지. Algorithm 2의 FC 모델은 `nn.Linear` 레이어를 사용하므로 float 입력을 별도의 수정 없이 그대로 받아들인다. "아키텍처 교체"가 아니라 **입력 데이터를 이진 → 연속으로 교체**하는 것이다. 모델이 학습하는 내용은 "이진 on/off" 대신 "연속 활성화 강도 → note 시퀀스" 매핑이 된다. 본 절은 **DFT baseline** $N=10$ 재실험 결과를 본문으로 한다 (Task A9, `soft_activation_dft_gap0_results.json`).

결과 ($N=10$, DFT baseline $\alpha=0.5$, 아키텍처별 비교).

모델	입력	JS (mean ± std)	val_loss (mean)	개선율
FC	Binary	0.00217 ± 0.000565	0.3395	—
FC	**Continuous**	0.000348 ± 0.000149	0.0232	$+84.0\%$
Transformer	Binary	0.00251 ± 0.000569	0.836	—
Transformer	Continuous	0.000818 ± 0.000255	0.152	$+67.4\%$
LSTM	Binary	0.233 ± 0.0289	0.408	—
LSTM	Continuous	0.170 ± 0.0272	0.395	$+27.3\%$

FC와 Transformer 모두 continuous 입력으로 JS가 크게 개선된다 (각각 $+84.0\%$, $+67.4\%$). LSTM 또한 continuous 입력이 binary 대비 $+27.3\%$ 개선되나 (Welch $p=7.82\times10^{-5}$), 절대 수치 0.170은 FC/Transformer와 비교 불가한 수준이다. 유의성 검증: FC-cont vs Transformer-cont Welch $p=1.66\times10^{-4}$ (FC 유의 우위); FC-cont vs FC-bin $p=1.50\times10^{-6}$; Transformer-cont vs Transformer-bin $p=1.37\times10^{-6}$.

최적 설정은 **FC + continuous** ($\mathrm{JS}=0.000348\pm0.000149$) — **DFT 조건 Algorithm 2 최저** 기록 (Tonnetz 조건 최저 §5.9 실험 B $=0.0003$ 은 hibari 비최적 거리 함수 기준임).

> **이전 실험 + 조건부 각주.** 초기 Tonnetz $N=5$ 실험: FC-cont 0.0004, Transformer-cont 0.0007, LSTM-cont -3.5% (binary 대비 악화). DFT $N=10$ 재실험에서 LSTM continuous 가 binary 대비 소폭 개선으로 나타났으나 ($+27.3\%$), LSTM이 FC/Transformer 대비 열위임은 동일하다. **gap·거리 조건부 주의:** gap = 3 + DFT 조건 (`dl_comparison_dft_gap3_results.json`)에서는 Transformer-bin (0.00276)이 FC-bin (0.00354) 보다 낮아 Transformer 가 우위였다; gap = 0 에서는 FC 가 우위 (본 절 결과). 조건에 따라 최적 모델이 달라지므로 §4.3 참조.

Phase 2 — $\alpha=0.25$ 재실험 (Task 51, $N=10$, $K=14$, `fc_cont_dft_alpha025_results.json`). §5.7 binary OM grid search와 §5.8.1 per-cycle τ_c 모두에서 $\alpha=0.25$가 Algo1 최적으로 확인됨에 따라, 동일 DFT + gap=0 + continuous OM 조건에서 Algo2 FC-cont를 $\alpha=0.25$ ($K=14$)로 재검증하였다.

모델	입력	α	K	JS (mean ± std)
FC	Continuous	0.5 (Phase 1, 본 절 표)	19	0.000348 ± 0.000149 ★
FC	Continuous	0.25 (Phase 2)	14	0.00057 ± 0.00046

Phase 1 대 Phase 2 Welch $p=0.168$. 관측 수치상으로는 $\alpha=0.5$가 더 낮지만(0.000348 vs 0.00057), 이 차이는 통계적으로 유의하지 않다 — random variation으로 설명 가능한 범위이며 $\alpha=0.25$가 $\alpha=0.5$ 대비 유의한 개선도, 유의한 열세도 달성하지 못했다. 따라서 **Algorithm 2 최저 기록은** $\alpha=0.5$, **JS** $=0.000348$**을 유지**한다.

해석: Algorithm 1에서는 K 감소(19 → 14)가 불필요한 cycle 제거 → sampling precision 향상으로 이어지지만, Algorithm 2(FC)에서는 동일한 K 감소가 학습 signal의 정보량 감소로 반전된다. 즉, $\alpha=0.25$**의 이점은 Algorithm 1 한정**이며, Algorithm 2는 더 많은 cycle 정보($K=19$)를 유지하는 $\alpha=0.5$가 유리하다. 이는 §5.8.1 Algo1 최적($\alpha=0.25$)과 다른 하이퍼파라미터에서 Algo2 최적이 형성됨을 뜻하며, 두 알고리즘의 학습 신호 특성 차이를 반영한다.

5.8.3 샘플링 온도(temperature) 조정

> **SHORT 미포함**

방법. NodePool 샘플링 가중치를

$$w(n) \propto \mathrm{freq}(n)^{1/T}$$

로 조정한다. $T=1$이면 원래 빈도, $T>1$이면 균등화(드문 note 강화), $T<1$이면 고빈도 note 집중. Algo1 JS로 N=10 평가.

결과.

T	JS (mean ± std)	개선율
0.3	0.0596 ± 0.0017	$+4.9\%$
0.5	0.0624 ± 0.0027	$+0.5\%$
1.0 (baseline)	0.0627 ± 0.0042	—
2.0	0.0587 ± 0.0032	$+6.5\%$
3.0	0.0585 ± 0.0046	$+6.7\%$
5.0	0.0607 ± 0.0039	$+3.3\%$

$T=3.0$이 최적이지만 개선폭은 6.7%로 §5.8.1/§5.8.2에 비해 미미하다. $T>1$이 최적인 것은 hibari의 높은 entropy (0.974)와 일관된다 — 드문 note도 자주 등장하는 곡에서는 빈도 분포를 더 균등하게 만드는 것이 유리하다. $T=3.0$이 현재 기본값으로 설정되었다 (`config.py GenerationConfig.temperature`).

5.9 Complex 가중치 모드 — Tonnetz 한정 유효

동기

§5.7까지의 실험은 모두 **timeflow 모드**에서 수행되었다. Timeflow 모드는 선율 인과 (inst 1 ↔ inst 2 시간 lag)를 포착하지만, 동시음(simul) 구조 — 한 시점에서 두 악기가 함께 활성화되는 패턴 — 는 반영하지 못한다. **Complex 모드**는 timeflow 행렬과 simul 행렬을 독립적으로 구성한 뒤 선형 결합하여 두 구조를 함께 반영한다.

공식 — 3개 독립 rate 파라미터

Complex 모드의 가중치 계산은 세 단계로 이루어진다 (`pipeline.py`, `_search_complex` 메서드):

$$W_{\text{timeflow}} = W_{\text{intra}} + r_t \cdot W_{\text{inter}}$$

$$W_{\text{simul}} = W_{\text{simul_intra}} + r_c \cdot W_{\text{simul_inter}}$$

$$W_{\text{complex}} = \widetilde{W}_{\text{timeflow}} + r_{\text{scan}} \cdot W_{\text{simul}}$$

여기서 $\widetilde{W}_{\text{timeflow}}$는 refine 후 대칭화된 timeflow 행렬이며, 세 파라미터는 독립적이다:

- $r_t = 0.3$ (고정): timeflow 내 inter lag 기여 비율
- r_c (고정, 실험 B = 0.1): simul 내 simul_inter 기여 비율
- $r_{\text{scan}} \in [0.0,\ 1.5)$ (PH sweep): complex 행렬에서 simul의 혼합 비율

Out-of-reach 기준 oor은 W_{simul}에서 계산된다 (`pipeline.py` line 370). 최종 거리 행렬에는 §5.7과 동일하게 α-hybrid Tonnetz 거리가 적용된다.

Grid Search — $r_c = 0.1$ 강건 최적 확인 ($N=5$, Tonnetz 기반)

> 본 Grid Search는 **Tonnetz 거리** 조건에서 수행된 실험이다. DFT 조건에서의 검증은 하단 "DFT 조건 검증" 절 참조.

$\alpha \in \{0.25, 0.5\}$, $w_o \in \{0.0, 0.3, 0.6\}$, $dw \in \{0.0, 0.3, 0.6\}$, $r_c \in \{0.1, 0.3, 0.6\}$ 총 54개 조합을 $N=5$ 반복으로 탐색하였다 (`complex_grid_results.json`).

r_c	평균 JS	최저 JS	최적 조합
0.1	0.0525	0.0340	$\alpha=0.25,\ w_o=0.0,\ dw=0.3$

0.3	0.0530	0.0342	—
0.6	0.0555	0.0345	—

$r_c = 0.1$이 평균·최저 모두에서 가장 낮아, 이후 실험은 $r_c = 0.1$로 고정하였다.

설정 비교 ($N = 20$, greedy per-cycle τ_c 공통 적용, Tonnetz 기반)

> 이 실험은 **Tonnetz 거리**에서 수행되었다. DFT 거리에서는 complex 모드가 적합하지 않음 (하단 "DFT 조건 검증" 절 참조).

설정	α	w_o	dw	r_c	K	Algo1 JS (mean ± std)
B ★ (Tonnetz 최저)	0.25	0.0	0.3	0.1	40	0.0183 ± 0.0009
D	0.5	0.3	0.3	0.1	45	0.0218 ± 0.0014
E	0.25	0.0	0.3	0.3	39	0.0214 ± 0.0011

실험 C ($N = 5$, JS $= 0.0172$)는 표본 수 부족으로 인한 샘플링 잡음으로 판단하여 $N = 20$ 재검증에서 제외하였다.

통계적 유의성 (Welch t-test)

비교	p-value	상대 차이
B vs D	$p < 0.001$	D가 $+19.6\%$ 높음
B vs E	$p < 0.001$	E가 $+17.5\%$ 높음
D vs E	$p = 0.350$	비유의

B가 D, E 대비 통계적으로 유의하게 우위이며, D와 E 사이에는 유의한 차이가 없다.

Greedy Per-cycle τ_c — 48.3% 개선

Greedy per-cycle τ_c 탐색은 $N = 10$에서 수행되었다 ($\tau_c \in \{0.1, 0.2, \ldots, 0.7\}$, cycle별 독립 최적화). 실험 B에서 균일 τ 대비 JS가 48.3% 개선된다 ($0.0358 \rightarrow 0.0185$, `complex_percycle_results.json`). $N = 20$ 재검증에서는 이 τ_c 벡터를 재사용하여 통계적 안정성을 확인하였다 (`best_taus_source: "reused_from_prev_experiment"`).

Algorithm 2 결과 — Tonnetz 조건 최저

설정	Algo2 FC JS
기존 Tonnetz 최저 (실험 A, timeflow)	0.0004
실험 D (complex, $\alpha = 0.5$)	0.0005
실험 B ★ (complex, $\alpha = 0.25$, Tonnetz)	0.0003

실험 B의 FC 모델이 JS $= 0.0003$을 기록하며, Tonnetz 조건 Algo2 최저를 확정한다. 단, DFT 조건 재기준 이후 §5.8.2 FC-cont (0.000348, $N = 10$, DFT)가 현행 hibari 최적 설정 Algo2 최저이다.

해석 (Tonnetz 조건)

Simul 소량 혼합 ($r_c = 0.1$)의 효과 (Tonnetz). $r_c \geq 0.3$에서는 K가 감소하거나 JS가 증가한다. 소량 ($r_c = 0.1$)의 simul 혼합이 timeflow만으로는 포착되지 않는 화성적 공존 패턴을 보완하면서도, timeflow의 선율 인과 구조를 손상시키지 않는 Tonnetz 조건의 최적점이다. 이 효과가 거리 함수에 따라 달라지는지 여부는 하단 DFT 조건 검증에서 확인한다.

$\alpha = 0.25$ vs $\alpha = 0.5$ (Tonnetz complex 한정). Tonnetz complex grid search에서는 $\alpha = 0.25$가 충분한 K를 유지하면서 JS를 최소화하는 최적점이다. DFT hybrid §5.7에서도 $\alpha = 0.25$가 최적으로 일치하지만, complex 모드 자체의 DFT 적합성은 하단 검증에서 확인한다.

DFT 조건 검증 (Task 34b, 2026-04-17)

DFT 거리 + $\alpha = 0.25$, per-cycle τ_c 조건에서 complex 모드를 검증하였다 (Task A10-b $\alpha = 0.25$ 재실험, `complex_percycle_dft_gap0_alpha25_results.json`). 아래 complex 행은 모두 $\alpha = 0.25$로 통일되었으며, §5.9 Tonnetz 행은 추가로 $\omega_o = 0.0$ 조건이다:

r_c	Algo1 JS (mean ± std, $N = 20$)	A8 대비 Welch p
0.1	0.0440 ± 0.0010	$p = 4.74 \times 10^{-39}$ (유의 악화)
0.3	0.0657 ± 0.0015	$p = 1.12 \times 10^{-48}$ (유의 악화)
(참조) A-3 DFT timeflow + per-cycle τ_c (★ 신기록)	0.00902 ± 0.00170	기준 ★

두 조건 모두 DFT timeflow + per-cycle τ_c (A8, §5.8.1) 대비 유의하게 악화되었다. $\alpha = 0.25$는 Task A10-a의 DFT α grid 최적값으로, α 선택 문제가 아닌 **complex 모드 자체**가 DFT 거리에서 부적합함을 확인한다.

판정: `complex_tonnetz_only_effective` — complex 모드의 simul 혼합은 Tonnetz 거리에서는 유효하지만, DFT 거리에서는 timeflow 모드 대비 오히려 cycle 활성 신호를 흐린다. 현재 hibari 최적 설정 (거리 함수: DFT)에서는 complex 모드를 사용하지 않는다.

Tonnetz 최적 조합 (참고)

> **Complex 모드 최적 조합 (Tonnetz 전용):**

>

> $\alpha = 0.25, \quad w_o = 0.0, \quad dw = 0.3, \quad r_c = 0.1, \quad r_t = 0.3$ + Greedy per-cycle τ_c

>

> → **Algo1 JS** $= 0.0183 \pm 0.0009$ (Tonnetz baseline 대비 -24.1%)

>

> → **Algo2 FC JS** $= 0.0003$

>

> ※ DFT baseline (§5.8.1 A8)의 Algo1 최저는 0.00902 ± 0.00170 로 이를 하회한다. 현재 hibari 최적 설정은 **DFT + per-cycle τ_c (timeflow)**.

---

6. 블록 단위 생성 + 구조적 재배치

본 장은 hibari의 32-timestep 블록 구조를 직접 활용하여, 한 블록만 생성한 뒤 hibari의 실제 구조에 따라 배치하는 접근의 구현과 결과를 보고한다. 여기서 블록은 32 timestep(음악적 4-bar 반복 단위, §2.9)을 가리키며, 원곡에서는 A-B-A'-C의 모듈 선율 단위가 inst 1에서 33회 연속 반복된다 (원곡 구조 서술에는 "모듈" 용어를 그대로 사용한다). 실험 러너는 `tda_pipeline/run_module_generation_unified.py` (단일 전략 본 실험 및 prototype 전략 비교 통합) 이며, 결과는 `docs/step3_data/step71_module_results.json`, 시각화는 Figure 8 (`docs/figures/fig8_module_gen.png`) 에 저장되어 있다.

6.1 구현 설계

설계 목표

기존 Algorithm 1은 전체 $T=1{,}088$ timesteps을 한 번에 생성한다. 본 §6은 이를 $T=32$ **(한 블록) 생성 + 65회 복제**로 바꾸어 (inst 1의 33회 + inst 2의 32회, 총 65회 복제), 다음 세 가지 목적을 달성하려 한다.

1. **계산 효율** — 생성 시간을 대폭 단축 (~ 40 ms $\rightarrow$ ~ 1 ms per module)
2. **구조적 충실도** — hibari의 모듈 정체성(그림 2.9)을 재샘플링이 아니라 복제로 보존
3. **변주 가능성** — 단일 블록의 seed만 바꾸면 곡 전체 변주가 자동으로 만들어짐

3단계 프로세스

Step 1 — Prototype module overlap 구축. Algorithm 1이 블록 1개를 생성하려면 32개 시점 각각에서 "지금 어떤 cycle이 활성인가"라는 정보가 필요하다. 이 정보를 담는 32-row 짜리 prototype OM $O_{\text{proto}} \in \{0,1\}^{32\times 14}$를 만드는 것이 본 단계의 핵심이다. 여기서 $K=14$는 DFT α=0.25 기반 hibari PH에서 발견된 cycle 수이다. 어떻게 만드는 것이 적절한지에 대해서는 다음 §6.2에서 4가지 전략을 비교 검증한 뒤 최적안을 채택한다.

Step 2 — Algorithm 1로 단일 블록 생성. 위에서 만든 O_{proto} 와 전체 cycle 집합 $\{V(c)\}_{c=1}^{14}$ (DFT α=0.25 기반 PH 계산에서 추출) 을 입력으로 받아, 길이 32 인 chord-height 패턴 $[4,4,4,3,4,3,\ldots,3]$ (hibari 실제 module pattern 32-element 1회) 을 따라 Algorithm 1을 실행한다. 결과는 32 timesteps 안의 note 리스트 $G_{\text{mod}}=[(s,p,e)_k]$이다. hibari의 경우 한 블록당 약 $45\sim 60$개 note가 생성되며, 소요 시간은 $\sim 1-2$ ms이다.

Step 3 — 구조적 재배치. G_{mod}를 hibari의 실제 두 악기 구조에 그대로 맞춰 배치한다. 이 배치 패턴은 그림 2.9에서 시각적으로 검증된 hibari의 두 악기의 활성/쉼 패턴 — Inst 1은 $t\in[0,1056)$ 동안 33 copies 연속, Inst 2는 $t=33$부터 32 copies가 각 사이에 1-step 쉼을 두고 입장 — 을 그대로 따른다.

6.2 Prototype module overlap 전략 비교

위 Step 1에서 가장 중요한 결정은 "어떤 방식으로 32-row 짜리 prototype overlap을 만들 것인가" 이다. 본 절은 네 가지 후보 전략을 정의하고 동일한 $N=10$ 반복 조건에서 비교한다.

네 가지 후보 전략

원본 OM $O_{\text{full}} \in \{0,1\}^{1088\times 14}$를 34×32로 reshape한 텐서 $\tilde{O}\in\{0,1\}^{34\times 32\times 14}$ 위에서 다음 네 가지 prototype을 정의한다. (각 블록은 32 timestep = 음악적 마디 8-step의 4배 단위이다.)

코드	정의	의미
P0	$O_{\text{proto}}=\tilde{O}[0,:,:]$	대표 샘플($t_{\text{start}}=0$) 32-step 구간 복제
P1	$O_{\text{proto}}[t,c]=\max_m \tilde{O}[m,t,c]$	34개 블록 합집합(OR)
P2	$O_{\text{proto}}[t,c]=\prod_m \tilde{O}[m,t,c]$	34개 블록 공통(exclusive, AND)
P3 ★	블록-local PH 재계산	두 악기 모두 $[32(m+1),32(m+2))$ 창 — 동일 물리 시각, index lag = physical lag

각 전략의 상세 설명:

P0 (첫 블록 복제, density 0.018). $\tilde{O}[0,:,:]$ — 절대 시간 $t\in[0,32)$ 구간(대표 샘플)의 binary overlap을 그대로 prototype으로 사용한다.

P1 (합집합, density = 1.000). $\max_m \tilde{O}[m,:,:]$ — 34개 블록 어느 하나에서라도 활성인 셀이면 1로 설정한다.

P2 (exclusive, density 0.000). 모든 블록에서 동시에 활성인 셀만 1로 두는 AND 전략. 실측상 원곡 overlap에서 전 블록 공통 활성 셀은 거의 없으므로 density가 실질적으로 0에 가깝다.

P3 (국소 PH 재계산, density 0.299) ★ best. 시작 블록 m에서 **두 악기 모두** $[32(m+1),32(m+2))$ **동일 창**을 잘라 블록-local PH를 재계산한다 (2026-04-19 Option B 설계). 두 창이 동일 물리 시각을 참조하므로 `compute_inter_weights(cs1[:L], cs2[:L], lag=1)`의 index lag=1이 physical lag=1과 일치한다. 전역

K=14 대신 이 구간에서 재추출한 local cycle (11개 내외)을 사용한다.[^window-history]

[^window-history]: 이전 설계에서는 inst1 창 $[32m, 32m+32)$과 inst2 창 $[32m+33, 32m+65)$으로 두 창을 33-step offset에 두었으나, 두 창이 sequential(1-step gap)하여 `lag=1`이 실제로는 34-step physical lag를 참조하는 설계 괴리가 발견되었다 (2026-04-19). 창 통일 후 index lag = physical lag 일치.

위 네 전략 중 P3은 대상 구간 데이터로 cycle을 새로 추출하는 접근이며, §6.5에서 개선 C와의 결합(P3 + C)을 별도로 평가한다.

결과 ($N=10$ trials, baseline full-song DFT JS $=0.0213 \pm 0.0021$)

> 참고: 위 baseline은 "거리 함수 효과" 비교를 위한 §4.1 기준값이다. 최종 설정에서는 §5.8.1의 신기록 0.00902 ($\alpha=0.25$, Task A-3)까지 추가 하향이 가능하다.

전략	Density	JS Divergence (mean ± std)	Best trial	Note coverage
P0 — 첫 블록 복제	0.018	0.1040 ± 0.0219	0.0698	0.809
P1 — 합집합(34블록 OR)	1.000	0.0621 ± 0.0167	0.0422	0.800
P2 — exclusive (AND)	0.000	0.0900 ± 0.0308	0.0515	0.809
P3 — local PH 재계산 ★	0.299	0.0474 ± 0.0187	0.0226	0.817

핵심 발견

발견 1: Option B (동일 창) 설계에서도 P3이 최우수. 평균 JS 0.0474 ± 0.0187로 네 전략 중 최저이며, best trial도 0.0226으로 최저다 (T38a-2 Option B 재실험).

발견 2: P0는 여전히 가장 약하다. 대표 시작 구간 단일 복사(P0)는 활성 밀도가 지나치게 낮아 평균 JS가 가장 높았다(0.1040).

발견 3: P1은 P2보다 낫지만 P3에는 미치지 못한다. OR(P1, 0.0621)는 blanket-active 전략으로 cycle 구조를 무차별 활성화하여 baseline 대비 개선은 있으나, 대상 구간의 국소 cycle을 직접 재추출하는 P3의 이득을 넘지 못한다. exclusive(P2, 0.0900)는 density가 사실상 0이라 P0에 가까운 열세이다.

발견 4: 블록-level randomness amplification은 여전히 지배적이다. 최선의 평균값(P3, 0.0474)도 full-song baseline(0.0213)보다 높아, 블록 1개 생성/65회 복제 구조의 분산 문제가 남아 있다 (§6.4).

본 실험의 채택 전략

본 §6.3 본 실험에서는 **P0 (첫 블록 복제)** 를 기본 전략으로 채택한다. P3이 비교 실험(§6.2)에서 best지만, §6.3은 "단일 전략 통계 확정"을 목적으로 N=20 시행을 수행하며, 처음 설계된 실험 구조를 유지한다. P3을 포함한 개선 조합은 §6.5에서 체계적으로 평가한다.

6.3 본 실험 결과 (P0 전략 사용)

P0 (첫 블록 복제) prototype을 사용하여 $N=20$회 독립 반복 (seed $7100 \sim 7119$):

지표	값 (mean ± std)	min – max
Pitch JS Divergence	0.1082 ± 0.0241	$0.0701 - 0.1725$
Note Coverage	0.787 ± 0.054	—
Total Generated Notes	$3{,}338 \pm 218$	—

Generation Time (per module)	~ 2 ms	—
최우수 trial (seed 7105)	$\mathrm{JS} = 0.0701$	—

기존 baseline과의 비교

방식	JS Divergence	소요 시간	비고
§4.1 Full-song DFT (baseline)	0.0213 ± 0.0021	~ 40 ms	$N = 20$
§6 (P0, 본 보고)	0.1082 ± 0.0241	~ 2 ms	$N = 20$
§6 (P0, best trial)	0.0701	~ 2 ms	seed 7105

세 가지 관찰

관찰 1: 최우수 trial도 baseline과 격차가 크다. 본 실험의 best trial (seed 7105)은 JS $= 0.0701$로 baseline (0.0213) 의 약 3.3배이다. P0의 best는 cycle 구조에 기반한 결과라는 점에서 의미가 있다.

관찰 2: 평균은 baseline의 약 5.1배. P0의 평균 JS는 baseline 대비 약 5.1배 나쁘다. 이는 prototype 전략 자체의 한계가 아니라 module-level randomness의 amplification 때문이다 (§6.4).

관찰 3: 빠른 생성 속도는 그대로. 모듈 1개 생성에 ~ 2 ms (full-song ~ 40 ms 대비 20배 빠름). 총 재배치까지 포함해도 < 5 ms 수준이며, 실시간 인터랙티브 작곡 도구에 충분히 적합한 속도를 유지한다.

6.4 한계와 개선 방향

한계 1 — Module-level randomness의 33× amplification

단일 모듈 생성은 32-step chord-height 합산 시 명목상 ~ 109개 note slot이 있으나, onset 차감 로직(동일 화음 내 onset 상한 적용)으로 실제 $45 \sim 60$**개 onset**에 의존하며, 각 choice의 결과가 이후 33번 (inst 1) + 32번 (inst 2) 반복되므로 **한 번의 random choice가 곡 전체에서 65번 반복된다**. 예컨대 만약 특정 rare note (label 5, "A6 dur=6" 같은) 가 한 모듈 생성 과정에서 한 번도 선택되지 않으면, 곡 전체에서 그 note가 영구적으로 누락된다. 본 실험의 평균 note coverage 0.791 (23개 중 약 18개) 가 이를 반영한다.

이는 prototype 선택과 무관한 본질적 한계이며 (§6.2 발견 3), 후술할 개선 C/P3가 직접적으로 완화한다.

개선 방향

위 한계를 다음 두 가지 후속 작업으로 해결할 수 있다.

개선 C — 모듈 수준 best-of-k selection. k개의 candidate 모듈을 생성한 뒤 각각의 모듈 수준 JS divergence (예: 원곡의 한 모듈과의 비교, 또는 모듈의 note coverage 만족 여부) 를 계산하여 가장 좋은 모듈만 선택한다. $k = 10$ 으로 두면 ~ 20 ms 추가 비용으로 분산을 크게 낮출 수 있을 것으로 기대된다. 이는 한계 1 (randomness amplification) 의 가장 직접적 대응이다.

6.5 한계 해결 — 개선 C / P3 / P3+C 구현 및 평가

§6.4 에서 정의한 개선 방향 중 **C, P3** 를 구현하고 결합 전략 **P3+C**를 포함해 P0 baseline 과 동일 조건 ($N = 10$ 반복, seed $7300 \sim 7309$) 에서 평가하였다. 실험 러너는 `tda_pipeline/run_module_generation_unified.py` 이며, 결과 원본은 `docs/step3_data/step71_improvements.json` 에 저장되어 있다.

구현 세부

개선 C — best-of-k selection ($k = 10$). 동일 prototype overlap에서 seed 만 달리한 k 개 candidate 모듈을 모두 생성한 뒤, 각 모듈의 내부 note coverage (모듈 안에서 사용된 unique (pitch, dur) label 수, 0~23) 를 계산하여 가장 높은

모듈을 선택한다. 동률 처리: 같은 coverage 를 갖는 candidate 가 여럿 있을 때는 **생성 순서가 빠른 것**(= 랜덤 시드 인덱스가 더 작은 것)을 선택한다 — 예를 들어 seed 7300 과 7305 가 둘 다 coverage 22 이면 먼저 생성된 seed 7300 을 채택. 이는 어느 것을 골라도 품질은 같지만, 실행마다 다른 모듈을 고르면 재현성이 깨지므로 "가장 먼저 만들어진 것"을 규약으로 정한 것이다. 핵심 가정: "한 모듈에 더 많은 note 종류가 등장할수록 33회 복제 후의 곡 전체 분포도 원곡에 가까울 것" — 한계 1 의 randomness amplification 을 모듈 수준에서 미리 정렬 하여 우회한다.

개선 P3 — Bar-local persistent homology. §6.2에서 정의된 block-local PH 전략(시작 블록 m에 대해 두 악기 모두 $[32(m+1), 32(m+2))$ 동일 창에서 intra/inter weight matrix를 재계산하고 PH를 재실행)을 본 §6.5 한계 해결 프레임에 통합하여 평가한다. 전역 $K=14$ 대비 local $K_{\text{local}} \approx 11$로 감소하며, 이 cycle 집합으로부터 만든 $32 \times K_{\text{local}}$ 활성 행렬을 Algorithm 1의 입력으로 사용한다.

P3 + C 결합. 블록-local cycle 위에서 best-of-k selection 을 동시에 적용. P3의 의미 있는 cycle 구조와 C 의 randomness 통제를 결합한다.

결과 ($N=10$, baseline full-song JS $=0.0213 \pm 0.0021$)

전략	JS Divergence (mean ± std)	best	full-coverage	per-trial 시간
Baseline P0 (§6.3, $N=20$)	0.1082 ± 0.0241	0.0701	0.787	~ 2 ms
C: best-of-10	0.0800 ± 0.0171	0.0570	0.913	~ 20 ms
D: cov $\geq 20/23$	0.0808 ± 0.0171	0.0570	0.887	~ 20 ms
P3: bar-local PH	0.0636 ± 0.0207	0.0335	0.791	~ 3 ms
P3 + C ★ 최강 조합	0.0417 ± 0.0145	0.0167	0.900	~ 25 ms

핵심 발견.

1. **P3 + C 가 최우수**: 평균 0.0417 ± 0.0145 로, P0 baseline (0.1082) 대비 61% 감소. 표준편차도 $0.0241 \rightarrow 0.0145$ 로 40% 감소. Full-song DFT baseline (0.0213) 대비로는 약 1.96배이며, best trial 0.0167 은 baseline 대비 0.78배 — 본 §6.5에서 최초로 full-song baseline을 하회한다.
2. **P3 단독 만으로도 유의한 효과**: baseline(0.1082) 대비 41% 감소(0.0636). 블록-local PH가 더 선택적인 cycle 구조를 제공하는 효과가 DFT 조건에서도 재현된다.
3. **개선 조합 효과는 distance-invariant**: Tonnetz 조건 P3+C best trial ≈ 0.0348, DFT 조건 P3+C best trial 0.0167 — 절대값은 달라도 "P3+C 조합이 최강"이라는 서열은 두 거리 함수에서 동일하게 관찰된다.

Best trial 분석 — P3 + C, seed 7300

이 실험의 best trial (P3 + C, seed 7300) 는 JS 0.0167, coverage 87.0% ($20/23$), 블록 내 53개 note 를 사용하였다. Full-song DFT baseline 평균 (0.0213) 대비로는 약 0.78배로, baseline을 **하회**한다. §6.7 시작 블록 전체 탐색에서는 이보다 더 낮은 JS=0.01479 (start=0, seed=9309) 가 관찰되며, 이는 §5.8.1 Phase 1 full-song 최저값 0.01489와 사실상 동등하다 (§6.7 참조); $\alpha=0.25$ 조건의 블록 재탐색은 이미 수행되어 best global JS $=0.01479$를 달성하였으나, §5.8.1 Phase 2 신기록 ($\alpha=0.25$ per-cycle τ, 0.00902) 대비로는 $+64.0\%$ 열세이다.

한계 해결 정도 정리

한계	해결 정도
한계 1 — 블록-level randomness 65× amplification	**대폭 해결**. P3 + C 로 std 가 baseline의 60% 수준으로 감소 ($0.0241 \rightarrow 0.0145$)

6.6 결론과 후속 과제

§6 의 핵심 주장 재정의. 본 §6 구현 + 한계 해결 (§6.5) + 시작 모듈 탐색 (§6.7) 의 결과로 다음을 주장할 수 있게 되었다.

> **블록 단위 생성 + 구조적 재배치는 단순한 효율 트릭이 아니라, 적절한 후처리와 시작 블록 탐색을 결합하면 full-song Algorithm 1 Phase 1 수준과 수치적으로 동등한 품질에 도달한다.** §6.5 P3+C 평균 JS 0.0417 (full-song baseline 0.0213의 1.96배, best 0.0167은 baseline 하회), 그리고 §6.7 시작 블록 탐색 전역 최적 trial JS 0.01479 — §5.8.1 Phase 1 DFT per-cycle τ ($\alpha = 0.5$) 최저 0.01489와 차이 0.00010으로, Phase 1 기준 동등하다. 다만 Phase 2 신기록 ($\alpha = 0.25$ per-cycle τ N=20, 0.00902) 대비로는 $+64.0\%$ 열세이며, Phase 2의 모든 구조적 정보를 복원하지는 못한다 — 블록-local PH는 local cycle 수가 11로 감소(전역 K=14 대비)하여 pitch-class 분포 해상도가 제한되기 때문으로 해석된다.

이는 §6.4 의 한계 1 ("randomness amplification") 가 적절한 selection mechanism (C) 과 local topology (P3) 의 결합으로 통제 가능하며, 나아가 시작 블록 선택이 추가 품질 향상의 여지를 열어준다는 것을 보여준다.

결론 강화: 블록 단위 full-song Phase 1 (Phase 2 미달). §6.7 에서 start=0, seed=9309 의 best global trial JS $= 0.01479$는 §5.8.1 Phase 1 full-song Algorithm 1 ($\alpha = 0.5$ per-cycle τ) 최저 JS 0.01489와 0.00010 차이로, Phase 1 기준 동등한 위상 구조 충실도를 33배 빠른 생성으로 달성한다. Phase 2 신기록 ($\alpha = 0.25$ per-cycle τ N=20, 0.00902) 대비는 $+64.0\%$ 열세 — **"잘 선택된 한 블록을 65회 복제하는 것이 full-song Phase 1과 동등한 품질을 만들 수 있으나, Phase 2의 모든 정보를 복원하지는 못한다"**가 §6 전체의 수치적 결론이다.

본 연구 전체에 미치는 함의. §6 은 본 연구의 "topological seed (Stage 2-3)" 와 "음악적 구조 배치 (Stage 4 arrangement)" 가 서로 직교하는 두 축임을 실증한 첫 사례이다. **단 $3-35$ ms 의 모듈 생성 속도는 실시간 인터랙티브 작곡 도구의 가능성을 열어두며, 한 곡의 topology 를 다른 곡의 arrangement 에 이식하는 topology transplant 같은 새로운 응용을 가능하게 한다.**

즉시 가능한 다음 단계

1. **청각적 평가**: P3 + C 의 best trial MusicXML 을 피아노 음색으로 렌더링하여 원곡과 A/B 청취 비교 (§6.7에서 수행)
2. **~~개선 F (continuous + Algorithm 2) 구현~~ → §4.5에서 완료.** FC-cont JS 0.00035, FC-bin (0.00217) 대비 6.2배 우수.
3. **악기별 모듈 분리 (옛 개선 E 재검토)**: inst 2 누락 한계 (§6.7 핵심 발견 1) 의 심화 해결을 위해 inst 1 용 / inst 2 용 모듈을 별도 생성
4. **~~다른 곡으로의 일반화~~ → §5.1에서 solari 실험 완료.** Transformer 최적 확인.

6.7 P3 의 시작 블록 탐색 — 32개 후보 비교

§6.5의 P3은 대표 시작점 **start marker** $m = 0$ (분석 창 $t \in [32, 64)$)의 데이터로 블록-local PH를 계산했다. 이 선택의 자의성을 검증하기 위해, **32개 시작 블록** (start = 0~31, 분석 창 $[32(m+1), 32(m+2))$) 전체에서 동일한 P3 + C 실험을 수행하였다. 여기서 k는 각 seed 내부의 best-of-k 후보 수($k = 10$), N은 시작 블록당 독립 반복 횟수($N = 10$ seeds)다. 실험 러너는 `tda_pipeline/run_phase3_task38a_dft_gap0.py`, 결과는 `docs/step3_data/section77_experiments_dft_gap0.json`.

블록 번호 정의: 시작 블록 m의 분석 창은 $[32(m+1), 32(m+2))$. 두 악기 모두 이 동일 창에서 local PH를 재계산한다 (2026-04-19 Option B). $m = 0$일 때 분석 창은 $[32, 64)$이므로 **원곡 첫 블록** $[0, 32)$**는 본 §6.7 탐색 대상에서 제외**된다 — 이는 창 동일성(index lag = physical lag)을 위한 설계 trade-off이며 32개 start marker 전수 탐색의 완전성에는 영향이 없다. $m = 32$는 inst 1 구간이 원곡에 없어 자연스럽게 배제된다.

결과 (상위/하위 대표 + 주요 블록)

시작 블록 m	#cycles	density	JS (mean ± std)	best trial	비고
13	15	0.862	0.0306 ± 0.0065	0.01980	평균 1위 ★
31	11	0.625	0.0324 ± 0.0136	0.01531	평균 2위
15	16	0.807	0.0363 ± 0.0105	0.01853	평균 3위
10	15	0.823	0.0364 ± 0.0088	0.02534	평균 4위

29	13	0.752	0.0373 ± 0.0073	0.02893	평균 5위
...	...	...	...	...	(중간 26개 생략)
0 (첫 분석 창)	11	0.636	0.0473 ± 0.0183	0.01479 ★★	평균 rank 16/32 · 전역 최저 trial
...	...	...	...	...	
25	17	0.846	0.0734 ± 0.0169	0.0437	평균 최하위

핵심 발견 — 첫 블록의 예외적 우수성은 소실

DFT α=0.25 Option B 조건에서 다음이 관찰된다.

1. **시작 블록 선택에 따른 특별한 우위 없음.** 32개 시작 블록 중 평균 JS 기준 $m=0$ rank $16/32$ — 정확히 중위. 평균 최저는 $m=13$ (JS 0.0306)이며, $m=0$이 아니다. 이전 (버그 창 기반) 실험에서 관찰되던 "$m=0$의 평균 rank 5" 우위는 Option B 창 통일 이후 소실되었다.
2. **시작 블록 선택에 따른 분산은 있으나, 특정 블록이 일관되게 우수하지는 않다.** 평균 JS 범위: $0.031 \sim 0.073$ (전체 블록 분산). 이 변동폭이 블록 내 trial-to-trial 표준편차(~ 0.015)보다 크므로 시작 블록 선택이 의미를 갖지만, "단 하나의 최적 블록"이 존재한다고 결론 내리기는 어렵다.
3. **최저 평균 JS($m=13$)와 전역 최저 단일 trial($m=0$)의 불일치.** $m=13$은 평균이 가장 좋지만 best trial은 0.01980. 반면 $m=0$은 평균이 16위임에도 불구하고 seed 9309에서 best trial JS $=0.01479$를 기록했다 — 평균 rank와 별개의 극값(variability)로 해석되며, 통계적으로 first-marker의 구조적 특권은 관찰되지 않는다. 이 수치는 §5.8.1 Phase 1 (α=0.5) full-song per-cycle τ 최저 0.01489와의 차이 0.00010으로 Phase 1 기준 실질 동등하다 (Phase 2 신기록 0.00902 대비 $+64.0\%$ 열세).
4. **Tonnetz α=0.5 조건과의 비교 (distance-specific 현상).** 동일 실험을 Tonnetz α=0.5 조건(8개 대표 블록)으로 수행한 결과, start=0이 평균 최우수였다. DFT α=0.25 Option B에서는 이 우위가 미재현되었으며, Option B 창 통일 후에도 재현되지 않았다. 이는 "첫 블록의 우수성"이 거리 함수 선택에 의존하는 distance-specific 현상임을 시사한다.

메타 통찰 — 거리 함수는 수치 차이를 넘어 구조적 특성을 변경한다

이 결과는 거리 함수 선택이 단순히 JS 수치를 높이고 낮추는 것을 넘어, **블록 기반 생성의 구조적 특성 자체를 변경**한다는 사실을 보여준다. Tonnetz 조건에서는 "첫 블록의 단순한 위상 구조"가 유리하게 작용했지만, DFT 조건에서는 중간 정도의 위상 복잡도를 가진 블록($m=13, 15, 10$번)이 더 안정적으로 좋은 평균 성능을 보인다. 생성 전략의 최적화는 거리 함수와 함께 설계되어야 한다.

Best global trial 정보

본 연구 §6 의 32-블록 탐색 전체를 통틀어 **가장 낮은 JS divergence** 는 다음과 같다.

설정: P3 + C ($k=10$), start marker $m=0$, seed = 9309, best_j $=1$

블록 내부: 21개 unique note 사용 (coverage $21/23=91.3\%$), $3{,}575$개 note

JS divergence: 0.01479 (§5.8.1 Phase 1 (α=0.5) full-song per-cycle τ 최저 0.01489와 차이 0.00010; Phase 2 신기록 ($\alpha=0.25$) 0.00902 대비는 $+64.0\%$ 열세)

의의: 한 블록을 65회 복제하는 방식이 Algorithm 1의 full-song Phase 1 최고 품질과 **수치적으로 동등**에 도달함을 실증. 단, Phase 2 per-cycle τ 재탐색 수준에는 미달.

6.8 Barcode Wasserstein 거리 기반 블록 선택 — 결과 및 주의사항

§6.7에서 시작 블록 탐색을 통해 전역 최저 trial JS=0.01479 ($m=0$, seed=9309)를 달성하였다. 이 절에서는 **Barcode Wasserstein 거리 기반 블록 선택 방법**의 유효성을 Option B 동일 창 설계에서 검토한다. 실험 러너는 `run_phase3_task38a_dft_gap0.py` (T38a-6), 결과는 `step_barcode_dft_gap0.json`.

방법. 이 비교는 **블록-local PD(단일 블록 32 timestep의 위상 구조)**와 **full song PD(전체 곡 1088 timestep의 위상 구조)** 사이의 Wasserstein 거리를 측정한다. 구체적으로, 각 시작 블록(32개)의 persistence barcode를 동일 창

$[32(m+1), 32(m+2))$의 데이터만으로 독립적으로 계산하고, 원곡 전체를 대상으로 계산된 barcode와의 Wasserstein 거리 W_p (rate = 0.0, 0.5, 1.0 평균)를 구하여 W_p가 작은 블록을 선택한다. 직관: "Wasserstein 거리가 작을수록 위상 구조가 원곡에 가깝고, 따라서 그 블록이 더 좋은 seed가 될 것."

결과 요약.

지표	값
전체 블록 평균 W_{mean}	0.486
전체 블록 평균 JS	0.0568
전체 블록 최저 JS	0.0332
W-JS Pearson 상관계수	$+0.2457$
Best W-trial ($W=0.288$)	JS $=0.0760$

서사 반전 — Option B 이후 약한 양의 상관 회복.

이 절의 가장 중요한 변화는 Pearson r의 부호 반전이다. **이전 (비대칭 창) 설계에서는** $r=-0.054$ (사실상 무상관)로, "DFT 조건에서는 Wasserstein이 유효하지 않음"이 결론이었다. Option B 동일 창 설계(2026-04-19) 이후 $r=+0.2457$로 회복되어 Wasserstein 거리가 JS 품질과 약한 양의 상관을 가진다. 이는 본래 설계 의도(index lag = physical lag)가 정량적으로 검증되었음을 의미한다.[^r-reversal]

[^r-reversal]: 이전 설계에서는 inst2 창을 33-step offset $[32m+33, 32m+65)$으로 정의하여 두 창이 시작 블록에 따라 비대칭적으로 원곡 구조를 sample하였고, 이 구조적 편향이 barcode Wasserstein 비교의 상관을 약화시켰다 ($r=-0.054$). 2026-04-19 두 창을 $[32(m+1), 32(m+2))$로 통일한 뒤 $r=+0.2457$로 회복되었다.

주의사항 — Wasserstein 기반 선택의 실무적 한계.

1. **상관의 강도는 중간 이하 ($|r|<0.5$).** Option B로 r 부호는 회복되었으나 값 자체는 여전히 약한 양의 상관이다(통계 교과서 기준 $|r|\in[0.2, 0.5]$는 weak-to-moderate). 실무적으로 W **기반 블록 선택만으로 JS 품질을 안정적으로 예측하기에는 상관 강도가 부족**하다. Best W-trial ($W=0.288$)의 JS는 0.0760으로 전역 최저 0.01479와 격차가 크다.
2. **거리 함수 의존성은 여전히 유효.** Tonnetz 조건: $r=0.503$ (중등도 양의 상관), DFT Option B: $r=+0.2457$. 부호는 같아졌으나 강도는 여전히 Tonnetz가 우세하다. W-based 블록 선택은 거리 함수와 α-파라미터가 고정된 조건에서만 신뢰할 수 있다.
3. **Chord 공간 불일치.** 원곡 전체 PH는 23개 note 기반 chord 공간에서 계산되지만, 블록-local PH는 실제 등장한 note만 사용한다. chord 공간 크기 불일치가 Wasserstein 거리 비교의 신뢰성을 제한한다.
4. **Rate 선택 민감도.** W_p는 선택한 rate(필터링 스케일)에 따라 달라진다. 본 실험에서는 rate = 0.0, 0.5, 1.0의 평균을 사용하였다.

이러한 한계로 인해, Option B 이후에도 Wasserstein 기반 블록 선택은 **초벌 screening**에는 사용 가능하나 **최종 선택**에는 JS divergence 직접 평가(§6.7의 전수 탐색)가 더 신뢰할 수 있다. 그럼에도 $r=+0.2457$로의 회복은 설계 의도 부합을 확인한 의미 있는 결과이다.

참고자료

> **SHORT 미포함**

- `tda_pipeline/run_phase3_task38a_dft_gap0.py` — §6 DFT α=0.25 전체 재수행 스크립트 (T38a-1~T38a-6)
- `tda_pipeline/docs/step3_data/step71_prototype_om_dft_gap0.json` — T38a-1: K=14 prototype OM 정보
- `tda_pipeline/docs/step3_data/section72_34blocks_prototype_results.json` — T38a-2: 34×32 재정의 P0/P1/P2/P3 전략 비교

`tda_pipeline/docs/step3_data/step71_module_results_dft_gap0.json` — T38a-3: P0 N=20 본 실험

`tda_pipeline/docs/step3_data/step71_improvements_dft_gap0.json` — T38a-4: C/D/P3/P3+C 개선 비교

`tda_pipeline/docs/step3_data/section77_experiments_dft_gap0.json` — T38a-5: 33모듈 시작 모듈 탐색

`tda_pipeline/docs/step3_data/step_barcode_dft_gap0.json` — T38a-6: Barcode Wasserstein 상관

`tda_pipeline/docs/figures/fig8_module_gen.png` — Figure 8

---

7. 차별화 포인트 — 기존 연구와의 비교

> **SHORT 미포함**

본 연구의 위치를 명확히 하기 위해, 두 가지 관련 연구 흐름과 비교한다. 하나는 **일반적인 AI 음악 생성 연구** 이며, 다른 하나는 **TDA를 음악에 적용한 선행 연구**들이다.

7.1 일반 AI 음악 생성 연구와의 차별점

> **SHORT 미포함**

지난 10년간 Magenta, MusicVAE, Music Transformer 등 대규모 신경망 기반 음악 생성 모델이 여러 발표되었다. 이들은 공통적으로 다음 구조를 따른다:

> 수만 곡의 MIDI 코퍼스 → 신경망 학습 → 샘플링 생성

본 연구는 이와 다음 네 가지 지점에서 근본적으로 다르다.

(1) 코퍼스 규모 vs 단일곡 심층 분석. 일반적인 AI 음악 생성은 "많은 곡을 보고 평균적인 음악 규칙을 배우는 것"을 목표로 한다. 본 연구는 반대로 **단 한 곡(hibari)의 구조를 가능한 한 깊이 해석한 뒤 그 구조를 재생성**하는 것을 목표로 한다. 음악학자가 한 곡의 악보를 정밀하게 분석하는 작업에 가깝다.

(2) Blackbox 학습 vs 구조화된 seed. 일반 신경망 모델은 학습이 끝난 후 "왜 이 음이 나왔는가"를 설명하기 어렵다. 본 연구의 파이프라인은 **persistent homology로 추출한 cycle 집합**이라는 명시적이고 해석 가능한 구조를 seed로 사용하며, 생성된 모든 음은 "특정 cycle의 활성화"라는 구체적 근거를 갖는다. 즉 생성 결과가 역추적 가능하다.

(3) 시간 모델링의 역설. 일반 음악 생성 모델은 "더 정교한 시간 모델일수록 더 좋다"는 암묵적 가정을 가지며, 그래서 Transformer 계열 모델이 주류가 되었다 (Music Transformer, Huang et al. 2018; MuseNet, OpenAI 2019; MusicGen, Meta 2023; MusicLM, Google 2023). 본 연구의 §4.5에서 관찰된 "가장 단순한 FC가 가장 좋은 결과를 낸다"는 결과는, 이러한 일반적 가정이 **곡의 미학적 성격에 따라 뒤집힐 수 있다**는 증거이다. hibari처럼 시간 인과보다 공간적 배치를 중시하는 곡에서는 시간 문맥을 무시하는 모델이 오히려 곡의 성격에 더 맞다.

(4) 곡의 구조에 기반한 설계. 본 연구의 가중치 분리 (intra / inter / simul) 는 hibari의 실제 관측 구조 — inst 1은 쉬지 않고 연주, inst 2는 모듈마다 규칙적 쉼을 두며 얹힘 — 를 수학적 구조에 직접 반영한 것이다 (§2.9). 일반적인 AI 음악 생성에서는 모델의 architectural choice가 "학습 효율"에 따라 결정되지만, 본 연구에서는 **곡의 실제 악기 배치 구조**가 설계의 출발점이다.

7.2 기존 TDA-Music 연구와의 차별점

> **SHORT 미포함**

TDA를 음악에 적용한 선행 연구는 몇 편이 있으며, 본 연구와 가장 가까운 것은 다음 두 편이다.

Tran, Park, & Jung (2021) — 국악 정간보(Jeongganbo)에 TDA를 적용하여 전통 한국 음악의 위상적 구조를 분석. 초기적 탐구 연구이며, 본 연구가 사용하는 파이프라인의 공통 조상.

이동진, Tran, 정재훈 (2024) — 국악의 기하학적 구조와 AI 작곡. 본 연구의 지도교수 연구실의 직전 연구이며, 본 연구가 계승한 pHcol 알고리즘 구현과 $\rho^* = 0.35$ 휴리스틱의 출처.

본 연구가 이들 대비 새로 기여하는 지점은 다음 다섯 가지이다.

(A) 네 가지 거리 함수의 체계적 비교. 선행 연구들은 frequency 기반 거리만을 사용했으나, 본 연구는 frequency, Tonnetz, voice leading, DFT 네 가지를 동일한 파이프라인 위에서 $N=20$회 반복 실험으로 정량 비교하였다 (§4.1). 이를 통해 "DFT가 frequency 대비 JS divergence를 38.2%, Tonnetz 대비 56.8%, voice leading 대비 62.4% 낮춘다"는 음악이론적 정당성을 실증적으로 제공한다.

(B) Continuous OM의 도입과 검증. 선행 연구들은 이진 OM만을 사용했다. 본 연구는 희귀도 가중치를 적용한 continuous 활성도 개념을 새로 도입했으며 (§2.5), DFT 조건에서는 이진 OM이 최우수임을 통계 실험으로 검증하였다 (§4.4). Algorithm 2 (FC)에서는 continuous OM 직접 입력이 83.9% 개선을 달성한다 (§4.5).

(C) 통계적 엄밀성. 선행 연구들은 단일 run 결과만 보고했지만, 본 연구는 모든 baseline에서 $N=20$회 반복 실행하여 mean ± std, Welch t-test, Cohen's d를 보고한다. 이는 TDA-music 분야에서 "효과가 실재하는가"를 통계적으로 검증한 최초의 사례 중 하나이다.

(D) 서양 현대음악으로의 확장. 기존 연구가 국악을 대상으로 했다면, 본 연구는 서양의 minimalism / ambient 계열 현대음악 (사카모토 out of noise) 으로 적용 범위를 확장했다. 이를 통해 TDA 기반 분석이 **장르 특이적이지 않음**을 보였다.

(E) 곡의 미학적 맥락과 모델 선택의 연결. 본 연구의 §4.5 해석 — FC 모델 우위를 out of noise 앨범의 작곡 철학으로 설명 — 은 기존 TDA-music 연구에 없던 관점이다. "어떤 곡에는 어떤 모델이 맞는가"가 단순히 성능 최적화 문제가 아니라 **미학적 정합성 문제**임을 제시하며, solari 실험 (§5.1)에서 Transformer 최적이라는 반대 패턴으로 이 가설이 실증되었다.

(F) 위상 보존 음악 변주. 기존 TDA-music 연구는 분석(analysis) 또는 재현(reproduction)에 그쳤다. 본 연구는 Tonnetz 최소매칭 거리 기반 note 재분배 + 시간 재배치 + 화성 제약을 결합하여, **위상 구조를 보존하면서 원곡과 다른 음악을 생성**하는 프레임워크를 제시하였다 (§5.3–§5.6). 이는 TDA를 음악 창작의 제약 조건 생성기로 사용한 최초의 시도이다.

7.3 세 줄 요약

> **SHORT 미포함**

1. 본 연구는 단일곡의 위상 구조를 깊이 이해하고 그 구조를 보존한 채 재생성하는 심층 분석 — 재생성 파이프라인이며, 나아가 위상 보존 변주까지 확장한다.
2. 네 가지 거리 함수, 두 가지 overlap 형식, 세 가지 신경망 모델, 통계적 반복, 그리고 세 곡(hibari 7-PC diatonic / solari · aqua 12-PC chromatic)의 대비 검증이 본 연구의 경험적 기여이다.
3. 작곡가의 작업 방식 (§2.9) 과 곡의 미학적 맥락 (§4.5, §5.1) 을 수학적 설계에 직접 반영한 것이 본 연구의 해석적 기여이다.

---

8. 결론

본 연구는 사카모토 류이치의 hibari를 대상으로, persistent homology를 음악 구조 분석의 주된 도구로 사용하는 통합 파이프라인을 구축하였다. 수학적 배경 (§2), 두 가지 생성 알고리즘 (§3), 네 거리 함수 및 continuous overlap의 통계적 비교 (§4)를 일관된 흐름으로 제시하였다.

핵심 경험적 결과:

1. **거리 함수 선택의 효과.** hibari의 Algorithm 1에서 DFT 거리는 frequency 대비 38.2%, Tonnetz 대비 56.8%, voice leading 대비 62.4% 낮은 JS를 달성했다 ($N=20$, §4.1). 반면 곡 일반화에서는 solari가 frequency Tonnetz 동률, Bach Fugue가 Tonnetz 우위, Ravel Pavane이 frequency 우위를 보이며 "곡의 성격이 최적 도구를 결정한다"는 관찰이 유지된다 (§5.1~§5.2).
2. **곡의 성격이 최적 모델을 결정한다.** hibari (diatonic, entropy 0.974)에서는 FC가 최적이고, solari (chromatic)에서는 Transformer가 최적이다. 특히 hibari + gap=0 조건에서 FC-cont는 Transformer-cont 대비 Welch $p=1.66\times10^{-4}$로 유의하게 우수하다 (§5.8.2).
3. **위상 보존 음악 변주.** Tonnetz 최소매칭 기반 note 재분배, 시간 재배치, 화성 제약을 결합하면 원곡과 위상적으로 유사하면서 선율은 유의미하게 다른 변주를 생성할 수 있다 (§5.4~§5.6, major_block32).
4. **본 연구 최저 수치.** Algorithm 1: DFT $\alpha=0.25$ per-cycle τ_c (α-grid 재탐색, §5.8.1 Phase 2)로 **JS** $=0.00902\pm0.00170$ ($N=20$, $p=1.44\times10^{-15}$, $\log 2$의 약 1.30%).[^seed-note2] $\alpha=0.25$가 §5.7 binary

OM과 §5.8.1 per-cycle τ_c 양쪽에서 최적임이 이중 확인된다. Algorithm 2: DFT $\alpha = 0.5$ FC + continuous overlap 직접 입력으로 **JS** $= 0.000348 \pm 0.000149$ ($N = 10$, $\log 2$의 약 0.05%) — FC-bin 대비 $+83.9\%$ 개선 ($p = 1.50 \times 10^{-6}$). Task 51에서 Algo2의 $\alpha = 0.25$ 재실험은 $p = 0.168$ 비유의로, **Algo2 최저는** $\alpha = 0.5$ **유지**. DFT 조건 옥타브 가중치 grid search에서 $w_o = 0.3$이 최적 (§4.1a).

5. **모듈 단위 생성의 full-song 품질 도달.** hibari의 32-timestep 모듈 1개를 생성 후 65회 복제하는 방식(§6)에서, DFT α=0.25 조건 33모듈 탐색의 전역 최저 trial이 JS $= 0.01479$ (§6.7, start=1, seed=9309) 를 달성하였다 — §5.8.1 Phase 1 (α=0.5) 최저 0.01489와 차이 0.00010 (당시 동등); Phase 2 신기록 (α=0.25) 0.00902 대비로는 $+64.0\%$ 열세. P3+C 조합의 우위(best trial 0.0250, §6.5)는 Tonnetz·DFT 양 조건에서 재현되어 distance-invariant하다. 한편 Barcode Wasserstein과 생성 품질의 상관은 Tonnetz($r = 0.503$) vs DFT($r = -0.054$)로 거리 함수에 따라 역전되며, W-based 모듈 선택은 거리 함수가 고정된 조건에서만 유효하다 (§6.8).

6. **거리 함수 최적은 음악적 목적에 따라 달라진다 (메타 통찰).** 구조 정밀도 목적(원곡 재현, cycle 분리, 모듈 탐색)에서는 DFT가 강점을 보였지만(§4, §5.7, §6), scale 제약 변주와 화성 정합성 목적에서는 Tonnetz가 우세했다(§5.6.1). 같은 패턴은 complex 모드에서도 재현되어, Tonnetz에서는 유효하고 DFT에서는 악화된다(§5.9). 즉 단일 거리 함수의 일괄 최적화보다 **목적-거리 정합성 설계**가 핵심이다.

7. **위상 구조를 보존한 음악의 미학적 타당성 (Q4).** 수학적으로 유사한 위상 구조를 가지도록 생성된 음악이 실제 청각적으로도 원곡의 인상을 전달하는가에 대해서는, 본 보고서 말미의 QR 코드로 직접 확인할 수 있다. 체계적 listening test는 향후 과제로 남는다.

8. **Complex 모드는 distance-specific이다.** simul 혼합은 Tonnetz 조건에서는 유효할 수 있으나, DFT + $\alpha = 0.25$에서는 $r_c = 0.1, 0.3$ 모두 timeflow 대비 유의한 성능 악화를 보였다 (각각 Welch $p = 4.74 \times 10^{-39}$, 1.12×10^{-48}, §5.9 Task 34b).

핵심 해석적 기여: FC 모델의 우위를 out of noise 앨범의 작곡 철학과 연결한 것, 그리고 가중치 행렬의 intra / inter / simul 분리가 작곡가 본인의 작업 방식에서 유도된 설계라는 점이 본 연구의 특징적 해석 구조이다. 그림 2.9의 관측 (inst 1 쉼 0개 vs inst 2 쉼 64개) 이 이 설계를 경험적으로 정당화한다.

본 연구는 "단일곡의 위상 구조를 보존한 재생성"에서 출발하여, "위상 구조를 보존한 음악적 변주"까지 확장되었다. 이 확장은 TDA가 음악 분석 도구일 뿐 아니라 **음악 창작의 제약 조건 생성기**로 기능할 수 있음을 시사한다.

후속 연구 과제.

H_2 **(void, 3-simplex) 활용 가능성.** 본 연구는 H_1 (1-차원 cycle)만 활용하였다. `topology.py` (ripser 기반)는 H_2도 계산 가능하나, 모든 실험에서 `listOfDimension=[1]`로 호출되어 H_1 전용으로 운용되었다 (`config.dimensions` 필드는 현재 미사용). H_2 cycle은 3개 이상의 PC가 둘러싸는 '공동(cavity)'에 대응하며, 화성 공간의 고차 조직을 반영할 가능성이 있다. diatonic 곡에서 H_2의 발견 가능성, 그리고 OM 및 생성 알고리즘에 대한 기여 여부는 후속 연구 과제이다.

Transposition-invariance 증명의 확장. 2026-04-16 증명 (T/I 변환 하 $M(f(S)) = M(S) \cdot \Pi$ 열 permutation 관계, $\omega_o = 0$ 또는 $f = T_{12m}$ 조건) 을 (a) PLR (parallel/leading-tone/relative neo-Riemannian) 변환, (b) octave-aware 설정으로 확장.

감쇄 lag × 거리 함수 상호작용의 이론적 설명. §4.1c에서 DFT에서만 감쇄 lag가 개선(-7.1%), Tonnetz에서는 악화($+4.8\%$)임을 경험적으로 관찰했다. DFT의 pitch class 스펙트럼 표현과 시간 lag 상호작용의 정합성에 대한 이론적 분석이 필요하다.

화성 제약 비용함수 grid search. §5.5 휴리스틱 기본값 $(\alpha_{\text{note}}, \beta_{\text{diss}}) = (0.5, 0.3)$에 대한 체계적 grid search.

Per-cycle τ_c + Algorithm 2 통합. per-cycle τ_c로 binarize한 OM을 Algorithm 2 (FC / Transformer)에 입력하는 확장 실험.

청취 실험. 8 stimuli 기반 체계적 evaluation (Task 45/46 인프라 완비).

감사의 글

본 연구는 KIAS 초학제 독립연구단 정재훈 교수님의 지도 아래 진행되었다. pHcol 알고리즘 구현 및 선행 파이프라인의 많은 부분을 계승하였음을 밝힌다. Ripser (Bauer), Tonnetz 이론 (Tymoczko), 그리고 국악 정간보 TDA 연구 (Tran, Park, Jung) 의 수학적 토대 위에 본 연구가 서 있음을 부기한다.

참고문헌

Bauer, U. (2021). "Ripser: efficient computation of Vietoris–Rips persistence barcodes". Journal of Applied and Computational Topology, 5, 391–423.

Carlsson, G. (2009). "Topology and data". Bulletin of the American Mathematical Society, 46(2), 255–308.

Catanzaro, M. J. (2016). "Generalized Tonnetze". arXiv preprint arXiv:1612.03519.

Chuan, C.-H., & Herremans, D. (2018). "Modeling temporal tonal relations in polyphonic music through deep networks with a novel image-based representation". Proceedings of AAAI, 32(1).

cifkao. (2015). tonnetz-viz: Interactive Tonnetz visualization. GitHub. https://github.com/cifkao/tonnetz-viz

Heo, E., Choi, B., & Jung, J.-H. (2025). "Persistent Homology with Path-Representable Distances on Graph Data". arXiv:2501.03553.

Cohen, J. (1988). Statistical Power Analysis for the Behavioral Sciences (2nd ed.). Lawrence Erlbaum.

Cohen-Steiner, D., Edelsbrunner, H., & Harer, J. (2007). "Stability of persistence diagrams". Discrete & Computational Geometry, 37(1), 103–120.

Edelsbrunner, H., & Harer, J. (2010). Computational Topology: An Introduction. AMS.

Hochreiter, S., & Schmidhuber, J. (1997). "Long Short-Term Memory". Neural Computation, 9(8), 1735–1780.

Kingma, D. P., & Ba, J. (2015). "Adam: A method for stochastic optimization". ICLR.

Nemhauser, G. L., Wolsey, L. A., & Fisher, M. L. (1978). "An analysis of approximations for maximizing submodular set functions". Mathematical Programming, 14(1), 265–294.

Nielsen, F. (2019). "On the Jensen–Shannon symmetrization of distances". Entropy, 21(5), 485.

Sakamoto, R. (2009). out of noise [Album]. commmons.

Satterthwaite, F. E. (1946). "An approximate distribution of estimates of variance components". Biometrics Bulletin, 2(6), 110–114.

Tran, M. L., Park, C., & Jung, J.-H. (2021). "Topological Data Analysis of Korean Music in Jeongganbo". arXiv:2103.06620.

Tymoczko, D. (2008). "Set-Class Similarity, Voice Leading, and the Fourier Transform". Journal of Music Theory, 52(2).

Tymoczko, D. (2011). A Geometry of Music: Harmony and Counterpoint in the Extended Common Practice. Oxford University Press.

Tymoczko, D. (2012). "The Generalized Tonnetz". Journal of Music Theory, 56(1).

Vaswani, A., et al. (2017). "Attention is all you need". NeurIPS.

Welch, B. L. (1947). "The generalization of 'Student's' problem when several different population variances are involved". Biometrika, 34(1/2), 28–35.

이동진, Tran, M. L., 정재훈 (2024). "국악의 기하학적 구조와 인공지능 작곡".